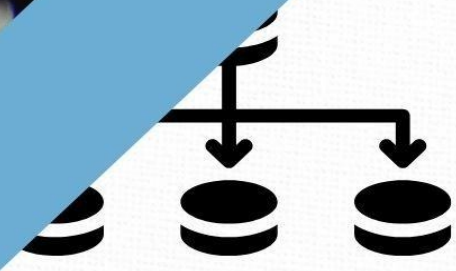


DATA STRUCTURES

- VINAY BHADANE



2024 PATTERN



www.omegaofts.news

Study material provided by: V. B. L

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_FE_ENGG

(for all engineering Resources)

SPPU Engineering & Technical UPDATES

WhatsApp channel

WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b

@SPPU_ENGINEERING_UPDATE

Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

SPPU_FE_ENGG Click to join telegram

INDEX

Chapter 1

Introduction to Data Structures and Algorithms..... 01

Chapter 2

Linear Data Structures, searching and sorting.....21

Chapter 3

Stacks, Queues and Linked Lists 65

Chapter 4

Hashing 107

Chapter 5

Graphs and Trees..... 125

SPPU_FE_ENGG Click to join telegram

UNIT 1 : Introduction to Algorithm and Data Structure

Syllabus

1. **Introduction:** Introduction to Data Structures: Abstract Data Types (ADT), Linear and Non-linear, Static and Dynamic, Persistent and Ephemeral data structures
2. **Algorithms:** Space complexity, Time complexity, Asymptotic notation- Bio-O, Theta and Omega, finding complexity using step count method, Analysis of programming constructs-Linear, Quadratic, Cubic, Logarithmic.
3. **Algorithmic Strategies:** Introduction to algorithm design strategies- Divide and Conquer, and Greedy strategy
4. **Case Study:** E-commerce Product Sorting using Divide and Conquer strategy Google Calendar application using Greedy strategy

1.1) Introduction to Data Structures:

A data structure is a way of organizing and storing data in a computer so that it can be accessed and used efficiently. It refers to the logical or mathematical representation of data, as well as the implementation in a computer program.

Applications of Data Structures :

i. **Databases :** Data structures are used to organize and store data in a database, allowing for efficient retrieval and manipulation.
ii. **Operating systems:** Data structures are used in the design and implementation of operating systems to manage system resources, such as memory and files.
iii. **Computer graphics:** Data structures are used to represent geometric shapes and other graphical elements in computer graphics applications.
iv. **Artificial intelligence:** Data structures are used to represent knowledge and information in artificial intelligence systems.

Advantages of Data Structures :

i. **Efficiency:** Data structures allow for efficient storage and retrieval of data, which is important in applications where performance is critical.
ii. **Flexibility:** Data structures provide a flexible way to organize and store data, allowing for easy modification and manipulation.
iii. **Reusability:** Data structures can be used in multiple programs and applications, reducing the need for redundant code.
iv. **Maintainability:** Well-designed data structures can make programs easier to understand, modify, and maintain over time.

1.1.1) Abstract Data Types (ADT)

An Abstract Data Type (ADT) is a conceptual model that defines a set of operations and behaviours for a data structure, without specifying how these operations are implemented or how data is organized in memory. The definition of ADT only mentions what operations are to be performed but not how these operations will be implemented. It does not specify how data will be organized in memory and what algorithms will be used for implementing the operations. It is called "abstract" because it provides an implementation-independent view.

The process of providing only the essentials and hiding the details is known as abstraction.

1.1.1.1) Features of ADT

Abstract data types (ADTs) are a way of encapsulating data and operations on that data into a single unit. Some of the key features of ADTs include:

i. **Abstraction:** The user does not need to know the implementation of the data structure only essentials are provided.
ii. **Better Conceptualization:** ADT gives us a better conceptualization of the real world.
iii. **Robust:** The program is robust and has the ability to catch errors.
iv. **Encapsulation**: ADTs hide the internal details of the data and provide a public interface for users to interact with the data. This allows for easier maintenance and modification of the data structure.
v. **Data Abstraction**: ADTs provide a level of abstraction from the implementation details of the data. Users only need to know the operations that can be performed on the data, not how those operations are implemented.
vi. **Data Structure Independence**: ADTs can be implemented using different data structures, such as arrays or linked lists, without affecting the functionality of the ADT.
vii. **Information Hiding:** ADTs can protect the integrity of the data by allowing access only to authorized users and operations. This helps prevent errors and misuse of the data.
viii. **Modularity**: ADTs can be combined with other ADTs to form larger, more complex data structures. This allows for greater flexibility and modularity in programming.

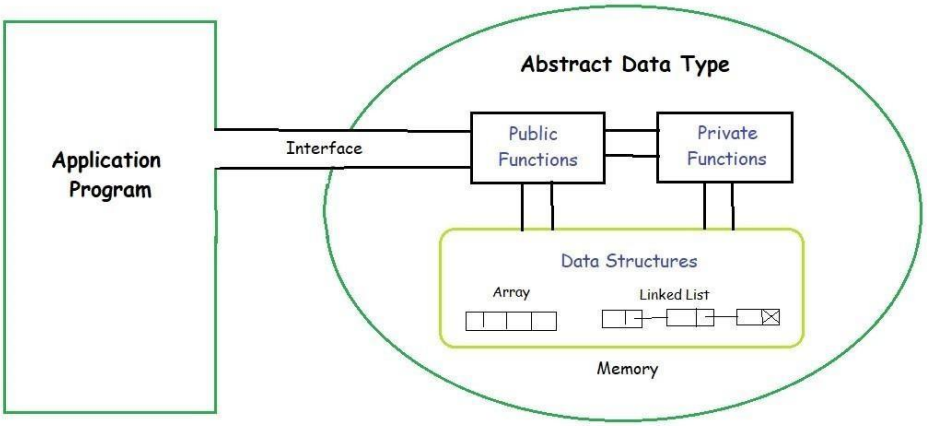
1.1.1.2) Why Use ADTs?

The key reasons to use ADTs in Java are listed below:

i. **Encapsulation:** Hides complex implementation details behind a clean interface.
ii. **Reusability**: Allows different internal implementations (e.g., array or linked list) without changing external usage.
iii. **Modularity:** Simplifies maintenance and updates by separating logic.
iv. **Security:** Protects data by preventing direct access, minimizing bugs and unintended changes.

Example of Abstraction

For example, we use primitive values like int, float, and char with the understanding that these data types can operate and be performed on without any knowledge of their implementation details. ADTs operate similarly by defining **what operations are possible without detailing their implementation.**



1.1 Abstract Data Types

1.1.2) Linear and Non-linear Data Structures

a) Linear Data Structures

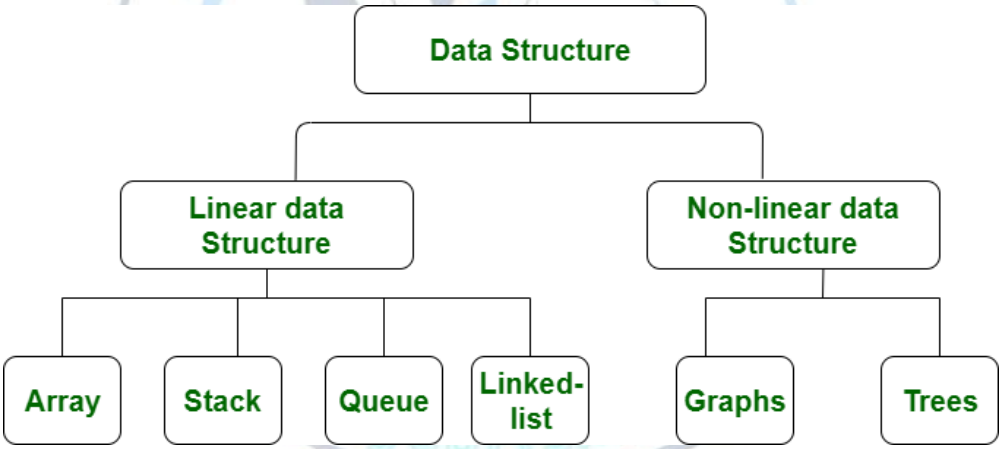
A data structure in which data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements, is called a linear data structure.

- **Analogy:** Think of a **train**. Each coach is connected to the one before it and the one after it, forming a straight line. You can go from one end of the train to the other by moving through each coach one by one.
- **Key Characteristics:**
    a) Elements are in a sequence.
    b) They are straightforward to implement.
    c) You can traverse all elements in a single run.
    d) Memory may not be used very efficiently, as in the case of arrays where you might allocate more space than you need.
- **Examples:**
    a) **Array:** A collection of items stored at contiguous memory locations.
    b) **Linked List:** A series of connected nodes, where each node points to the next.
    c) **Stack:** A "Last-In, First-Out" (LIFO) structure. Think of a stack of plates.
    d) **Queue:** A "First-In, First-Out" (FIFO) structure. Think of a line of people waiting.

b) Non-linear Data Structures

In a non-linear data structure, elements are not arranged in a sequence. Instead, they are organized in a hierarchical or interconnected manner. An element can be connected to one or more other elements.

- **Analogy:** Think of a **family tree** or a **city map**. In a family tree, a parent can have multiple children. In a city map, one location can have roads connecting it to many other locations.
- **Key Characteristics:**
  a) Elements are arranged hierarchically or in a network.
  b) They represent more complex relationships.
  c) Traversal can be more complex and may require multiple runs to visit all elements.
  d) They often use memory more efficiently.
- **Examples:**
  a) **Trees:** Used to represent hierarchical data, like a file system.
  b) **Graphs:** Used to represent networks, like social networks or road maps.



1.2 Linear Data Structure vs Non-Linear Data Structure

1.1.3) Static vs. Dynamic Data Structures

a) Static Data Structures

A static data structure has a **fixed size**. The memory for it is allocated at compile time (before the program runs), and it cannot be changed during runtime.

- **Analogy:** Imagine buying a **bookshelf with a fixed number of shelves**. You know exactly how much space you have. You can't add more shelves later, and if you don't fill it, the extra space is wasted.
- **Key Characteristics:**

a) Size is fixed.
b) Memory is allocated at compile time.
c) Can lead to memory wastage if not all allocated space is used, or a "stack overflow" error if you try to add more data than it can hold.
d) Generally faster access to elements because the memory layout is predictable.

- **Example:**

a) **Array:** When you declare an array in many programming languages (like C++ or Java), you have to specify its size, which cannot be changed.

b) Dynamic Data Structures

A dynamic data structure can **grow or shrink in size** during program execution. Memory is allocated at runtime, as needed.

- **Analogy:** Think of a **Lego collection**. You can start with a few bricks and keep adding more as you build. Your structure can grow as large as you need it to be, and you only use the bricks you need.
- **Key Characteristics:**

a) Size is flexible.
b) Memory is allocated at runtime (from the "heap").
c) More efficient use of memory, as it only uses what it needs.
d) Can be slightly slower due to the overhead of managing memory.

- **Examples:**

a) **Linked List:** You can easily add or remove nodes.
b) **Trees and Graphs:** Can expand as more nodes and edges are added.
c) **Dynamic Arrays (like std::vector in C++ or ArrayList in Java):** These look like arrays but can automatically resize themselves.

1.1.3) Persistent vs. Ephemeral Data Structures

a) Ephemeral Data Structures

"Ephemeral" means temporary or short-lived. An ephemeral data structure is the standard type we use every day. When you modify it, the previous version is **destroyed and replaced** by the new version.

- **Analogy:** Editing a **document without "Undo"**. When you delete a paragraph and type a new one, the old paragraph is gone forever. You only have the current version of the document.
- **Key Characteristics:**

a) Modifications are destructive.
b) Only the current version of the data structure is available.
c) This is the default behavior for most data structures in imperative programming languages.

- **Examples:**
    a) A standard hash table, stack, or queue. When you add an item, the structure changes, and its previous state is lost.

b) Persistent Data Structures

A persistent data structure **preserves the previous version of itself** when it is modified. When you update it, you get a new version, but the old version is still accessible.

- **Analogy:** Think of **version control systems like Git**. When you make changes and "commit" them, you create a new version of your project. However, you can always go back and look at any of the previous versions.
- **Key Characteristics:**
    a) Modifications are non-destructive.
    b) Multiple versions of the data structure coexist.
    c) They are effectively "immutable" (unchangeable).
    d) Very useful in multi-threaded environments and functional programming.
    e) This is achieved through clever techniques like path copying, so you don't have to copy the entire structure every time.
- **Examples:**
    a) Purely functional data structures, like those found in languages like Haskell, are often persistent by nature. For example, when you add an element to a list, you get a new list, but the original list remains unchanged.

a) Difference between Linear vs Non-Linear Data Structures

S.NO	Linear Data Structure	Non-linear Data Structure
1.	In a linear data structure, data elements are arranged in a linear order where each and every element is attached to its previous and next adjacent.	In a non-linear data structure, data elements are attached in hierarchically manner.
2.	In linear data structure, single level is involved.	Whereas in non-linear data structure, multiple levels are involved.
3.	Its implementation is easy in comparison to non-linear data structure.	While its implementation is complex in comparison to linear data structure.

S.NO	Linear Data Structure	Non-linear Data Structure
4.	In a linear data structure, memory is not utilized in an efficient way.	While in a non-linear data structure, memory is utilized in an efficient way.
5.	Its examples are: array, stack, queue, linked list, etc.	While its examples are: trees and graphs.
6.	Applications of linear data structures are mainly in application software development.	Applications of non-linear data structures are in Artificial Intelligence and image processing.
7.	Linear data structures are useful for simple data storage and manipulation.	Non-linear data structures are useful for representing complex relationships and data hierarchies, such as in social networks, file systems, or computer networks.

b) Difference between Static vs. Dynamic Data Structures

Aspect	Static Data Structure	Dynamic Data Structure
Memory allocation	Memory is allocated at compile-time	Memory is allocated at run-time
Size	Size is fixed and cannot be modified	Size can be modified during runtime
Memory utilization	Memory utilization may be inefficient	Memory utilization is efficient as memory can be reused
Access	Access time is faster as it is fixed	Access time may be slower due to indexing and pointer usage
Examples	Arrays, Stacks, Queues, Trees (with fixed size)	Lists, Trees (with variable size), Hash tables

1.2) Algorithms

1.2.1) Space Complexity:

Space complexity in data structures refers to the amount of memory an algorithm uses to solve a problem, including input data, temporary variables, and any auxiliary space, as a function of the input size. It's a measure of how the memory usage grows as the input size increases. Space complexity is typically expressed using Big O notation, similar to time complexity.

Space complexity is like, how much space your program needs in the computer's memory (RAM) when it runs.

Space complexity includes :

1. **Input Space:** Space required to store the input.
2. **Auxiliary Space:** Extra space used by the algorithm (like temporary variables, stacks, etc.)
3. **Function Call Stack:** Space taken by recursive calls or function calls.

1.2.2) Time Complexity:

Time complexity in data structures refers to how the execution time of an algorithm grows as the input increases. It's a crucial concept for evaluating the efficiency of algorithms, particularly when dealing with large datasets. It helps predict how an algorithm will perform with varying input sizes, allowing for the selection of optimal data structures and algorithms for specific tasks.

Why Use Time Complexity?

We use time complexity to:

i. Compare different algorithms.
ii. Choose the most efficient algorithm when dealing with large data.
iii. Predict how a program will perform before running it.

1.2.3) Asymptotic Notation – Big-O, Theta and Omega

Asymptotic notations are mathematical tools used to describe the efficiency of an algorithm in terms of input size. They provide a way to express the growth rate of an algorithm's running time or space requirement as the input size approaches infinity.

These notations help us compare algorithms by focusing only on their dominant behaviour, ignoring constant factors and lower-order terms. The three primary asymptotic notations are:

1.2.3.1) Big-O Notation (O)

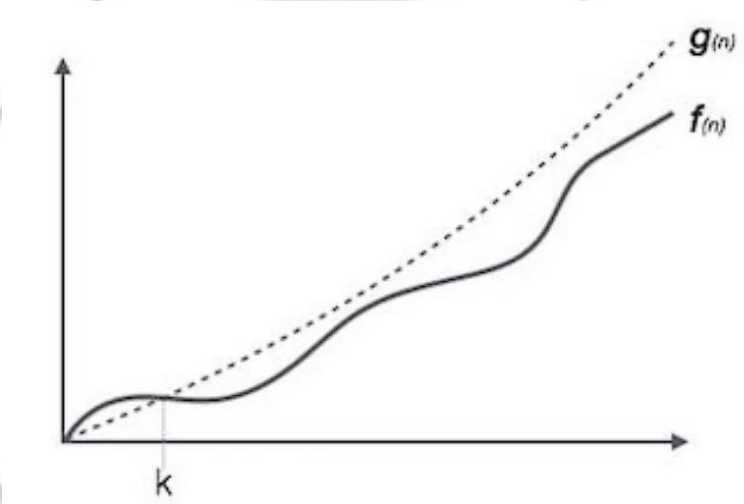
Big-O notation describes the upper bound of an algorithm's growth rate. It represents the worst-case scenario, meaning it defines the maximum time (or space) an algorithm can take to complete, relative to the input size n.

In other words, Big-O gives us a guarantee that the algorithm will not take more time than this bound, even in the worst possible situation.

Mathematically, a function *f(n)* is O(g(n)) if there exist positive constants *c* and n_0 such that:

$$\mathbf{f(n) \leq c \times g(n), \text{ for all } n \geq n_0}$$

Big-O is widely used during algorithm analysis, especially when analyzing performance bottlenecks and scalability.



For example, for a function f(n)=O(g(n))

1.2.3.2) Theta Notation (Θ)

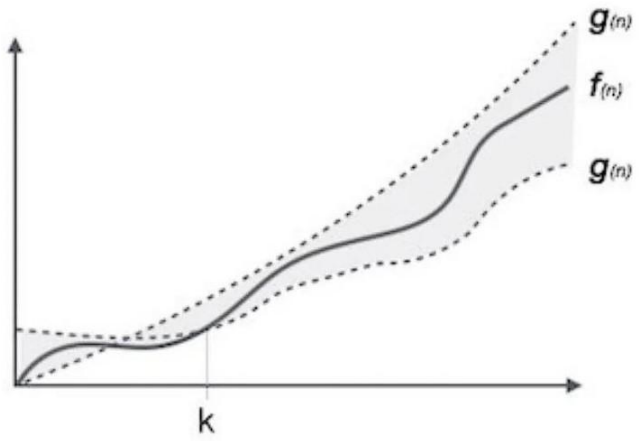
Theta notation defines a tight bound on the growth rate of an algorithm. It captures both the upper and lower bounds, meaning it gives an exact asymptotic behaviour of the algorithm for large inputs.

It indicates that the algorithm will always take a time (or space) proportional to *g(n)*, regardless of variations in input.

Mathematically, a function *f(n)* is Θ(g(n)) if there exist positive constants c_1, c_2, and n_0 such that:

$$\mathbf{c_1 \times g(n) \leq f(n) \leq c_2 \times g(n), \text{ for all } n \geq n_0}$$

Theta notation is used when the performance of an algorithm is consistent and well understood.



For example, for a function f(n)=Θ(g(n))

1.2.3.3) Omega Notation (Ω)

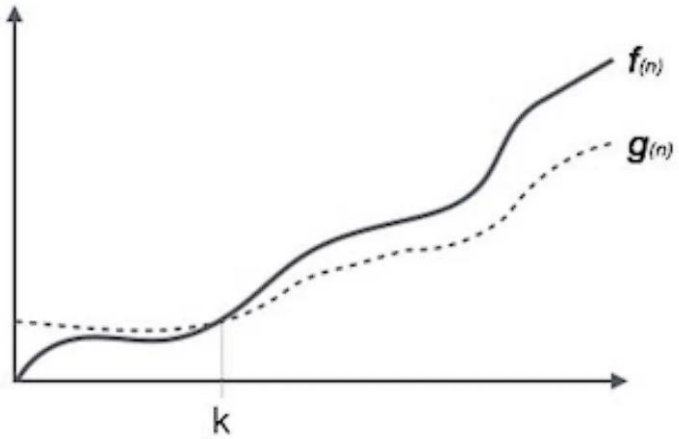
Omega notation provides the lower bound of an algorithm's growth rate. It describes the best-case scenario, indicating the minimum time (or space) the algorithm will take on some input of size n.

It tells us that the algorithm will take at least this much time, though it may take more depending on the input.

Mathematically, a function *f(n)* is Ω(g(n)) if there exist positive constants *c* and n_0 such that:

$$\mathbf{f(n) \geq c \times g(n), \text{ for all } n \geq n_0}$$

Although Omega is less commonly used in practical analysis, it is helpful in proving theoretical lower limits on algorithm performance.



For example, for a function f(n)=Ω(g(n))

Summary of Notations

Notation	Meaning	Describes	Used For
O(g(n))	Upper Bound	Worst-case	Maximum growth rate
Θ(g(n))	Tight Bound	Average/exact-case	Exact growth rate
Ω(g(n))	Lower Bound	Best-case	Minimum growth rate

Asymptotic notations provide a standardized and scalable method to express the performance of algorithms. They allow developers and researchers to abstract away hardware-level differences and focus on how an algorithm behaves as input size grows. Understanding Big-O, Theta, and Omega is essential for analysing, comparing, and selecting the most suitable algorithms for a given problem.

1.2.4) Finding Complexity Using Step Count Method :

The step count method is one of the methods to analyze the Time complexity of an algorithm. In this method, we count the number of times each instruction is executed. Based on that we will calculate the Time Complexity. The step count method is also called as **Frequency Count method.**

The Step Count Method is a technique where we go through each line or block of an algorithm and count how many times it executes, based on the input size.

Example : Analysis of Linear Search algorithm

Let us consider a Linear Search Algorithm.

```
Linearsearch(arr, n, key)
{
    i = 0;
    for(i = 0; i < n; i++)
    {
        if(arr[i] == key)
        {
            printf("Found");
        }
}
```

Where,

- **i = 0**, is an initialization statement and takes O(1) times.

- **for(i = 0;i < n ; i++)**, is a loop and it takes O(n+1) times .
- **if(arr[i] == key)**, is a conditional statement and takes O(1) times.
- **printf("Found")**, is a function and that takes O(0)/O(1) times.

Therefore, Total Number of times it is executed is n + 4 times. As we ignore lower exponents in time complexity total time became O(n).

Time complexity: O(n).

Auxiliary Space: O(1)

Explanation of Step Count Method

In the step count method, we analyze an algorithm by counting the number of basic operations it performs during execution. These basic operations include assignments, comparisons, loops, and function calls.

In the above example of Linear Search, we have broken the algorithm into parts and assigned an estimated number of steps (or time) to each part. The goal is to understand how the total number of steps depends on the input size n, and then represent that using Big-O notation.

What We Did in the Example:

1. **Identified all operations** inside the algorithm:
   - Initialization (i = 0)
   - Loop (for loop runs from 0 to n - 1)
   - Condition check (if(arr[i] == key))
   - Output statement (printf("Found"))
2. **Estimated the time each operation takes:**
   - Initialization takes **O(1)** time (constant time).
   - The for loop runs **(n + 1)** times (including the last failed check), so it takes **O(n + 1)** time.
   - The condition check inside the loop takes **O(1)** time, but since it's inside the loop, it executes **n** times.
   - The printf() may execute **once** or not at all, so we count it as **O(0) or O(1)**.
3. **Added up all these steps:**
   - Total steps ≈ n + 4 (where 4 comes from constant time operations)
4. **Applied asymptotic analysis:**
   - We ignore constants and lower-order terms.
   - So the final time complexity is **O(n)**, which means the algorithm grows linearly with the input size.

Why We Do This:

This method helps us:

- Understand how the algorithm behaves as input size increases.
- Calculate an approximate number of operations.
- Represent the result in Big-O notation, which is standard for measuring algorithm efficiency.

1.2.5) Analysis of programming constructs-Linear, Quadratic, Cubic, Logarithmic.

When analyzing algorithms, it's important to understand how different programming constructs behave in terms of time complexity. The structure of loops and nested blocks in a program directly affects how the running time grows with respect to the input size (n).

Below are the commonly seen patterns: Linear, Quadratic, Cubic, and Logarithmic time complexities.

1. Linear Time – O(n)

In linear time, the number of operations increases directly in proportion to the input size.

- If input size n doubles, the number of steps also doubles.
- This usually occurs when there is a single loop that runs n times.

Example Pattern:

- A loop that runs from 1 to n
- Scanning or searching through an array once

Time Complexity: O(n)

2. Quadratic Time – O(n^2)

Quadratic time occurs when the number of operations grows proportionally to the square of the input size.

- If input size n doubles, the operations increase four times.
- This happens when there are two nested loops, each running n times.

Example Pattern:

- Comparing all pairs of elements in a list
- Simple sorting algorithms like bubble sort, selection sort

Time Complexity: O(n^2)

3. Cubic Time – O(n^3)

Cubic time complexity means the number of operations grows as the cube of the input size.

- If input size n doubles, the steps grow eight times.
- This occurs in programs with three nested loops, each running up to n.

Example Pattern:

- Operations involving 3D data
- Matrix multiplication using three nested loops

Time Complexity: O(n^3)

4. Logarithmic Time – O(log n)

In logarithmic time, the number of operations increases slowly, even if the input size increases rapidly.

- If input size becomes 10 times larger, steps increase only slightly.
- This often occurs when the problem size is divided into halves in each step.

Example Pattern:

- Binary search
- Algorithms that repeatedly divide the problem size (e.g., divide and conquer)

Time Complexity: O(log n)

description	*order of growth*	*example*	*framework*
constant	1	`count++;`	*statement (increment an integer)*
logarithmic	$\log n$	`for (int i = n; i > 0; i /= 2)` `  count++;`	*divide in half (bits in binary representation)*

linear	n	`for (int i = 0; i < n; i++)`
		`  if (a[i] == 0)`
		`  count++;`
			single loop (check each element)

linearithmic	$n \log n$	[*see mergesort* (PROGRAM 4.2.6)]	*divide-and-conquer (mergesort)*

quadratic	n^2	`for (int i = 0; i < n; i++)`
		`  for (int j = i+1; j < n; j++)`
		`  if (a[i] + a[j] == 0)`
		`  count++;`
			double nested loop (check all pairs)

cubic	n^3	`for (int i = 0; i < n; i++)`
		`  for (int j = i+1; j < n; j++)`
		`  for (int k = j+1; k < n; k++)`
		`  if (a[i] + a[j] + a[k] == 0)`
		`  count++;`
			triple nested loop (check all triples)

exponential	2^n	[*see Gray code* (PROGRAM 2.3.3)]	*exhaustive search (check all subsets)*

1. Constant Time – O(1)

- **Example:** count++;
- **What it means:** No matter how big the input is, it always takes the same time.
- **Framework:** A single statement like increasing a number.

2. Logarithmic Time – O(log n)

- **Example:** for (int i = n; i > 0; i /= 2)
- **What it means:** The input size is reduced by half in each step.
- **Framework:** Used in binary search and similar divide-in-half processes.

3. Linear Time – O(n)

- **Example:** for (int i = 0; i < n; i++)
- **What it means:** Time increases directly with input size.
- **Framework:** A single loop that checks or processes each element once.

4. Linearithmic Time – O(n log n)

- **Example:** Merge sort algorithm
- **What it means:** Combines linear and logarithmic behavior.
- **Framework:** Divide-and-conquer strategy where the problem is split and merged.

5. Quadratic Time – $O(n^2)$

- **Example:** Two nested loops for (i) -> for (j)
- **What it means:** Time increases with the square of input size.
- **Framework:** Used when comparing all pairs of elements.

6. Cubic Time – $O(n^3)$

- **Example:** Three nested loops for (i) -> for (j) -> for (k)
- **What it means:** Time increases with the cube of the input size.
- **Framework:** Used when comparing all possible triplets.

7. Exponential Time – $O(2^n)$

- **Example:** Recursive exhaustive search
- **What it means:** Time doubles with each extra input.
- **Framework:** Used in problems that check all subsets or combinations.

1.3) Algorithmic Strategies

1.3.1) Introduction to algorithm design strategies- Divide and Conquer, and Greedy strategy.

Algorithm design strategies are general methods or patterns used to solve problems efficiently. They help in breaking down complex problems and building structured, optimized solutions.

Two widely used design strategies are:

- Divide and Conquer
- Greedy Strategy

1.3.1.1) Divide and Conquer

The Divide and Conquer strategy solves a problem by dividing it into smaller subproblems, solving each subproblem independently, and then combining their results to get the final solution.

This approach works well for problems that can be naturally broken down into smaller instances of the same type.

Steps Involved:

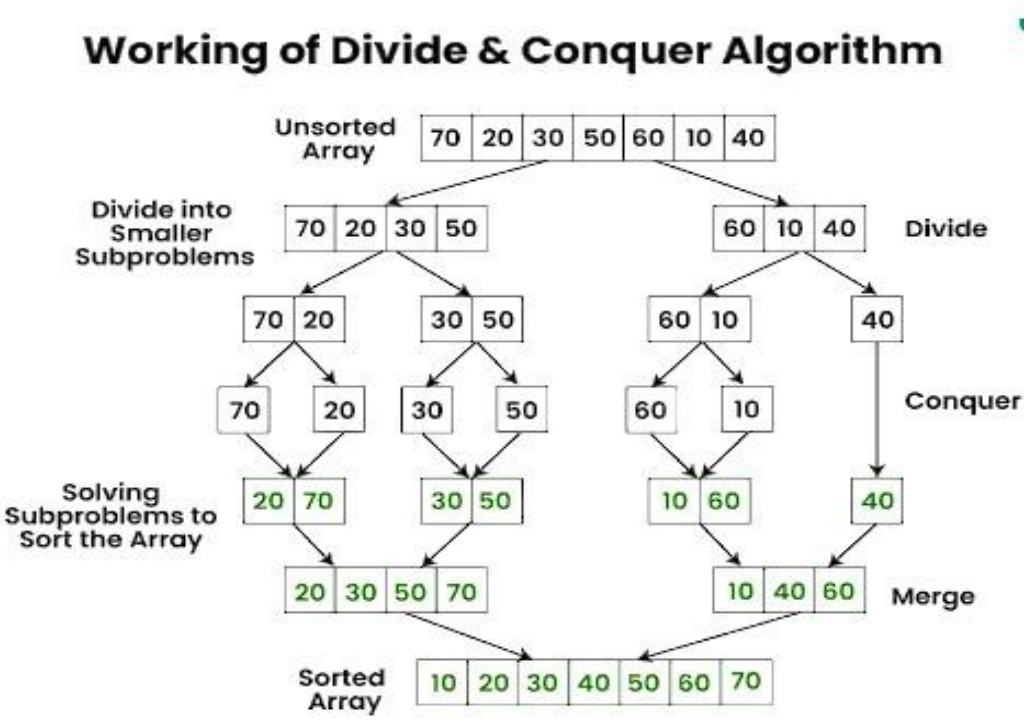
1. **Divide:** Split the problem into smaller subproblems.
2. **Conquer:** Solve each subproblem recursively.
3. **Combine:** Merge the solutions of subproblems to form the solution to the original problem.

Applications:

- Merge Sort
- Quick Sort
- Binary Search
- Matrix Multiplication (Strassen's algorithm)

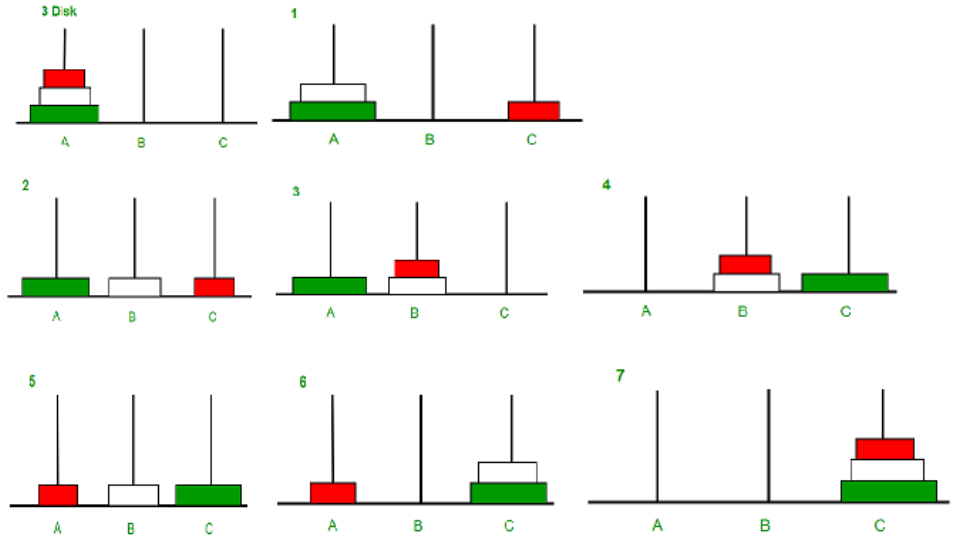
Advantages:

- Reduces time complexity for large problems
- Makes use of recursion and parallelism



Example 1 : Merge Sort is a Divide and Conquer algorithm. It divides input array in two halves, calls itself for the two halves and then merges the two sorted halves. The merge() function is used for merging two halves. The merge(arr, l, m, r) is key process that assumes that arr[l..m] and arr[m+1..r] are sorted and merges the two sorted sub-arrays into one.

Example 2 : Tower of Hanoi is a mathematical puzzle where we have three rods and n disks. The objective of the puzzle is to move the entire stack to another rod, obeying the following simple rules: 1) Only one disk can be moved at a time. 2) Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack i.e. a disk can only be moved if it is the uppermost disk on a stack. 3) No disk may be placed on top of a smaller disk



1.3.1.2) Greedy Strategy

The Greedy strategy builds up a solution step-by-step, always choosing the best option available at the moment, with the hope that this leads to an optimal overall solution.

It does not look ahead or consider future consequences—it only makes locally optimal choices at each step.

Steps Involved:

1. Select the best available choice at the current step.
2. Repeat the process for the remaining problem.
3. Continue until the entire problem is solved.

Applications:

- Activity Selection Problem
- Fractional Knapsack Problem
- Huffman Coding
- Dijkstra's Shortest Path Algorithm

Advantages:

- Simple and efficient for many problems
- Often faster than other approaches

Limitations:

- May not always give the globally optimal solution for all problems
- Works only when greedy choice property and optimal substructure are present

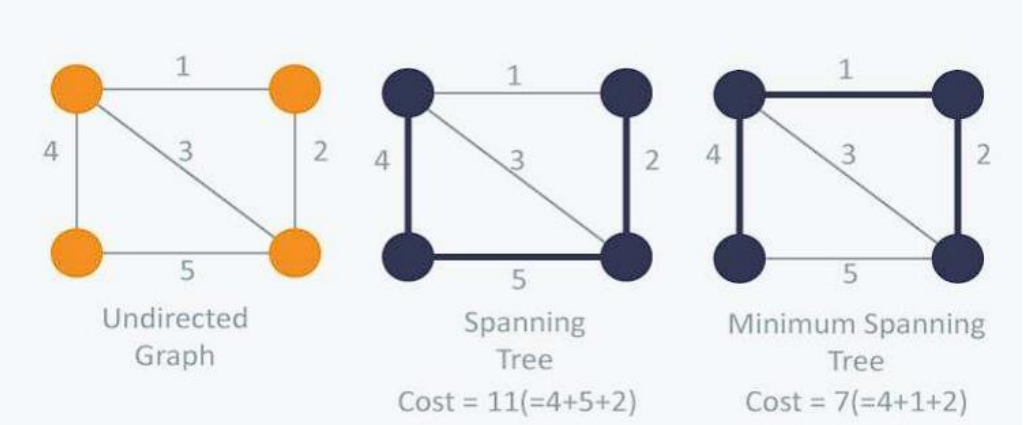
Example of greedy strategy :

1. Travelling Salesman Problem
2. Prim's Minimal Spanning Tree Algorithm
3. Kruskal's Minimal Spanning Tree Algorithm

4. Dijkstra's Minimal Spanning Tree Algorithm
5. Knapsack Problem 6. Job Scheduling Problem

Example 1 : Minimum Spanning tree (Prims or Kruskal's algorithms)

The cost of the spanning tree is the sum of the weights of all the edges in the tree. There can be many spanning trees. Minimum spanning tree is the spanning tree where the cost is minimum among all the spanning trees. There also can be many minimum spanning trees.

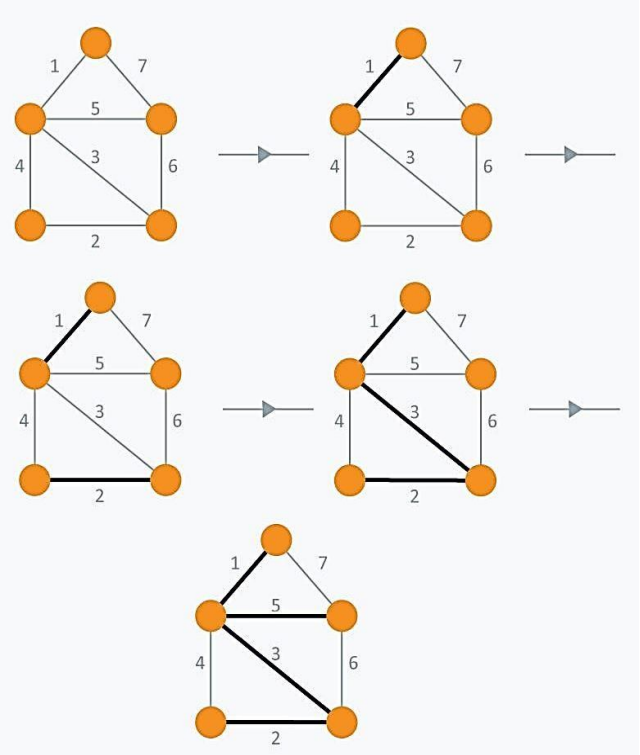


Example 2 : Kruskal's algorithms

Kruskal's Algorithm builds the spanning tree by adding edges one by one into a growing spanning tree. Kruskal's algorithm follows greedy approach as in each iteration it finds an edge which has least weight and add it to the growing spanning tree.

Algorithm Steps :

1. Sort the graph edges with respect to their weights.
2. Start adding edges to the MST from the edge with the smallest weight until the edge of the largest weight.
3. Only add edges which doesn't form a cycle, edges which connect only disconnected components.



1.4) Case Study: E-commerce Product Sorting using Divide and Conquer strategy Google Calendar application using Greedy strategy

1.4.1) Case Study 1: E-commerce Product Sorting using Divide and Conquer Strategy

Problem Context:

In an e-commerce platform such as Amazon or Flipkart, users often sort products by attributes like price, rating, or popularity. When there are thousands of products, the system must sort them quickly and efficiently to enhance user experience.

Why Use Divide and Conquer?

Sorting large datasets is a common problem where the Divide and Conquer strategy is highly effective. The Merge Sort algorithm, for example, applies this approach to achieve efficient sorting.

How Divide and Conquer Works in This Case:

1. **Divide:** The entire product list is divided into smaller sub lists. For example, a list of 10,000 products can be split into halves and then further subdivided.
2. **Conquer:** Each smaller sublist is sorted individually. Sorting is simpler and faster when lists are small.
3. **Combine:** The sorted sub lists are merged to form one complete, sorted list.

Benefits in E-commerce:

- Efficient sorting even for large numbers of products
- Enables sorting by various attributes such as price or rating
- Can be parallelized for improved performance on servers

1.4.2) Case Study 2: Google Calendar Application using Greedy Strategy

Problem Context:
Google Calendar is used to manage time and schedule meetings or events. A common challenge is fitting as many events as possible in a single day without time overlaps.
This is a scheduling problem where we aim to select non-overlapping events that maximize time utilization.

Why Use Greedy Strategy?
The Greedy strategy is suitable because it picks the best option at each step. In this case, it selects the event that finishes earliest, which leaves more time for future events.

How Greedy Strategy Works in This Case:
1. Sort all events by their end times.
2. Select the first event that ends earliest.
3. Skip all events that overlap with the selected one.
4. Continue picking the next non-overlapping event that ends earliest.
5. Repeat until no more events can be added.

Benefits in Google Calendar:
- Maximizes the number of scheduled events
- Avoids time overlaps automatically
- Improves productivity by using available time efficiently

Conclusion:
Both Divide and Conquer and Greedy strategies are widely used in practical applications. Divide and Conquer is used for efficiently solving problems like sorting in e-commerce systems.
Greedy strategy is used for scheduling problems, as seen in applications like Google Calendar. These strategies help in designing efficient and scalable solutions to real-world problems.

UNIT 2 : Linear Data Structures, searching and sorting

Syllabus

5. **Overview of Array:** Array as an Abstract Data Type, Operations on Array, Storage Representation, Multidimensional Arrays[2D, nD], Sparse matrix representation using 2D searching: Sequential Search/Linear Search, Binary Search, Fibonacci Search and Indexed Sequential Search.
6. **Sorting:** Concepts- Stability, Efficiency, and Number of Passes, Internal and External Sorting, Bubble sort, Insertion Sort, Selection Sort, Quick Sort, Merge sort.
7. **Case Study:** Social Network Adjacency Matrix Representing friendship connections among millions of users.

2.1) Overview of Array:

An array is a fundamental linear data structure that stores a collection of elements, all of the same data type, in contiguous memory locations. These elements can be accessed directly using their index, which represents their position within the array.

Characteristics of an Array:

i. **Homogeneous Elements :** Arrays store elements of the same data type (e.g., all integers, all characters, etc.).
ii. **Contiguous Memory :** Elements are stored in consecutive memory locations, making it efficient to access them.
iii. **Indexed Access :** Each element in an array is associated with a unique index, starting from 0 for the first element.
iv. **Fixed Size:** In many programming languages, arrays have a fixed size that is determined at the time of declaration and cannot be changed during program execution.

How Arrays Work:

i. **Declaration:** When an array is declared, memory is allocated for a specific number of elements.
ii. **Storage:** The elements are stored one after another in memory.
iii. **Access:** To access an element, you use its index within square brackets []. For example, array would access the third element (index 2) of the array.

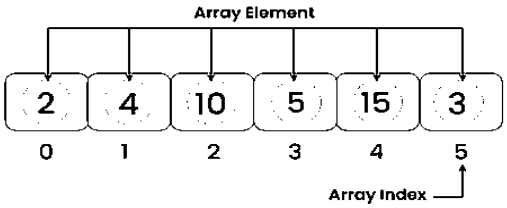
Example :

```
nums = [2, 4, 10, 5, 15, 3]

# Access the third element (index 2)

print("The third number is:", nums[2])
```

Array
Data Structure



Advantages of Arrays :

i. **Efficient Element Access (Indexing):** Allows direct access to any element using its index in constant time O(1).
ii. **Memory Locality:** Stored in contiguous memory blocks, enhancing performance due to CPU caching.
iii. **Easy Iteration:** Elements can be easily traversed using loops without extra structure.
iv. **Simple to Implement:** Arrays are easy to use and supported in almost all programming languages.
v. **Supports Efficient Sorting/Search (for static data):** Algorithms like binary search work efficiently on sorted arrays.
vi. **Low Overhead (Static Arrays):** Uses less memory than structures like linked lists as no extra pointers are needed.

Disadvantages of Arrays :

i. **Fixed Size (Static Arrays):** Size must be defined initially and cannot be changed later.
ii. **Costly Insertions and Deletions:** Inserting or deleting elements in the middle of an array can be inefficient as it may require shifting other elements.
iii. **Wasted or Insufficient Memory:** Allocating too much or too little space can lead to inefficiency or overflow.
iv. **Homogeneous Data:** Arrays typically store elements of the same data type only.
v. **Inefficient for Dynamic Data Needs:** Growing arrays dynamically may involve copying and reallocating memory.
vi. **No Built-in Flexibility (in low-level languages):** Lacks built-in methods for operations like append or pop without extra code.

2.1.1) Array as an Abstract Data Type :

An Abstract Data Type (ADT) is a theoretical concept that defines a data structure by its behaviour from the point of view of a user. It focuses on what operations can be performed on the data and how the data behaves, rather than how it is implemented in code.

An Array is one of the most fundamental and widely used ADTs in computer science. It provides a way to store a fixed-size sequence of elements of the same type, accessible by a numeric index.

Definition of Array as an ADT

As an abstract data type, an **Array** is defined by the following:

- **Data**: A collection of elements, all of the same type (e.g., integers, strings).
- **Operations**: A set of operations that can be performed on the array, such as:

  i. Access an element at a given index
  ii. Insert an element at a specific position
  iii. Delete an element from a position
  iv. Traverse all elements
  v. Update the value at an index

Basic Characteristics of Array ADT

1. **Index-based access**: Each element in an array can be accessed directly using an index.
2. **Fixed size**: The size of the array is usually set at the time of creation and cannot be changed.
3. **Homogeneous elements**: All elements in the array are of the same data type.
4. **Contiguous memory**: In most implementations (including low-level languages), arrays are stored in contiguous memory blocks.

An Array as an Abstract Data Type provides a logical model for organizing and accessing a sequence of data elements using indices. It hides the complexity of memory management and focuses on what operations can be done. Understanding array ADT is essential as it forms the basis for many other data structures such as stacks, queues, and matrices.

2.1.2) Operations on Array

Arrays are one of the most fundamental and widely used data structures. Once an array is created, various operations can be performed on it to manage or manipulate data. These operations allow us to **access**, **insert**, **delete**, **update**, and **traverse** array elements.

1. Traversal

Definition:
Traversal refers to the process of visiting each element of the array **one by one**, usually to display the elements or perform operations on them.

Purpose:

- Printing all elements
- Searching for a value
- Performing a calculation (e.g., sum, average)

Time Complexity: O(n) (each element must be visited once)

2. Insertion

Definition:
Insertion means adding a new element to the array at a specific position.

Types of Insertion:

- **At the end:** Easiest type; done in constant time if space is available.
- **At a specific index:** Requires shifting elements to the right.
- **At the beginning:** Requires shifting all existing elements one position to the right.

Steps for Insertion at Position pos:

1. Shift elements from position pos to the right by one index.
2. Insert the new value at index pos.

Time Complexity:

- Best case (insert at end): O(1)
- Worst case (insert at beginning): O(n)

3. Deletion

Definition:
Deletion refers to removing an element from a specific position in the array.

Steps for Deletion at Position pos:

1. Remove the element at index pos.
2. Shift all elements from pos + 1 to the left by one index.

Time Complexity:

- Best case (delete last element): O(1)
- Worst case (delete first element): O(n)

4. Searching

Definition:
Searching is the process of finding the position (index) of a specific element in the array.

Types of Searching:

- **Linear Search:** Checks each element one by one.
- **Binary Search:** Used only for sorted arrays; divides the array into halves to find the target.

Time Complexity:

- Linear Search: O(n)
- Binary Search: O(log n)

5. Updating

Definition:
Updating means changing the value of an element at a specific index in the array.

Steps:

- Directly access the index and assign a new value.

Time Complexity: O(1) (Array allows direct access by index)

6. Merging

Definition:
Merging is the process of combining two arrays into a single array.

Steps:

- Create a new array with enough space to hold both arrays.
- Copy elements of both arrays into the new array.

Time Complexity: O(n + m) (where n and m are the sizes of the two arrays)

2.1.3) Storage Representation

An array is a linear data structure that stores elements contiguously in memory. This means all elements of the array are kept next to each other in memory locations. Because of this feature, arrays allow fast access to any element using its index.

To understand how arrays work internally, it is important to know how data is stored in memory and how indexing is calculated.

Memory Representation of Array

Let's say we have an array:

arr = [10, 20, 30, 40]

If this array is stored starting at memory address 1000 and each integer takes **4 bytes**, then the memory will look like this:

Index	Value	Address
0	10	1000
1	20	1004
2	30	1008
3	40	1012

As shown above, each element is stored at a memory address that is **offset** from the base address by a fixed number of bytes (depending on data type size).

Base Address and Indexing

If:

- BaseAddress is the starting address of the array
- i is the index of the element
- w is the size (in bytes) of each element

Then, the memory address of element at index i is calculated using the formula:

Address of arr[i] = BaseAddress + (i × w)

Example:
For arr[2] with base address = 1000 and size = 4 bytes:

Address = 1000 + (2 × 4) = 1008

This formula allows the computer to directly compute the address of any element in constant time, making access **O(1)**.

Storage in One-Dimensional Array

In a **1D array**, elements are stored in a single row of contiguous memory locations.

Example:

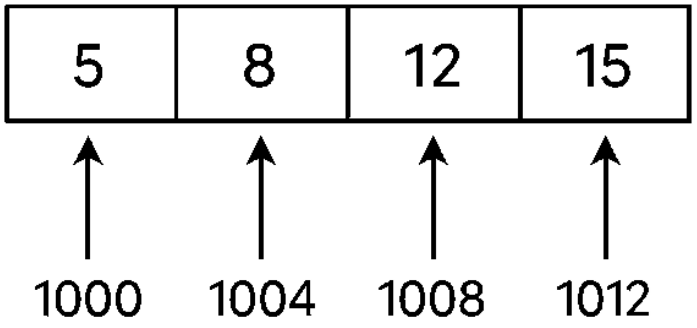
arr = [5, 8, 12, 15]

Memory Representation:

[5] [8] [12] [15]

↑ ↑ ↑ ↑

1000 1004 1008 1012 (assuming base = 1000 and int = 4 bytes)



Storage in Two-Dimensional Array

A **2D array** is stored in memory in one of two ways:

- **Row-Major Order** (used in C/C++, Python): Rows are stored one after another.
- **Column-Major Order** (used in Fortran): Columns are stored one after another.

Row-Major Example:

arr = [

[1, 2],

[3, 4]

]

Memory layout in row-major:

[1] [2] [3] [4]

If base address = 1000 and size = 4 bytes:

Index (0,0): 1000

Index (0,1): 1004

Index (1,0): 1008

Index (1,1): 1012

Row-Major Address Formula:

For 2D array arr[i][j], address =

BaseAddress + [(i × total_columns) + j] × w

Advantages of Contiguous Storage:

- **Fast indexing** using a simple formula
- **Efficient memory use** with no gaps
- **Better cache performance** in modern processors

Limitations of Array Storage:

- **Fixed size**: Size must be known in advance
- **Costly insertion/deletion**: Shifting of elements may be needed
- **Wasted memory**: If array is not completely filled

2.1.4) Multidimensional Arrays [2D, nD]

A **multidimensional array** is an array of arrays. Unlike a one-dimensional array that stores data in a linear form, a multidimensional array stores data in a **tabular or matrix form** (2D), or in **higher dimensions** (nD) to represent more complex structures.

The most commonly used multidimensional arrays are:

- **2D arrays**: Used to represent tables or matrices
- **nD arrays (3D, 4D, etc.)**: Used for representing more complex data like tensors, images, or scientific computations

1) Two-Dimensional Array (2D Array)

Definition:

A 2D array is a collection of data elements arranged in **rows and columns**.

Syntax (Python-style):

```
arr = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
```

This is a **3x3 matrix** (3 rows × 3 columns).

Accessing Elements:

Elements are accessed using two indices:

```
print(arr[1][2])  # Output: 6 (second row, third column)
```

2) n-Dimensional Array (nD Array)

Definition:

An nD array is an array with **more than two dimensions**, used to represent complex datasets like 3D models, color images (3D: height × width × channels), or scientific grids.

Example (3D array in Python using NumPy):

```
import numpy as np
arr = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
```

])

This is a **2x2x2 array**.

Accessing Elements:

print(arr[1][0][1]) # Output: 6

3) Storage in Memory

Multidimensional arrays are stored in memory as a **contiguous block** in either:

- **Row-major order**: Rows are stored one after another (used in C, Python).
- **Column-major order**: Columns are stored one after another (used in Fortran, MATLAB).

Memory Address Calculation (for 2D array in row-major):

Address = BaseAddress + [i × total_columns + j] × w

Where:

- i = row index
- j = column index
- w = size of one element in bytes

4) Advantages of Multidimensional Arrays

1. **Organized Data Representation**: Natural way to represent tables, grids, images, etc.
2. **Fast Access**: Direct access using index positions.
3. **Efficient Memory Usage**: Stored in contiguous memory locations.
4. **Simplicity**: Easy to implement matrices and perform mathematical operations.
5. **Support for Matrix-Based Algorithms**: Ideal for scientific, numerical, and image-based applications.

5) Disadvantages of Multidimensional Arrays

1. **Fixed Size**: Cannot dynamically resize like lists or linked structures.
2. **Memory Intensive**: Requires large memory blocks for higher dimensions.
3. **Complex Access Patterns**: Higher dimensions can be difficult to manage and visualize.
4. **Limited Flexibility**: Insertions and deletions are complex and costly (require shifting data).

Applications of Multidimensional Arrays

Use Case	Description
Matrices in Mathematics	Representation of matrix operations, linear algebra
Image Processing	2D arrays for grayscale, 3D arrays for color images
Scientific Computing	Multi-dimensional grids for simulations and models
Game Development	Board games, 3D object representations
Database Tables	Rows and columns structure stored in memory
Neural Networks / ML	Tensors in deep learning frameworks (NumPy, TensorFlow)
Geographic Mapping	3D terrain maps or spatial models

Multidimensional arrays are powerful tools for handling structured and complex data. While 2D arrays are ideal for representing tables and matrices, nD arrays are useful in advanced domains like image processing, scientific computing, and machine learning. Despite some memory and complexity issues, their speed and organization make them essential in many technical fields.

2.1.5) Sparse matrix representation using 2D Searching: Sequential Search/Linear Search, Binary Search, Fibonacci Search, and Indexed Sequential Search.

1) Sparse Matrix Representation with 2D Searching

Imagine you have a giant grid of numbers, like a spreadsheet. Now, imagine most of the cells in this grid are empty, or contain the number zero. If you store every single one of these zeros, you're wasting a lot of computer memory! This type of grid, where most of the elements are zero, is called a **Sparse Matrix**.

Think of it like this: If you have a phone book with 1,000 pages, but only 10 people actually have phone numbers, would you write down all 1,000 pages, marking 990 of them "empty"? Or would you just list the 10 people who actually have numbers? You'd choose the latter to save space!

Similarly, in computer science, we have special ways to store sparse matrices efficiently, so we only store the non-zero elements. This saves a lot of memory and can make operations much faster.

2) Why Not Just a Regular 2D Array?

A regular 2D array (like matrix[row][column]) would store every single element, including all the zeros. For a large sparse matrix, this is extremely inefficient.

Example:

Imagine a 1000x1000 matrix. That's 1,000,000 cells! If only 100 of them are non-zero, storing all 1,000,000 cells is a huge waste.

3) How Do We Represent Sparse Matrices?

Instead of storing the zeros, we only store the non-zero elements along with their location (row and column). There are a few common ways to do this, but for simplicity, let's consider representing it as a list of "triplets," where each triplet is (row, column, value).

Example:

Let's say we have a small 3x3 sparse matrix:

0 0 5

1 0 0

0 0 8

Instead of storing all zeros, we can represent it as:

[(0, 2, 5), (1, 0, 1), (2, 2, 8)]

Each (row, column, value) tells us exactly where a non-zero element is and what its value is.

4) 2D Searching in Sparse Matrix Representations

Now that we have our sparse matrix represented efficiently, what if we want to find a specific element, or check if an element exists at a particular (row, column)? This is where 2D searching comes in. We're looking for a specific (row, column) pair within our list of non-zero elements.

Let's explore different search techniques:

1. Sequential Search (Linear Search)

The Linear Search algorithm searches through an array and returns the index of the value it searches for.

How it works ?

1. Go through the array value by value from the start.
2. Compare each value to check if it is equal to the value we are looking for.
3. If the value is found, return the index of that value.
4. If the end of the array is reached and the value is not found, return -1 to indicate that the value was not found.

Example :

Let's try to do the searching manually, just to get an even better understanding of how Linear Search works before actually implementing it in a programming language. We will search for value 11.

Step 1: We start with an array of random values.
[12, 8, 9, 11, 5, 11]

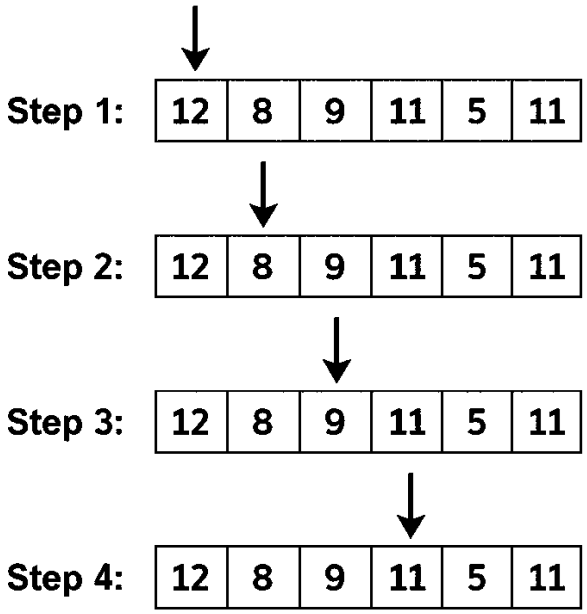
Step 2: We look at the first value in the array, is it equal to 11?
[12, 8, 9, 11, 5, 11]

Step 3: We move on to the next value at index 1, and compare it to 11 to see if it is equal.
[12, 8, 9, 11, 5, 11]

Step 4: We check the next value at index 2.
[12, 8, 9, 11, 5, 11]

Step 5: We move on to the next value at index 3. Is it equal to 11?
[12, 8, 9, 11, 5, 11]

We have found it!
Value 11 is found at index 3.
Returning index position 3.
Linear Search is finished.



Value 11 is found at index 3.
Returning index position 3.
Linear Search is finished.

Time and Space Complexity of Linear Search Algorithm:

Time Complexity:

- **Best Case:** In the best case, the key might be present at the first index. So the best case complexity is O(1)

- **Worst Case:** In the worst case, the key might be present at the last index i.e., opposite to the end from which the search has started in the list. So the worst-case complexity is O(N) where N is the size of the list.
- **Average Case:** O(N)

Applications of Linear Search Algorithm:

- **Unsorted Lists:** When we have an unsorted array or list, linear search is most commonly used to find any element in the collection.
- **Small Data Sets:** Linear Search is preferred over binary search when we have small data sets with
- **Searching Linked Lists:** In linked list implementations, linear search is commonly used to find elements within the list. Each node is checked sequentially until the desired element is found.
- **Simple Implementation:** Linear Search is much easier to understand and implement as compared to Binary Search or Ternary Search.

Disadvantages of Linear Search Algorithm:

- Linear search has a time complexity of O(N), which in turn makes it slow for large datasets.
- Not suitable for large arrays.

When to use Linear Search Algorithm?

- When we are dealing with a small dataset.
- When you are searching for a dataset stored in contiguous memory.

Example :

```
def linear_search(arr, target):
    print("Step-by-step Linear Search:")
    for i in range(len(arr)):
        print(f"Step {i + 1}: Looking at index {i}, value = {arr[i]}")
        if arr[i] == target:
            print(f"\nValue {target} is found at index {i}.")
            print(f"Returning index position {i}.")
            print("Linear Search is finished.")
            return i
    print(f"\nValue {target} is not found in the array.")
    return -1

arr = [12, 8, 9, 11, 5, 11]
target = 11

linear_search(arr, target)
```

2. Binary Search

The Binary Search algorithm is a searching technique used on sorted arrays. It repeatedly divides the search space in half and compares the target value with the middle element of the array. Binary search is faster than linear search for large, sorted datasets.

How it works?

1. Make sure the array is sorted in either ascending or descending order.
2. Find the middle index of the current search space.
3. Compare the target value with the middle element:
    - If they are equal, the search is successful. Return the index.
    - If the target is smaller than the middle element, repeat the search in the **left** half.
    - If the target is larger than the middle element, repeat the search in the **right** half.
4. Repeat the process until the element is found or the search space is empty.

Example:

Let's understand Binary Search manually before implementing it. We will search for value 11 in the sorted array:

Step 1: We start with a sorted array:

[5, 8, 9, 11, 11, 12]

Step 2: Find the middle index.

Middle index = (0 + 5) // 2 = 2
Value at index 2 = 9
Is 9 equal to 11? No.

Step 3: Since 11 > 9, search the right half.

New search range = index 3 to 5 → [11, 11, 12]

Step 4: Find the new middle index.

Middle index = (3 + 5) // 2 = 4
Value at index 4 = 11
Is 11 equal to 11? Yes.
We have found it!
Value 11 is found at index 4.
Returning index position 4.
Binary Search is finished.

Time and Space Complexity of Binary Search Algorithm:

Time Complexity:

- **Best Case:** O(1) → When the middle element is the target.
- **Worst Case:** O(log N) → Each step reduces the search space by half.
- **Average Case:** O(log N)

Space Complexity:

- O(1) for iterative implementation
- O(log N) for recursive implementation (due to call stack)

Applications of Binary Search Algorithm:

- **Large Sorted Arrays:** Efficient for searching in sorted arrays.
- **Search in Databases:** Useful in indexing systems and lookup tables.
- **Game Development:** Used in various decision-making or range-query operations.
- **Dictionary Lookup:** Quickly find words in sorted word lists.

Disadvantages of Binary Search Algorithm:

- Cannot be used on unsorted data.
- Slightly complex to implement compared to linear search.
- In dynamic data (like streaming inputs), maintaining sorted order can be costly.

When to use Binary Search Algorithm?

- When you have a large dataset that is already sorted.
- When performance is critical (faster than linear search).
- When you want efficient log-scale searching.

Example:

```
def binary_search(arr, target):
    print("Step-by-step Binary Search:")
    low = 0
    high = len(arr) - 1
    step = 1

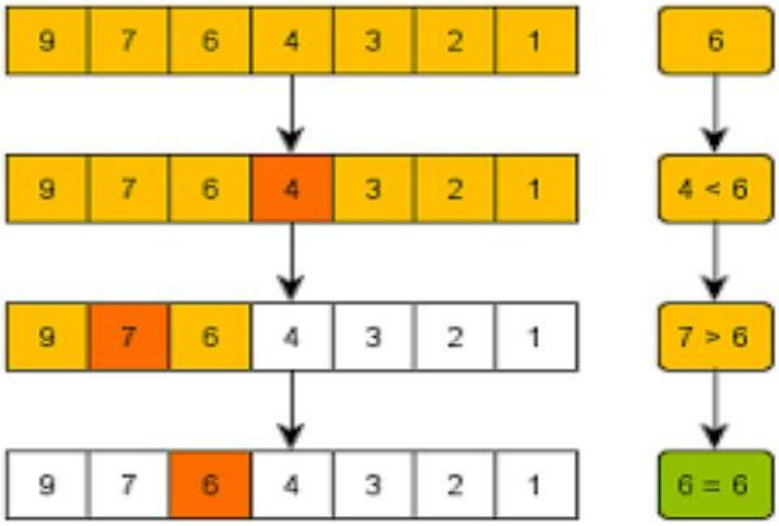
    while low <= high:
        mid = (low + high) // 2
        print(f"Step {step}: Searching in range index {low} to {high}, middle = {mid}, value =
{arr[mid]}")
        step += 1

        if arr[mid] == target:
            print(f"\nValue {target} is found at index {mid}.")
            print(f"Returning index position {mid}.")
            print("Binary Search is finished.")
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1

    print(f"\nValue {target} is not found in the array.")
    return -1

# Example usage:
arr = [5, 8, 9, 11, 11, 12]  # Sorted array
target = 11

binary_search(arr, target)
```



Working of Binary Search

3. Fibonacci Search

Fibonacci Search is an efficient searching algorithm similar to Binary Search, but it divides the array using Fibonacci numbers instead of the middle element.

How it works

1. Fibonacci Search begins by identifying the smallest Fibonacci number that is greater than or equal to the size of the array.
2. It uses this number to split the array into sections.
3. At each step, it compares the target value to the element located at the current Fibonacci index.
4. Based on the comparison, it reduces the search space using smaller Fibonacci numbers and moves accordingly.
5. It continues this process until the element is found or the search space is exhausted.

Fibonacci Search avoids division operations and uses only addition and subtraction, making it suitable for systems where division is expensive.

Step-by-step Explanation with Manual Trace

Let's manually try Fibonacci Search with a sorted array:
Array:
[5, 8, 9, 11, 12, 13, 17, 20]
We are searching for value: 12

Step 1

Length of array = 8
Find the smallest Fibonacci number ≥ 8.
Fibonacci numbers: 0, 1, 1, 2, 3, 5, 8, 13 → Choose 8 (FibM = 8)
Initialize:

- offset = -1
- fibM = 8, fibM1 = 5, fibM2 = 3

Calculate index:
i = min(offset + fibM2, n-1) = min(-1 + 3, 7) = 2
Compare arr[2] = 9 with 12
→ Since 9 < 12, move the search window to the right
Update:

- fibM = 5, fibM1 = 3, fibM2 = 2
- offset = 2

Step 2
Now check:
i = min(offset + fibM2, n-1) = min(2 + 2, 7) = 4
Compare arr[4] = 12 with 12
→ Match found
Return index 4.
Fibonacci Search is finished.

Final Output
Value 12 is found at index 4.
Returning index position 4.

Time and Space Complexity

- Best Case: O(1)
- Average Case: O(log N)
- Worst Case: O(log N)
- Space Complexity: O(1)

Applications of Fibonacci Search

- Efficient on sorted arrays
- Useful for large datasets when division operations are expensive
- Can be applied in memory-limited systems where only sequential access is allowed

Disadvantages

- Requires the array to be sorted
- Slightly more complex than binary search
- Less commonly used in practical applications compared to binary search

Python Example Code

```
def fibonacci_search(arr, target):
    n = len(arr)
    # Initialize Fibonacci numbers
    fibM2 = 0
    fibM1 = 1
    fibM = fibM2 + fibM1

    # Find the smallest Fibonacci number greater than or equal to n
    while fibM < n:
        fibM2 = fibM1
```

```
        fibM1 = fibM
        fibM = fibM2 + fibM1

    offset = -1

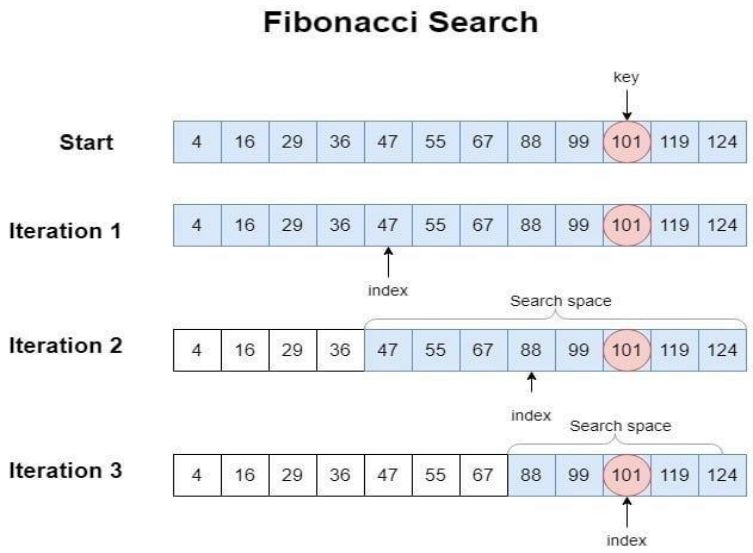
    while fibM > 1:
        i = min(offset + fibM2, n - 1)

        if arr[i] < target:
            fibM = fibM1
            fibM1 = fibM2
            fibM2 = fibM - fibM1
            offset = i
        elif arr[i] > target:
            fibM = fibM2
            fibM1 = fibM1 - fibM2
            fibM2 = fibM - fibM1
        else:
            print(f"\nValue {target} is found at index {i}.")
            print(f"Returning index position {i}.")
            print("Fibonacci Search is finished.")
            return i

    # Check if the last element is the target
    if fibM1 and offset + 1 < n and arr[offset + 1] == target:
        print(f"\nValue {target} is found at index {offset + 1}.")
        print(f"Returning index position {offset + 1}.")
        print("Fibonacci Search is finished.")
        return offset + 1

    print(f"\nValue {target} is not found in the array.")
    return -1

# Example usage:
arr = [5, 8, 9, 11, 12, 13, 17, 20]
target = 12
fibonacci_search(arr, target)
```



4. Indexed Sequential Search

Indexed Sequential Search is a hybrid searching technique that combines the simplicity of Sequential Search and the efficiency of Binary Search by using an **index table** to divide the dataset into blocks.

How it works

1. **Create an index table** where each entry points to the first element of a block in the sorted data.
2. Perform a **sequential search** on this index to locate the appropriate block where the target element might exist.
3. Once the correct block is found, perform a **linear search** within that block to locate the target value.

This method is especially useful when the data is **large and sorted**, and it's expensive to search the entire dataset sequentially.

Step-by-step Explanation with Manual Trace

Consider the following sorted array:

[5, 8, 9, 11, 12, 13, 17, 20]

We divide it into blocks of size 3 (for simplicity), and create an index table:

Index Table (Block Start Elements):

- Index[0] → Value = 5 (points to arr[0])
- Index[1] → Value = 11 (points to arr[3])
- Index[2] → Value = 17 (points to arr[6])

We are searching for value: 12

Step 1: Search the Index Table

Go through the index entries one by one:

- Index[0]: 5 → target 12 > 5 → check next
- Index[1]: 11 → target 12 > 11 → check next
- Index[2]: 17 → target 12 < 17 → Target must be in the previous block (block starting at index 3)

So, the block where the value may exist is [11, 12, 13] (starting at index 3 and ending at index 5)

Step 2: Sequential Search within the Block

Now search the block linearly:

- arr[3] = 11 → not equal
- arr[4] = 12 → match found

Return index 4.
Indexed Sequential Search is finished.

Final Output

Value 12 is found at index 4.
Returning index position 4.

Time and Space Complexity

- **Best Case:** O(1)
- **Average Case:** O(√N)
- **Worst Case:** O(N)
- **Space Complexity:** O(√N) (for storing the index)

Applications of Indexed Sequential Search

- When data is **sorted** and **very large**
- Suitable for **file systems**, **databases**, and **disk-based structures**
- Works well with **secondary storage** (like hard drives) where block-level access is more efficient than random access

Disadvantages

- Requires **additional memory** for the index
- Index must be **maintained** when inserting or deleting elements
- Slightly more complex to implement than linear search
- Only works with sorted data

Example Code

```
def indexed_sequential_search(arr, target, block_size=3):
    n = len(arr)
    index = []

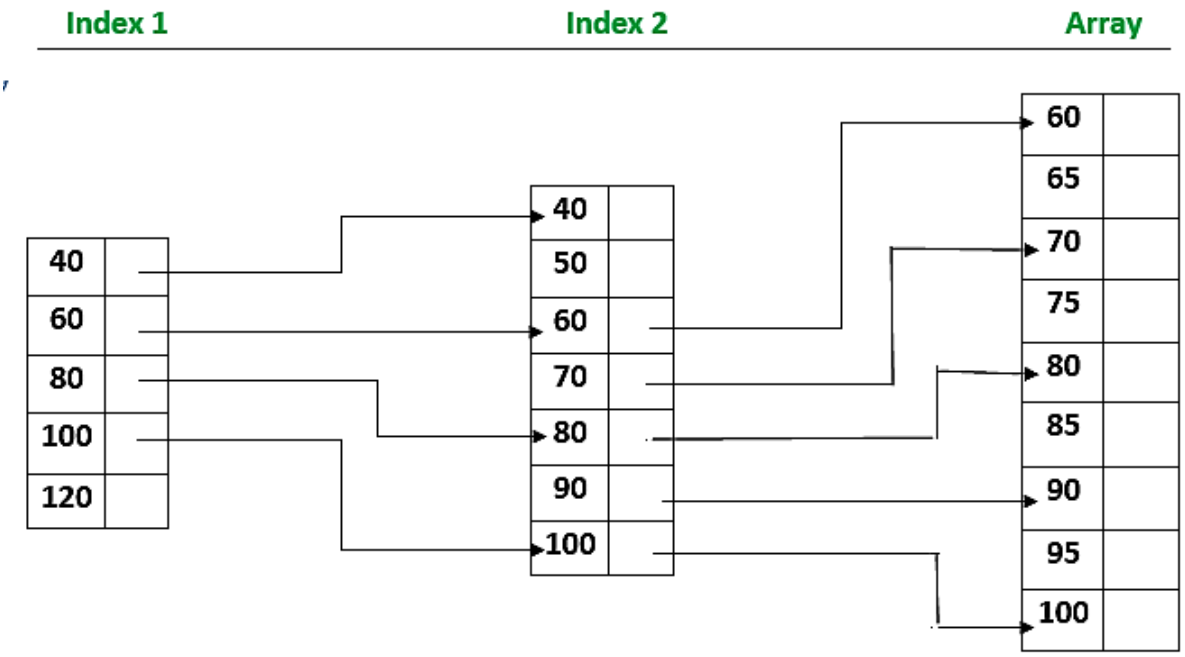
    # Step 1: Build the index table
    for i in range(0, n, block_size):
        index.append(i)

    # Step 2: Search index to find the block
    start = 0
    for i in range(len(index)):
        if arr[index[i]] > target:
            break
        start = index[i]

    # Step 3: Linear search in the identified block
    end = min(start + block_size, n)
    for i in range(start, end):
        print(f"Checking arr[{i}] = {arr[i]}")
        if arr[i] == target:
            print(f"\nValue {target} is found at index {i}.")
            print(f"Returning index position {i}.")
            print("Indexed Sequential Search is finished.")
            return i

    print(f"\nValue {target} is not found in the array.")
    return -1


# Example usage
arr = [5, 8, 9, 11, 12, 13, 17, 20]
target = 12
indexed_sequential_search(arr, target, block_size=3)
```



2.2) Sorting :

2.2.1) Concepts- Stability, Efficiency, and Number of Passes

Concepts in Sorting

When studying sorting algorithms, it's important to understand certain key concepts that help compare and evaluate them. The most common parameters used are **Stability**, **Efficiency**, and **Number of Passes**.

1. Stability in Sorting

Definition:
A sorting algorithm is said to be **stable** if it preserves the **relative order** of equal elements in the original list after sorting.

Why is stability important?

Stability is useful when we want to sort data based on multiple criteria (multi-level sorting). For example, if we first sort a list of students by roll number and then by name, we would want the students with the same name to maintain their roll number order. Stable sorting makes this possible.

Example:

Original list:
[(4, "A"), (2, "B"), (4, "C")] — sorted by first value.

After stable sort:
[(2, "B"), (4, "A"), (4, "C")] — relative order of (4, "A") and (4, "C") is preserved.

After unstable sort:
[(2, "B"), (4, "C"), (4, "A")] — relative order is lost.

Stable Algorithms: Insertion Sort, Merge Sort, Bubble Sort (if implemented correctly)

Unstable Algorithms: Selection Sort, Quick Sort, Heap Sort

2. Efficiency of Sorting

Definition:
Efficiency refers to how much **time and memory** a sorting algorithm uses to complete the sorting process.

It is generally measured by:

- **Time Complexity** (how fast it runs):
  - Best Case
  - Average Case
  - Worst Case
- **Space Complexity** (how much extra memory it uses):
  - In-place sort: uses constant extra space (O(1))
  - Out-of-place sort: uses extra memory (like Merge Sort)

Comparison Example:

Algorithm	Best Case	Average Case	Worst Case	Space
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)

In-place Algorithms: Bubble Sort, Insertion Sort, Quick Sort
Out-of-place Algorithms: Merge Sort

3. Number of Passes

Definition:
A **pass** refers to a complete traversal or scan of the array during sorting.

Each pass attempts to move the elements closer to their correct position.

Why count passes?
It helps understand how many times the algorithm goes through the data, which directly impacts performance.

Examples:

- **Bubble Sort** makes **n-1 passes** to sort n elements.
- **Selection Sort** also makes **n-1 passes**, selecting the smallest or largest element in each pass.
- **Insertion Sort** makes up to n-1 passes, as each element (after the first) is inserted into the sorted portion.

In more optimized or recursive algorithms like Merge Sort or Quick Sort, passes are not always counted in a linear way, as they involve divide-and-conquer techniques and recursion.

2.2.2) Internal and External Sorting

Sorting is a fundamental operation in computer science that helps organize data for efficient searching, analysis, and processing. Based on **where the data is stored during the sorting process**, sorting algorithms are broadly classified into two categories:

- **Internal Sorting**
- **External Sorting**

1. Internal Sorting

Definition:
Internal Sorting refers to the process of sorting data that fits entirely in the main memory (RAM). All the elements to be sorted are loaded into the memory at once, and the sorting is performed using internal memory.

Key Characteristics:

- Entire dataset is available in RAM.
- Fast access to data due to direct memory access.
- Suitable for small to medium-sized datasets.
- No need for disk read/write during the sorting process.

Examples of Internal Sorting Algorithms:

- **Bubble Sort**
- **Selection Sort**
- **Insertion Sort**
- **Quick Sort**
- **Heap Sort**
- **Merge Sort** (when implemented to work entirely in memory)

Advantages:

- Faster than external sorting because memory access is faster than disk access.
- Easier to implement and debug.
- Efficient for datasets that fit in available RAM.

Disadvantages:

- Not suitable for very large datasets that exceed available memory.
- Limited by the size of RAM.

Applications:

- Sorting data in applications where the dataset is small (like sorting lists, arrays, small files).
- Sorting during data processing inside software systems.

2. External Sorting

Definition:
External Sorting is used when the data to be sorted is too large to fit in main memory and resides in external storage such as hard disks, SSDs, or tapes. In this case, sorting is done using a combination of main memory and external storage.

Key Characteristics:

- Only parts of the data are loaded into memory at a time.
- Requires reading and writing to disk.
- Designed for large-scale data processing.
- Uses techniques like divide-and-conquer to split data into chunks.

Examples of External Sorting Techniques:

- **External Merge Sort (K-way merge sort):** Most commonly used external sorting algorithm. It works in two phases:
    1. **Run Generation Phase:** Divide the large file into chunks that fit in memory, sort each chunk using an internal sort, and write the sorted chunks (runs) back to disk.
    2. **Merge Phase:** Merge the sorted runs using a multi-way merge to get the final sorted file.

Advantages:

- Handles huge datasets that exceed memory limitations.
- Efficient for sorting files stored on disk (e.g., log files, databases).

Disadvantages:

- Slower than internal sorting due to disk I/O operations.
- More complex to implement.
- Requires additional external storage space.

Applications:

- Database systems (sorting tables too large to load into memory).
- Big data applications.

- Operating systems dealing with file sorting.
- Search engines (indexing and sorting large document sets).

Comparison Table

Feature	Internal Sorting	External Sorting
Memory Requirement	Fits entirely in main memory	Exceeds main memory, uses external storage
Speed	Faster (RAM is faster than disk)	Slower (due to disk access)
Data Size	Small to medium	Very large datasets
Complexity	Easier to implement	More complex due to file handling
Examples	Quick Sort, Bubble Sort, Heap Sort	External Merge Sort, K-way Merge Sort
Use Cases	Software applications, embedded systems	Databases, big data systems, file sorting

2.2.3) Bubble sort

Bubble Sort is the simplest sorting algorithm that works by repeatedly swapping adjacent elements if they are in the wrong order. The name “bubble” refers to how the largest (or smallest) element "bubbles up" to the top (or bottom) of the array after each pass.

It is mainly used for educational purposes due to its simplicity, not for performance in real-world applications.

How Bubble Sort Works?

1. Start from the first element and compare it with the next one.
2. If the current element is greater than the next one, swap them.
3. Move to the next pair of elements and repeat the comparison and swap.
4. Continue this process till the end of the array. This completes one **pass**.
5. Repeat the entire process for the remaining (n-1) elements.
6. After every pass, the largest unsorted element is placed at its correct position at the end.

This process continues until no swaps are needed, meaning the list is sorted.

Step-by-Step Example

Unsorted Array:
[5, 1, 4, 2, 8]

Pass 1:

- Compare 5 and 1 → Swap → [1, 5, 4, 2, 8]
- Compare 5 and 4 → Swap → [1, 4, 5, 2, 8]
- Compare 5 and 2 → Swap → [1, 4, 2, 5, 8]
- Compare 5 and 8 → No swap → [1, 4, 2, 5, 8]

Pass 2:

- Compare 1 and 4 → No swap
- Compare 4 and 2 → Swap → [1, 2, 4, 5, 8]
- Compare 4 and 5 → No swap
- Compare 5 and 8 → No swap

Pass 3:

- Compare 1 and 2 → No swap
- Compare 2 and 4 → No swap
- Compare 4 and 5 → No swap → (Array is now sorted)

Since no swaps occurred, sorting is complete.

Code Example

```
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        swapped = False
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break  # No swaps means array is already sorted

arr = [5, 1, 4, 2, 8]
bubble_sort(arr)
print("Sorted array:", arr)
```

Time Complexity of Bubble Sort

Case	Comparisons	Swaps	Time Complexity

Best Case	n-1	0	**O(n)**
Average Case	~$n^2/2$	~$n^2/4$	**O(n^2)**
Worst Case	n(n-1)/2	n(n-1)/2	**O(n^2)**

- **Best case** occurs when the array is already sorted.
- **Worst case** occurs when the array is sorted in reverse order.

Space Complexity:

- **O(1)** — Bubble Sort is an in-place sorting algorithm, meaning it doesn't require extra space.

Stability of Bubble Sort

- **Stable:** Yes. It maintains the relative order of elements with equal values.

Number of Passes

- At most (n-1) passes are required for sorting an array of n elements.

Applications of Bubble Sort

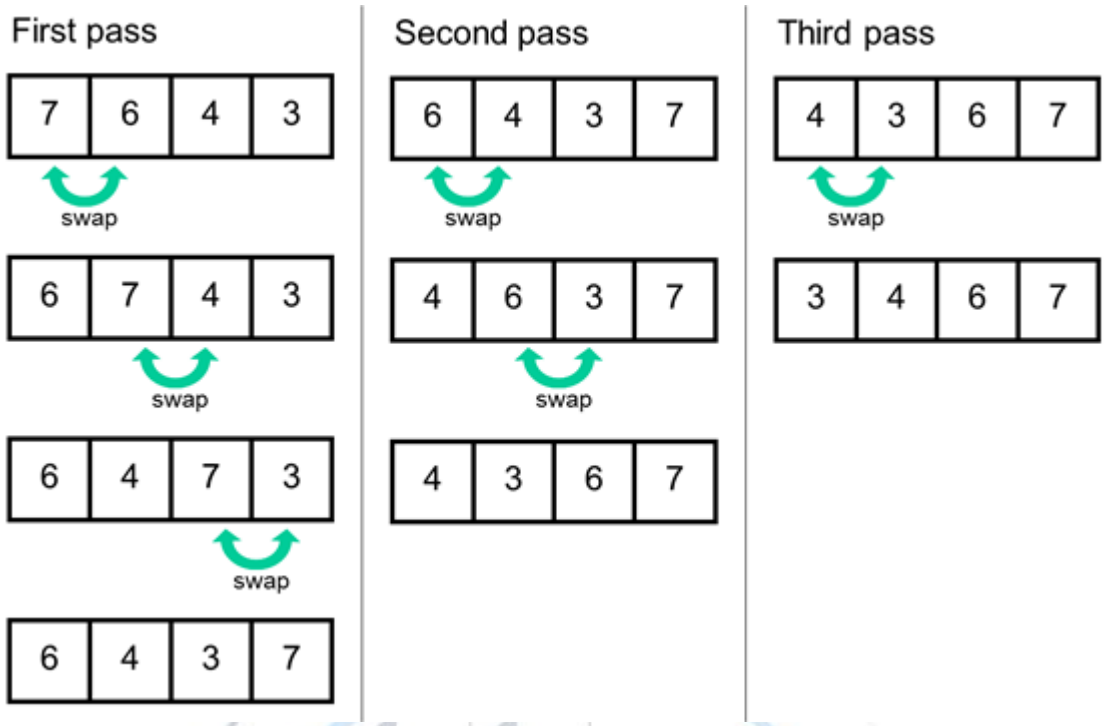
- Educational use to teach basic sorting logic.
- Sorting very small datasets.
- Useful where memory space is extremely limited.
- Detecting whether a list is already sorted (can terminate early if no swaps).

Advantages

- Very simple to understand and implement.
- No extra memory required (in-place).
- Efficient for already sorted or nearly sorted arrays.

Disadvantages

- Very inefficient for large datasets.
- High time complexity O(n^2) even for modest input sizes.
- Not suitable for performance-critical applications.



2.2.4) Insertion Sort

Insertion Sort is a simple and intuitive sorting algorithm that works similarly to the way we sort playing cards in our hands. It builds the final sorted array one item at a time, placing each new element into its correct position relative to the previously sorted elements.

It is an **in-place**, **stable** sorting algorithm, best suited for **small or nearly sorted datasets**.

How Insertion Sort Works?

1. Start with the second element (index 1), and compare it with the first.
2. If the second element is smaller than the first, insert it before the first.
3. Then move to the third element, compare it with all previous sorted elements, and insert it at the correct position.
4. Repeat the process for all remaining elements.

At each step, the left part of the array is sorted, and the current element is inserted into its correct position.

Step-by-Step Example

Unsorted Array:
[5, 1, 4, 2, 8]

Pass 1:
Compare 1 with 5 → 1 < 5 → Insert 1 before 5
→ [1, 5, 4, 2, 8]

Pass 2:
Compare 4 with 5 → 4 < 5 → Shift 5
Compare 4 with 1 → 4 > 1 → Insert after 1
→ [1, 4, 5, 2, 8]

Pass 3:
Compare 2 with 5 → 2 < 5 → Shift 5
Compare 2 with 4 → 2 < 4 → Shift 4
Compare 2 with 1 → 2 > 1 → Insert after 1
→ [1, 2, 4, 5, 8]

Pass 4:
Compare 8 with 5 → 8 > 5 → Already in correct place
→ [1, 2, 4, 5, 8]

Now the array is sorted.

Code Example

```
def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1

        # Move elements greater than key to one position ahead
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j -= 1

        arr[j + 1] = key

arr = [5, 1, 4, 2, 8]
insertion_sort(arr)
print("Sorted array:", arr)
```

Time Complexity of Insertion Sort

Case	Comparisons & Shifts	Time Complexity
Best Case	n - 1	**O(n)**
Average Case	~n^2 / 4	**O(n^2)**

Worst Case	n(n - 1)/2	**$O(n^2)$**

- **Best case** occurs when the array is already sorted.
- **Worst case** occurs when the array is in reverse order.

Space Complexity:

- **O(1)** — It is an **in-place sorting** algorithm.

Stability of Insertion Sort

- **Stable:** Yes. It maintains the relative order of equal elements.

Number of Passes

- Requires **(n - 1)** passes to sort n elements.

Applications of Insertion Sort

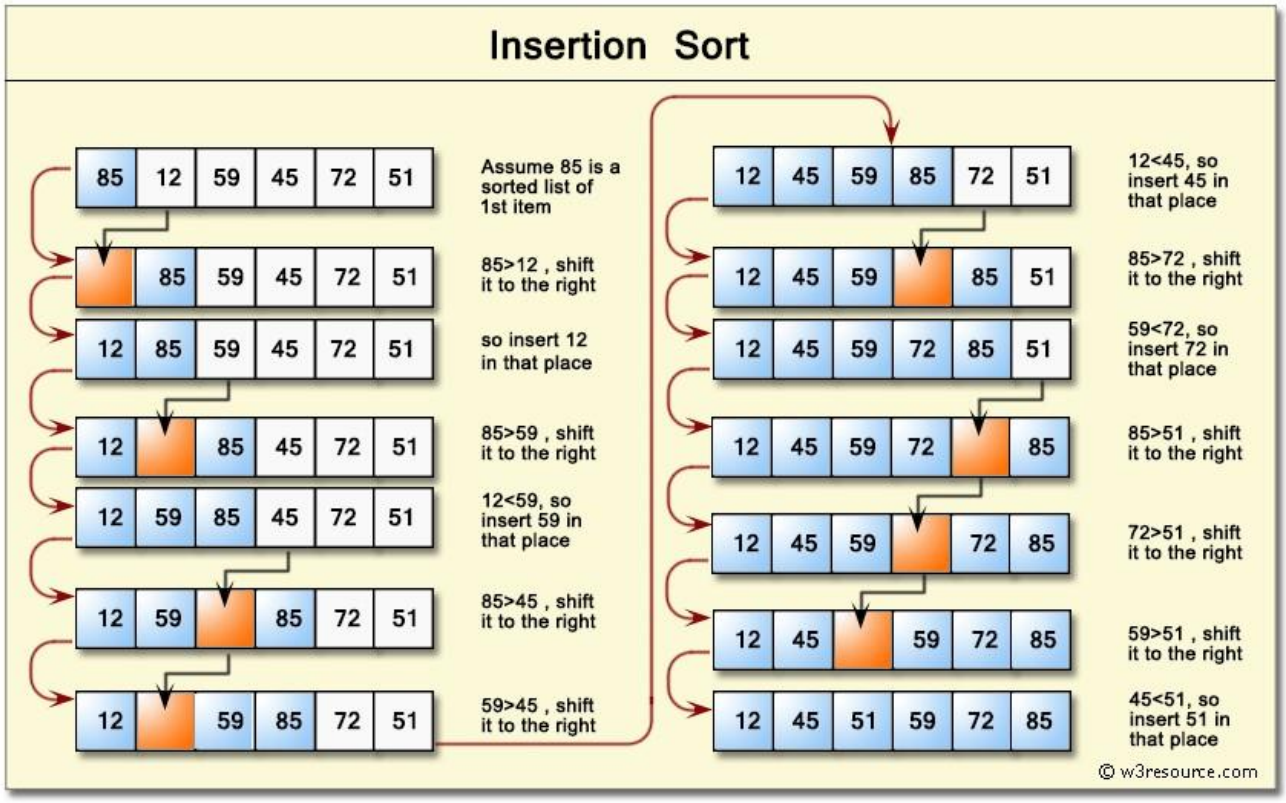
- When the **data is almost sorted** (nearly in order).
- For **small datasets** (like 10 to 20 elements).
- In **online algorithms** where data keeps arriving, and we must sort as we go.
- Used as a **building block in hybrid algorithms** like TimSort.

Advantages

- Very simple to implement and understand.
- Efficient for **small or nearly sorted arrays**.
- Requires **no extra memory** (in-place).
- Maintains stability of elements.

Disadvantages

- Inefficient on **large lists** due to $O(n^2)$ time.
- Slower than algorithms like Merge Sort, Quick Sort for large datasets.



2.2.5) Selection Sort

Selection Sort is a simple comparison-based sorting algorithm. It works by repeatedly selecting the smallest (or largest) element from the unsorted part of the array and placing it in its correct sorted position.

It is called "selection" sort because it selects the smallest element in each pass and moves it to the correct place.

How Selection Sort Works?

1. Start from the first element. Find the smallest element in the entire array.
2. Swap it with the first element.
3. Move to the second position. Again, find the smallest element from the remaining unsorted part and swap it with the second element.
4. Repeat this process for all elements until the array is sorted.

Unlike bubble or insertion sort, Selection Sort makes fewer swaps, but still performs many comparisons.

Step-by-Step Example

Unsorted Array:
[5, 1, 4, 2, 8]

Pass 1:

- Minimum value in [5, 1, 4, 2, 8] is 1.

- Swap 1 with 5 → [1, 5, 4, 2, 8]

Pass 2:

- Minimum value in [5, 4, 2, 8] is 2.
- Swap 2 with 5 → [1, 2, 4, 5, 8]

Pass 3:

- Minimum value in [4, 5, 8] is 4.
- No swap needed → [1, 2, 4, 5, 8]

Pass 4:

- Minimum in [5, 8] is 5.
- No swap needed

Array is now sorted.

Python Code Example

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n):
        min_index = i
        # Find the smallest element in the remaining unsorted array
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j
        # Swap the found minimum element with the first element of the unsorted part
        arr[i], arr[min_index] = arr[min_index], arr[i]

arr = [5, 1, 4, 2, 8]
selection_sort(arr)
print("Sorted array:", arr)
```

Time Complexity of Selection Sort

Case	Comparisons	Swaps	Time Complexity
Best Case	n(n - 1)/2	n	**O(n²)**
Average Case	n(n - 1)/2	n	**O(n²)**
Worst Case	n(n - 1)/2	n	**O(n²)**

- Selection Sort **always performs the same number of comparisons**, regardless of the input.

Space Complexity:

- **O(1)** — It's an **in-place sorting algorithm**, needing no extra memory.

Stability of Selection Sort

- **Unstable:** Because it might change the relative order of equal elements during swaps.

Number of Passes

- For n elements, it requires exactly **(n - 1) passes**.

Applications of Selection Sort

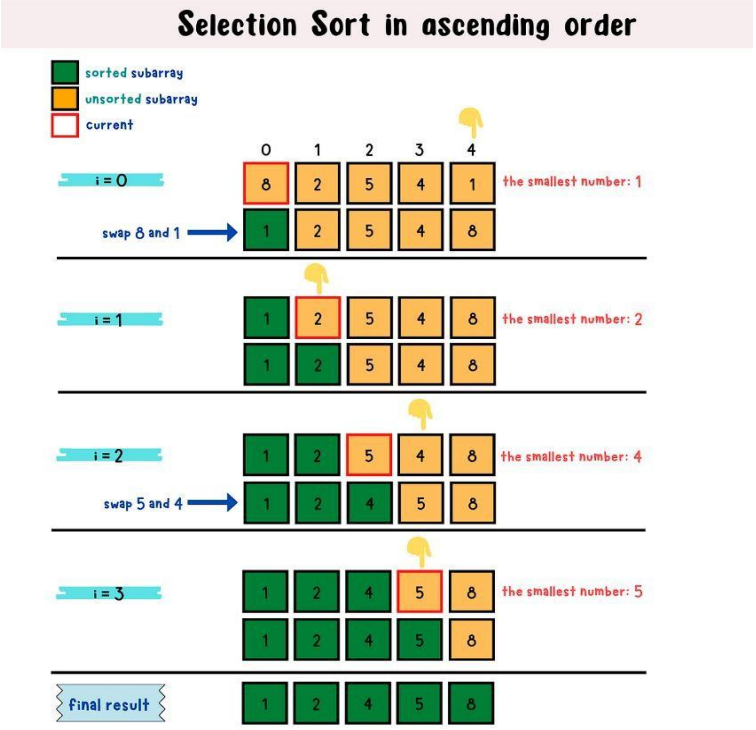
- Good for **small datasets**.
- When **memory write operations are costly** (fewer swaps).
- Used when the number of **swaps needs to be minimized** (e.g., in embedded systems).

Advantages

- Simple and easy to implement.
- Performs well on small lists.
- Makes the **minimum number of swaps** among all simple sorting algorithms.

Disadvantages

- **Inefficient for large lists** due to $O(n^2)$ time.
- Performs a high number of **comparisons**, even if the list is already sorted.
- **Unstable**, which may be a problem in some applications.



2.2.6) Quick Sort

Quick Sort is one of the fastest and most widely used sorting algorithms. It follows the Divide and Conquer strategy to sort elements by partitioning the array into sub-arrays and sorting them recursively.

Quick Sort is not a stable sort, but it is highly efficient and performs well on large datasets. It is often the default sorting algorithm used in many programming libraries.

How Quick Sort Works?

1. **Choose a Pivot Element** – Select one element from the array to act as a pivot (commonly first, last, middle, or a random element).
2. **Partition the Array** – Rearrange the array so that:
    - All elements **less than the pivot** are moved to the **left** of the pivot.
    - All elements **greater than the pivot** are moved to the **right** of the pivot.
3. **Recursively Apply Quick Sort** – Apply the same process to the left and right sub-arrays (excluding the pivot).

This process continues until all sub-arrays contain only one element, which means the array is sorted.

Step-by-Step Example

Unsorted Array:
[8, 4, 7, 3, 10, 2]
Pivot: Choose the last element, 2.

Partition 1:
After rearranging:

- Left side: [] (no element < 2)
- Right side: [8, 4, 7, 3, 10]
- Pivot (2) is placed at the beginning.
  → [2, 4, 7, 3, 10, 8]

Apply Quick Sort on [4, 7, 3, 10, 8]
Pivot: 8

Partition 2:

- Left: [4, 7, 3]
- Right: [10]
- Pivot (8) is placed correctly.
  → [2, 4, 7, 3, 8, 10]

Now sort [4, 7, 3]
Pivot: 3

- Left: []
- Right: [4, 7]
  → [2, 3, 4, 7, 8, 10]

Continue sorting sub-arrays until fully sorted.

Python Code Example

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr  # Base case

    pivot = arr[-1]  # Choosing the last element as pivot
    left = [x for x in arr[:-1] if x <= pivot]  # Elements ≤ pivot
    right = [x for x in arr[:-1] if x > pivot]  # Elements > pivot

    return quick_sort(left) + [pivot] + quick_sort(right)

arr = [8, 4, 7, 3, 10, 2]
sorted_arr = quick_sort(arr)
print("Sorted array:", sorted_arr)
```

Time Complexity of Quick Sort

Case	Description	Time Complexity
Best Case	Pivot divides array evenly	**O(n log n)**
Average Case	Random order of elements	**O(n log n)**
Worst Case	Array is already sorted (unbalanced partitioning)	**$O(n^2)$**

- Best and average cases make Quick Sort very fast.
- Worst case occurs when the pivot is always the smallest or largest element (poor partitioning).

Space Complexity:

- **O(log n)** (due to recursive calls stack space)

Stability of Quick Sort

- **Not stable:** Equal elements may not maintain original order after sorting.

Number of Passes

- Varies depending on pivot positions.
- In average case, about **$\log_2 n$ levels** of recursion are needed.

Applications of Quick Sort

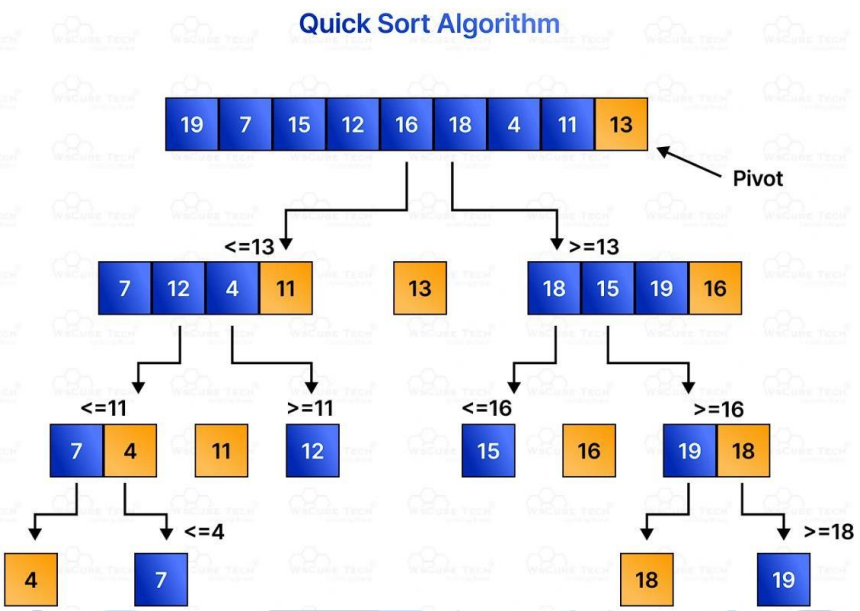
- Suitable for **large datasets**.
- Often used in **system-level libraries** and **built-in sort functions** (like in C++, Python, Java).
- Used in **divide and conquer problems** requiring in-place sorting.

Advantages

- Very fast in practice, especially with randomized pivot.
- Efficient for **average-case performance**.
- Requires **no additional memory** for in-place implementations.

Disadvantages

- **Not stable**
- Worst-case performance is **O(n²)**
- Not good for small arrays unless optimized with hybrid approaches



2.2.7) Merge Sort

Introduction

Merge Sort is a powerful and efficient **Divide and Conquer** based sorting algorithm. It works by dividing the array into smaller sub-arrays, sorting them, and then **merging** the sorted sub-arrays to produce the final sorted array.

Merge Sort is known for its **consistent performance** and is a **stable sorting algorithm**.

How Merge Sort Works?

1. **Divide** the unsorted array into two halves.
2. **Recursively sort** each half using Merge Sort.
3. **Merge** the two sorted halves to produce a single sorted array.

This process continues until each sub-array has only one element (which is inherently sorted), and then we merge all sub-arrays in a way that results in a sorted array.

Step-by-Step Example

Unsorted Array:
[38, 27, 43, 3, 9, 82, 10]

Step 1: Divide

Split into halves recursively:

- [38, 27, 43] and [3, 9, 82, 10]
- [38] [27, 43] and [3, 9] [82, 10]
- [27] [43], [3] [9], [82] [10]

Step 2: Conquer (Sort and Merge)

- Merge [27] and [43] → [27, 43]
- Merge [38] and [27, 43] → [27, 38, 43]
- Merge [3] and [9] → [3, 9]
- Merge [82] and [10] → [10, 82]
- Merge [3, 9] and [10, 82] → [3, 9, 10, 82]

Step 3: Final Merge

- Merge [27, 38, 43] and [3, 9, 10, 82]
  → [3, 9, 10, 27, 38, 43, 82]

The array is now sorted.

Code Example

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2  # Find the midpoint
        left_half = arr[:mid]
        right_half = arr[mid:]

        merge_sort(left_half)  # Sort left half
        merge_sort(right_half)  # Sort right half

        # Merge the sorted halves
        i = j = k = 0
        while i < len(left_half) and j < len(right_half):
            if left_half[i] < right_half[j]:
                arr[k] = left_half[i]
                i += 1
            else:
                arr[k] = right_half[j]
                j += 1
            k += 1

        # Copy remaining elements
        while i < len(left_half):
            arr[k] = left_half[i]
            i += 1
            k += 1
```

```
    while j < len(right_half):
        arr[k] = right_half[j]
        j += 1
        k += 1

arr = [38, 27, 43, 3, 9, 82, 10]
merge_sort(arr)
print("Sorted array:", arr)
```

Time Complexity of Merge Sort

Case	Time Complexity
Best Case	**O(n log n)**
Average Case	**O(n log n)**
Worst Case	**O(n log n)**

Merge Sort divides the array into halves (log n times) and merges each level (n operations), resulting in **O(n log n)** complexity.

Space Complexity:

- **O(n)** – Extra memory is required to store temporary sub-arrays.

Stability of Merge Sort

- **Stable:** Equal elements retain their original order.

Number of Passes

- For an array of size n, it takes approximately **$\log_2 n$ passes** to fully divide and merge the array.

Applications of Merge Sort

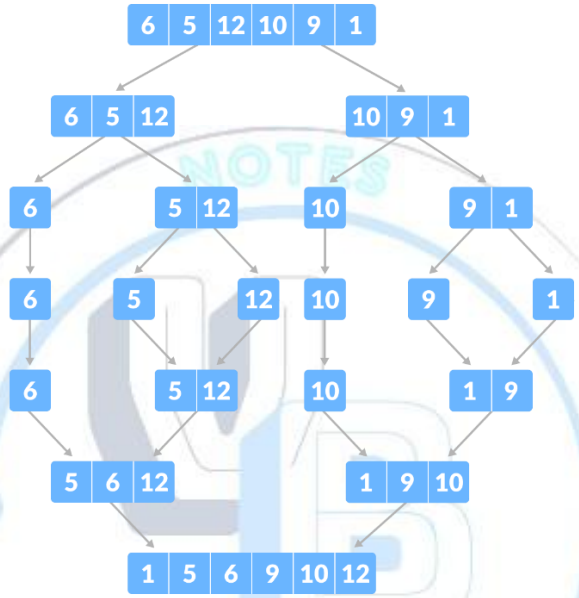
- Used in **external sorting**, where large datasets are sorted using file systems.
- Best suited when **stability is required**.
- Preferred in scenarios where **linked lists** are sorted (as merge sort can be implemented without extra space in linked lists).
- Useful in **multithreaded environments**, as different subarrays can be processed in parallel.

Advantages

- Predictable time complexity of **O(n log n)**.
- **Stable** sort.
- Works well with large datasets.
- Great for **linked lists** and **external sorting**.

Disadvantages

- Requires **extra memory** (not in-place).
- Slightly slower than quick sort in practice due to additional space and copying.



2.3) Case Study : Social Network Adjacency Matrix Representing friendship connections among millions of users

In large-scale social networks such as Facebook, Instagram, or LinkedIn, millions of users are connected to each other through friendships or follow relationships. To understand and analyse these relationships programmatically, we need a suitable data structure. One such structure is the **Adjacency Matrix**, which is a 2D array used to represent graphs.

In this case study, we explore how a **Social Network** can be modelled using an adjacency matrix to represent friendships.

Problem Scenario

Imagine a social networking platform with **millions of users**. Each user can connect (be friends) with many others. We need a way to:

- Efficiently **store connections**.
- Quickly **check if two users are friends**.
- Possibly **count the number of friends** a user has.

We model this using a **graph**, where:

- **Each user** is a **node (vertex)**.
- **Each friendship** is an **edge** between two nodes.

For this, we can use an **Adjacency Matrix** representation of the graph.

What is an Adjacency Matrix?

An adjacency matrix is a **2D array** where the cell at [i][j] indicates whether user i is connected to user j.

- If user i and user j are friends: matrix[i][j] = 1
- If they are not friends: matrix[i][j] = 0

Since friendship is mutual in most social platforms, this matrix is typically **symmetric**:

If matrix[i][j] = 1 ⇒ matrix[j][i] = 1

Example (Small Scale for Understanding)

Let's say we have 4 users:

- User 0 is friends with Users 1 and 2
- User 1 is friends with Users 0 and 3
- User 2 is friends with User 0
- User 3 is friends with User 1

Then the adjacency matrix would look like:

	0	1	1	2	3
0	0	1	1	1	0
1	1	0	0	1	1
2	0	1	0	0	0
3	0	0	1	0	0

This matrix clearly shows all the friendship relationships.

Scaling to Millions of Users

When modelling **millions of users**, this matrix becomes extremely large. For example:

- 1 million users → matrix size = 1,000,000 × 1,000,000 cells
- That's **1 trillion** entries, which is extremely memory-intensive.

This leads to an important consideration:

Most users are not friends with all other users. Each user typically has hundreds or thousands of friends, not millions.
So the matrix will have mostly **zeros** → It is a **Sparse Matrix**.

Optimization: Use Sparse Matrix Representation

Since most entries are zeros, storing the full matrix is inefficient. Instead, we use a **sparse matrix representation**, where only the non-zero (active friendship) entries are stored.

One common approach:

- Store friendships as a list of tuples: (i, j) → means user i is friends with user j
- This saves a huge amount of space

Operations on the Adjacency Matrix

Operation	Time Complexity	Description
Check if user A is friends with B	**O(1)**	Direct access using matrix[A][B]
Add a friendship	**O(1)**	Set matrix[A][B] = matrix[B][A] = 1
Remove a friendship	**O(1)**	Set matrix[A][B] = matrix[B][A] = 0

Advantages of Using Adjacency Matrix

- **Fast lookups**: Friendship between two users can be checked in constant time O(1)
- **Simple structure**: Easy to implement using 2D arrays
- Well-suited for **dense graphs** where most users are connected

Disadvantages

- **High memory usage** for large networks with sparse connections
- Not practical for **millions of users** unless optimized (e.g., sparse matrix, adjacency list)

Applications in Social Networks

1. **Friend Recommendation**:
   - Use matrix multiplication or DFS/BFS on the adjacency matrix to suggest mutual friends.
2. **Network Analysis**:
   - Count number of connected components, shortest paths between users, etc.
3. **Detect Communities**:
   - Use clustering algorithms on the matrix to find friend groups.

4. **Spam/Bot Detection**:
    - Identify unusual patterns in connections.
5. **Privacy Checks**:
    - Enforce who can see or interact with whom based on matrix entries.

SPPU_FE_ENGG Click to join telegram

UNIT 3 : Stacks, Queues and Linked Lists

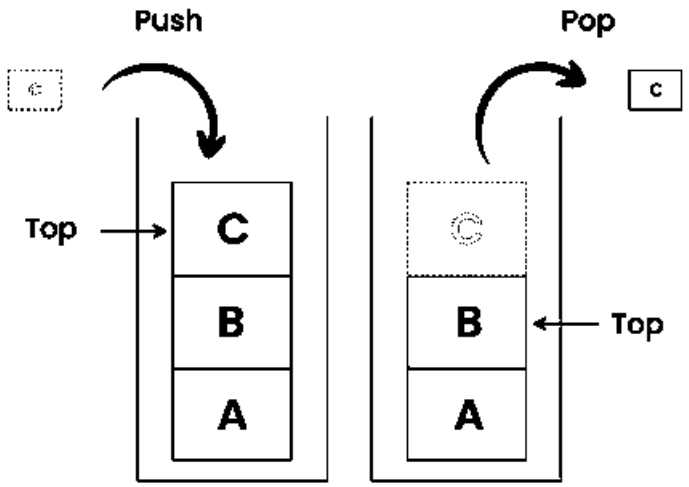
Syllabus

Stacks: Stack operations, Multiple Stacks, Applications of Stack for Expression Conversion [infix, prefix and postfix], Postfix expression evaluation
Queues: Queue Operations, Circular Queue, Priority Queue and its advantages and applications
Linked list: Introduction of Linked Lists, Primitive Operations on Linked List- Create, Traverse, Search, Insert, Delete, Sort, and Concatenate. Types of Linked List: Singly linked, linear and Circular Linked Lists, Doubly Linked List,
Case study: Implementation of Stack and Queue operations using Linked lists

3.1) Stacks

A Stack is a linear data structure that follows a particular order in which the operations are performed. The order may be LIFO (Last in First Out) or FILO (First in Last Out). LIFO implies that the element that is inserted last, comes out first and FILO implies that the element that is inserted first, comes out last.
It behaves like a stack of plates, where the last plate added is the first one to be removed. Think of it this way :

- Pushing an element onto the stack is like adding a new plate on top.
- Popping an element removes the top plate from the stack.

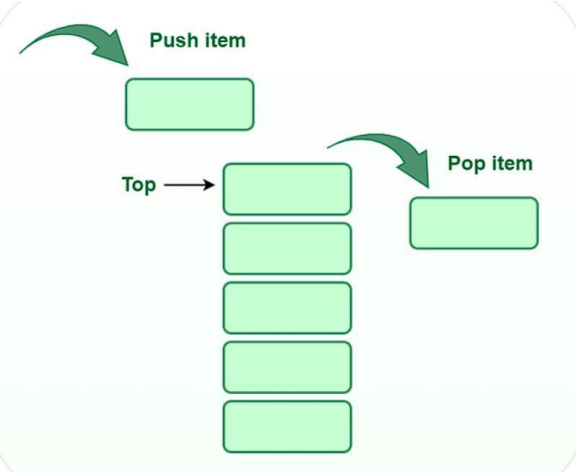


3.1.1) Stack operations

In order to make manipulations in a stack, there are certain operations provided to us for Stack, which include :

1) push() to insert an element into the stack.
2) pop() to remove an element from the stack.
3) top() Return the top element of the stack.

4) isEmpty() return true if the stack is empty else false
5) size() returns the size of the stack



1) Push Operation in Stack : Push operation adds an item to the stack. If the stack is full, then it is said to be an Overflow condition.

Example :

```
stack = []

stack.append(1) # This pushes 1 to the stack top
stack.append(2) # This pushes 2 to the stack top
stack.append(3) # This pushes 3 to the stack top
stack.append(4) # This pushes 4 to the stack top
stack.append(5) # This pushes 5 to the stack top

# printing the stack

while stack:
    print(stack[-1], end=" ")
    stack.pop()

# The above loop prints "5 4 3 2 1"
```

2) Pop Operation in Stack : Pop operation is used to remove an item from the stack. The items are popped in the reversed order in which they are pushed. If the stack is empty, then it is said to be an Underflow condition.

Example :

```
stack = []

stack.append(1) # This pushes 1 to the stack top
stack.append(2) # This pushes 2 to the stack top
stack.append(3) # This pushes 3 to the stack top
stack.append(4) # This pushes 4 to the stack top
stack.append(5) # This pushes 5 to the stack top

# Now, let us remove elements from the stack using pop function
while stack:
```

```
    print(stack[-1], end=" ")
    stack.pop() # removes the top element from the stack
```

3) Top Operation in Stack : Top operation is used to return the top element of the stack.

Example :

```
def topElement(s):
    return s[-1]

s = [] # creating a stack of integers

s.append(1) # This pushes 1 to the stack top
print(topElement(s)) # Prints 1 since 1 is present at the stack top

s.append(2) # This pushes 2 to the stack top
print(topElement(s)) # Prints 2 since 2 is present at the stack top

s.append(3) # This pushes 3 to the stack top
print(topElement(s)) # Prints 3 since 3 is present at the stack top
```

4) isEmpty() Operation in Stack : isEmpty operation is a Boolean operation that is used to determine if the stack is empty or not. This operation will return true if the stack is empty, else false.

Example :

```
def isEmpty(s):
    isStackEmpty = len(s) == 0 # checking whether stack is empty or
                               # not and storing it into isStackEmpty variable
    return isStackEmpty # returning bool value stored in isStackEmpty

s = []

# The if - else conditional statements below prints "Stack is empty."
if isEmpty(s):
    print("Stack is empty.")
else:
    print("Stack is not empty.")

s.append(1) # Inserting value 1 to the stack top

# The if - else conditional statements below prints "Stack is not empty."
if isEmpty(s):
    print("Stack is empty.")
else:
    print("Stack is not empty.")
```

5) size() Operation in Stack : Size operation in Stack is used to return the count of elements that are present inside the stack.

Example :

```
stack = [] # creating an empty list as a stack
```

```
print(len(stack)) # Prints 0 since the stack is empty

stack.append(1) # This appends 1 to the stack
stack.append(2) # This appends 2 to the stack
print(len(stack)) # Prints 2 since the stack contains two elements

stack.append(3) # This appends 3 to the stack
print(len(stack)) # Prints 3 since the stack contains three element
```

6) isFull() Operation in Stack : isFull() operation is a Boolean operation used to determine if the stack has reached its maximum capacity. This operation will return True if the stack is full, else False. It is mainly used in **fixed-size stacks** (like array-based implementations).

Example:

```
def isFull(s, capacity):
    isStackFull = len(s) == capacity  # checking whether stack size has reached its capacity
    return isStackFull  # returning bool value stored in isStackFull

s = []  # empty stack
capacity = 2  # defining maximum capacity for the stack

s.append(10)  # inserting 1st element
s.append(20)  # inserting 2nd element

# The if - else conditional statements below prints "Stack is full."
if isFull(s, capacity):
    print("Stack is full.")
else:
    print("Stack is not full.")

s.pop()  # removing one element

# The if - else conditional statements below prints "Stack is not full."
if isFull(s, capacity):
    print("Stack is full.")
else:
    print("Stack is not full.")
```

7) peek() Operation in Stack : peek() operation is used to **view the top element** of the stack **without removing it**. It helps to access the most recently added element. This is useful when we want to see what's on top of the stack without modifying it.

Example:

```
def peek(s):
    if len(s) == 0:
        return None  # return None if the stack is empty
    topElement = s[-1]  # accessing top element using -1 index
```

```
    return topElement

s = [4, 7, 9]

# Prints "Top element is: 9" since 9 is at the top
print("Top element is:", peek(s))

s.pop()  # removes 9

# Prints "Top element is: 7" now
print("Top element is:", peek(s))
```

8) createStack() Operation : createStack() operation is used to **initialize a new empty stack**. In Python, this can simply be done by creating an empty list, since lists support stack operations like push and pop.

Example:

```
def createStack():
    newStack = []  # creating an empty list to represent stack
    return newStack

s = createStack()  # creating a new stack

# The if - else conditional statements below prints "New stack is created and empty."
if len(s) == 0:
    print("New stack is created and empty.")
else:
    print("Stack creation failed or not empty.")
```

3.1.2) Multiple Stacks :

When a single array is used to implement only one stack, it often leads to **inefficient use of memory**. If the stack overflows but there's still free space in the array (unused by the stack), that space cannot be used. This problem is **rectified** by using **more than one stack within the same array**, which is known as **Multiple Stack Implementation**.

In this technique, the available memory (array) is **logically divided** among multiple stacks. This helps in **efficient memory utilization** and is commonly used when we need to manage multiple independent stacks but with limited total space.

Example: Two Stacks in One Array

Let us consider an array STACK[n] to represent **two stacks**, say **Stack A** and **Stack B**.

- **Stack A** grows **from left to right** (i.e., starting at index 0).
- **Stack B** grows **from right to left** (i.e., starting at index n-1).
- Two top pointers are maintained:
    - topA initialized to -1 for Stack A
    - topB initialized to n for Stack B

Pushing an element into any stack checks whether there is **free space** (i.e., topA < topB - 1). If this condition is false, it indicates **Stack Overflow**.

Advantages of Multiple Stack in One Array:

- **Efficient Space Utilization**: The unused space of one stack can be used by the other.
- **Memory Management**: Suitable when memory is limited but multiple stacks are needed.
- **Logical Separation**: Ideal for systems requiring separate function call stacks, e.g., in threads.

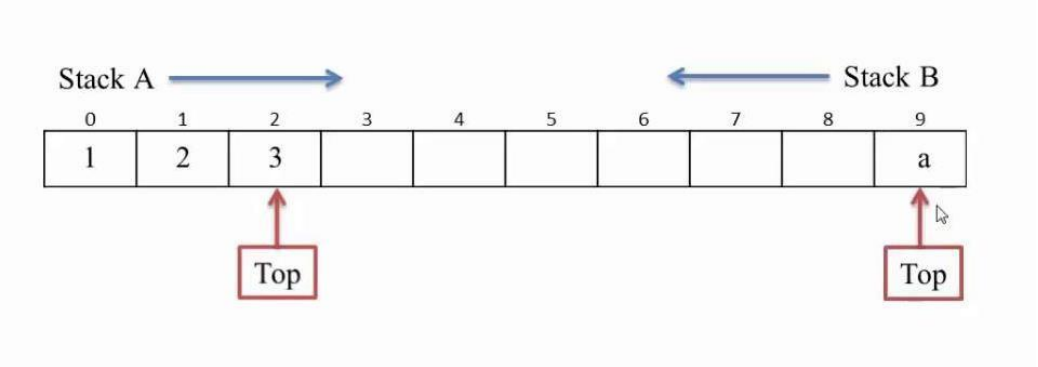
Limitations:

- **Complex Implementation**: Managing multiple top pointers and checks increases complexity.
- **Static Memory**: The total size is still fixed unless dynamic memory allocation is used.
- **Limited Scalability**: Efficient mainly for 2 or 3 stacks; beyond that, management becomes harder.

Real-World Applications:

- Multithreading environments (separate stack per thread)
- Compilers and interpreters
- Virtual machines (e.g., Java Virtual Machine stack frames)

Diagram:



3.1.3) Applications of Stack for Expression Conversion [infix, prefix and postfix] :

In computer science, **expressions** represent mathematical computations. The order of operators and operands determines the correctness of expressions. Computers process expressions efficiently using **stack data structures**, especially when converting between different expression formats.

The three main forms of expressions are:

- **Infix Expression:** Operators are placed **between operands** (human-readable form).
- **Prefix Expression (Polish Notation):** Operators are placed **before operands**.
- **Postfix Expression (Reverse Polish Notation):** Operators are placed **after operands**.

Why Expression Conversion is Needed?

- Computers and compilers prefer **Prefix** or **Postfix** for expression evaluation as they avoid ambiguities and the need for parentheses.
- **Infix expressions** are suitable for human readability but need conversion before evaluation.

3.1.3.1) Role of Stack in Expression Conversion

A **stack** is a Last-In-First-Out (**LIFO**) data structure. During expression conversion, a stack helps in:

- Storing **operators** temporarily.
- Handling **precedence** and **associativity** of operators.
- Managing **parentheses** during parsing.

The algorithm uses stack operations:

- **Push:** Place an element on top of the stack.
- **Pop:** Remove the top element.
- **Peek:** View the top element without removing it.

3.1.3.2) Expression Forms Explained

1. Infix Expression

- Example: (A + B) * C
- Characteristics:
  - Requires parentheses to change order of operations.
  - Human-readable but complex for machine parsing.

2. Prefix Expression (Polish Notation)

- Example: * + A B C
- Characteristics:
  - No parentheses needed.
  - Evaluation is straightforward (scanned **right to left**).

3. Postfix Expression (Reverse Polish Notation)

- Example: A B + C *
- Characteristics:
  - Also doesn't require parentheses.
  - Simple to evaluate (scanned **left to right**).

3.1.3.3) Expression Conversion Algorithms Using Stack

1. Infix to Postfix Conversion

Algorithm Steps:

1. Initialize an empty stack for **operators**.
2. Scan the infix expression from **left to right**.
3. For each element:
   - **Operand:** Add to postfix expression.
   - **'(' (Opening parenthesis):** Push onto stack.
   - **')' (Closing parenthesis):** Pop and add to postfix until '(' is found.
   - **Operator:**
     - Pop operators from stack with **greater or equal precedence** and add them to postfix.
     - Push the current operator on the stack.
4. After the scan, pop any remaining operators on the stack.

Example:

Infix: (A + B) * C

Postfix: A B + C *

2. Infix to Prefix Conversion

Algorithm Steps:

1. **Reverse** the infix expression:
   - Replace (with) and vice versa.

2. Apply **Infix to Postfix conversion** on the reversed expression.
3. Reverse the resulting postfix expression to get the prefix.

Example:
Infix: (A + B) * C
Reversed Infix: C * (B + A)
Postfix of Reversed: C B A + *
Final Prefix: * + A B C

3. Postfix to Infix Conversion
Algorithm Steps:

1. Initialize an empty stack for **operands**.
2. Scan the postfix expression from **left to right**.
3. For each element:
   - **Operand:** Push onto the stack.
   - **Operator:** Pop two operands, form a sub-expression (operand1 operator operand2), push it back onto the stack.
4. The final element on the stack is the infix expression.

Example:
Postfix: A B + C *
Infix: ((A + B) * C)

4. Prefix to Infix Conversion
Algorithm Steps:

1. Initialize an empty stack.
2. Scan the prefix expression from **right to left**.
3. For each element:
   - **Operand:** Push onto the stack.
   - **Operator:** Pop two operands, form a sub-expression (operand1 operator operand2), push it back onto the stack.
4. The final element on the stack is the infix expression.

Example:
Prefix: * + A B C
Infix: ((A + B) * C)

5. Infix to Prefix/Postfix Summary Table

Infix Expression	Prefix Expression	Postfix Expression
A + B	+ A B	A B +
(A + B) * C	* + A B C	A B + C *
A + B * C	+ A * B C	A B C * +
(A + B) / (C - D)	/ + A B - C D	A B + C D - /

3.1.3.4) Operator Precedence and Associativity

Operator	Precedence	Associativity

^	Highest	Right to Left
*, /, %	High	Left to Right
+, -	Low	Left to Right

- **Higher precedence** operators are evaluated first.
- **Associativity** defines the order when operators have the same precedence.

3.1.3.5) Real-life Applications of Expression Conversion

- **Compiler Design:** Parsing arithmetic expressions during code compilation.
- **Calculators:** Evaluation of mathematical expressions.
- **Postfix Expression Evaluation in Algorithms:** Simpler stack-based evaluation.
- **Expression Parsing in Interpreters & Virtual Machines.**

3.1.3.6) Example Problem

Problem: Convert (A + B * C - D) / E into Postfix and Prefix.

Postfix:

Infix: (A + B * C - D) / E

Postfix: A B C * + D - E /

Prefix:

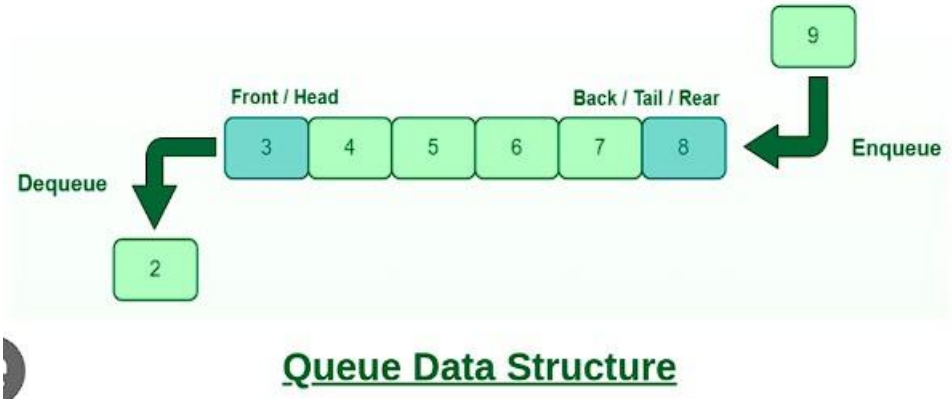
Infix: (A + B * C - D) / E

Prefix: / - + A * B C D E

3.2) Queues

3.2.1) Queue Operations

A Queue is an ordered list in which insertions are done at one end (rear/tail) and deletions are done at the other end (front/head). This behavior is often referred to as FIFO (First-In, First-Out).



Queue Data Structure

Key Terminologies:

- **Front (Head):** The end of the queue from which elements are deleted (dequeued).
- **Rear (Tail):** The end of the queue where new elements are inserted (enqueued).

Basic Queue Operations:

1. **enqueue(item) / insert(item) / add(item):**

- **Purpose:** Adds an item to the rear of the queue.
- **Logic:**
  - Check if the queue is full (overflow condition). If full, an error or exception is returned.
  - If not full, increment the rear pointer.
  - Insert the item at the new rear position.
  - If it's the first element being enqueued, also set front to 0 (or the initial index).
- **Time Complexity:** O(1) (constant time).

2. **dequeue() / delete() / remove():**
   - **Purpose:** Removes and returns the element from the front of the queue.
   - **Logic:**
     - Check if the queue is empty (underflow condition). If empty, an error or exception is returned.
     - If not empty, retrieve the element at the front position.
     - Increment the front pointer.
     - If front becomes greater than rear after deletion, it means the queue has become empty. In this case, reset both front and rear to their initial invalid state (e.g., -1) to signify an empty queue.
   - **Time Complexity:** O(1) (constant time).
3. **peek() / front() / top():**
   - **Purpose:** Returns the element at the front of the queue without removing it.
   - **Logic:**
     - Check if the queue is empty. If empty, an error or exception is returned.
     - If not empty, return the element at the front position.
   - **Time Complexity:** O(1) (constant time).
4. **isEmpty():**
   - **Purpose:** Checks if the queue contains any elements.
   - **Logic:** Returns true if front is less than 0 (or front > rear in some implementations) or if the queue size is 0. Otherwise, returns false.
   - **Time Complexity:** O(1) (constant time).
5. **isFull():**
   - **Purpose:** Checks if the queue has reached its maximum capacity.
   - **Logic:** Returns true if rear has reached the maximum allowed index. Otherwise, returns false.
   - **Time Complexity:** O(1) (constant time).

Queue Representation (using Arrays):
When implementing a queue using a fixed-size array, two pointers, front and rear, are typically used.

- Initially, front = -1 and rear = -1 (or front = 0, rear = -1 depending on convention).
- **enqueue:** rear = rear + 1; queue[rear] = item; (handle front for first element).
- **dequeue:** item = queue[front]; front = front + 1; (handle empty condition).

Limitations of Linear Array Queue:
A major limitation of a simple linear array-based queue is the **"Unused Space"** or **"Queue Full"** problem, even when there are empty slots at the beginning of the array.

- After several enqueue and dequeue operations, front keeps moving forward.
- Eventually, rear might reach the end of the array, indicating a "full" queue, even if front is far from the beginning, leaving empty spaces at the front of the array that cannot be reused.
- This leads to inefficient memory utilization.

3.2.2) Circular Queue

A **Circular Queue** (also known as a Ring Buffer) is an enhanced version of a linear queue that overcomes the "unused space" problem by treating the array as a circular structure. When the rear pointer reaches the end of the array, it wraps around to the beginning if there are empty slots there.

How it works:

- The last element of the queue is connected to the first element, forming a circle.
- Modulo operator (%) is extensively used to achieve the circular movement of front and rear pointers.



State Management:

- **Initial State:** front = -1, rear = -1 (or front = 0, rear = 0 and one slot is intentionally left empty to distinguish full/empty).
- **Queue Empty Condition:**
  - front == -1 (initial state)
  - front == rear (if one slot is left empty to distinguish full/empty, and front and rear point to the same empty slot).
- **Queue Full Condition:**
  - (rear + 1) % capacity == front (if one slot is left empty to distinguish full/empty).
  - front == 0 && rear == capacity - 1 OR front == rear + 1 (if the array is completely filled).

Circular Queue Operations:

1. **enqueue(item):**
   - **Check for Full:** If (rear + 1) % capacity == front (for the "one slot empty" approach), the queue is full.

- **Insert:**
  - If front == -1 (first element), set front = 0.
  - rear = (rear + 1) % capacity
  - queue[rear] = item
- **Time Complexity:** O(1).

2. **dequeue():**
   - **Check for Empty:** If front == -1, the queue is empty.
   - **Remove:**
     - item = queue[front]
     - If front == rear (only one element was present), reset front = -1, rear = -1.
     - Else, front = (front + 1) % capacity
   - **Time Complexity:** O(1).

Advantages of Circular Queue:

- **Efficient Memory Utilization:** Overcomes the problem of unused space in linear queues by reusing empty slots at the beginning of the array.
- **Improved Performance:** Constant time complexity for enqueue and dequeue operations (O(1)).
- **Simpler Logic for Fixed Size:** Ideal for scenarios where a fixed-size buffer is required, and elements need to be processed in FIFO order.

Disadvantages of Circular Queue:

- **Fixed Size:** Like linear arrays, circular queues implemented with arrays have a fixed maximum size, which must be determined at creation time.
- **Difficulty in Distinguishing Full/Empty:** Requires careful handling of front and rear pointers to distinguish between a full queue and an empty queue (e.g., by leaving one slot empty, or using a count variable).

Applications of Circular Queue:

- **CPU Scheduling:** Managing processes in an operating system (e.g., Round Robin scheduling).
- **Traffic Light Systems:** Sequencing traffic flow.
- **Buffer Management:** Data buffering in applications like media players, network packet handling, and printer spooling.
- **Resource Sharing:** Managing shared resources in a system.

3.2.3) Priority Queue and its Advantages and Applications

A **Priority Queue** is an abstract data type that is similar to a regular queue, but with a crucial difference: each element has an associated **priority**. Elements are dequeued (removed) based on their priority, not strictly on their arrival time.

Key Principle:

- The element with the highest priority is served before the element with lower priority.
- If two elements have the same priority, they are served according to their order in the queue (FIFO).

Implementation Approaches:

Priority queues are typically implemented using:

1. **Heaps (most common and efficient):** Binary heaps (min-heap or max-heap) are the preferred choice due to their efficient O(log n) time complexity for insertion and deletion.
2. **Unordered Array/List:** Simple to implement, but enqueue is O(1) and dequeue (finding the highest priority) is O(n).
3. **Ordered Array/List:** enqueue is O(n) (to maintain order), and dequeue is O(1).
4. **Linked List:** Similar to array implementations in terms of complexity, but more flexible with size.

Priority Queue Operations:

- **insert(item, priority) / enqueue(item, priority):** Adds an element with its priority to the queue.
  - **Time Complexity (Heap-based):** O(log n)
- **extractMax() / extractMin() / dequeue():** Removes and returns the element with the highest (or lowest, depending on implementation) priority.
  - **Time Complexity (Heap-based):** O(log n)
- **peek() / getMax() / getMin():** Returns the element with the highest (or lowest) priority without removing it.
  - **Time Complexity (Heap-based):** O(1)
- **isEmpty():** Checks if the priority queue is empty.
  - **Time Complexity:** O(1)

Advantages of Priority Queue:

- **Prioritized Processing:** Allows for efficient processing of tasks based on their importance, rather than just arrival order.
- **Flexible Ordering:** Can be configured to serve either the highest or lowest priority elements first.
- **Dynamic Prioritization:** Priorities can be changed (though this often involves re-inserting or updating the element, which can be more complex).
- **Optimized Resource Allocation:** Ensures critical tasks or data are handled promptly.

Applications of Priority Queue:

1. **Operating Systems:**
   - **CPU Scheduling:** Processes with higher priority (e.g., interactive tasks) get CPU time before lower priority processes (e.g., batch jobs).
   - **Interrupt Handling:** High-priority interrupts are processed immediately.
   - **Load Balancing:** Distributing tasks to servers based on their load and task priority.
2. **Networking:**
   - **Packet Scheduling:** Routers use priority queues to prioritize network packets (e.g., voice/video data over file transfers) to ensure quality of service (QoS).
   - **Bandwidth Management:** Allocating bandwidth based on application priority.
3. **Graph Algorithms:**
   - **Dijkstra's Shortest Path Algorithm:** Uses a priority queue to efficiently select the next vertex with the smallest known distance.

- **Prim's Algorithm for Minimum Spanning Tree:** Uses a priority queue to select the next edge with the minimum weight.

4. **Event Simulation:**
   - Managing events in a simulation where events need to be processed in chronological order (time as priority).
5. **Artificial Intelligence:**
   - *Best-First Search Algorithms (e.g., A search):** Uses a priority queue to explore the most promising paths first.
6. **Data Compression (Huffman Coding):**
   - Uses a priority queue to build the Huffman tree by repeatedly extracting the two nodes with the lowest frequencies.
7. **Task Management Systems:**
   - Managing a list of tasks where urgent tasks need to be completed first.
8. **Medical Systems:**
   - Prioritizing patient care in emergency rooms based on the severity of their condition.

Python program for Queue :

```
# Simple Queue Program using List

# Create an empty queue
queue = []

# Enqueue (Add elements to the queue)
queue.append('A')
queue.append('B')
queue.append('C')

print("Initial Queue:", queue)

# Dequeue (Remove elements from the queue)
print("Element removed:", queue.pop(0))
print("Queue after removing an element:", queue)

# Enqueue another element
queue.append('D')
print("Queue after adding one more element:", queue)

# Dequeue another element
print("Element removed:", queue.pop(0))
print("Queue after removing another element:", queue)
```

Python program for Circular Queue :

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
```

```
        self.front = self.rear = -1

    def enqueue(self, data):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Full")
            return
        if self.front == -1:  # First element
            self.front = self.rear = 0
        else:
            self.rear = (self.rear + 1) % self.size
        self.queue[self.rear] = data

    def dequeue(self):
        if self.front == -1:
            print("Queue Empty")
            return
        print("Removed:", self.queue[self.front])
        if self.front == self.rear:  # Last element
            self.front = self.rear = -1
        else:
            self.front = (self.front + 1) % self.size

    def display(self):
        if self.front == -1:
            print("Queue Empty")
            return
        i = self.front
        print("Queue:", end=" ")
        while True:
            print(self.queue[i], end=" ")
            if i == self.rear:
                break
            i = (i + 1) % self.size
        print()

# Example usage
q = CircularQueue(3)
q.enqueue(1)
q.enqueue(2)
q.enqueue(3)  # Queue Full on next enqueue
q.display()
q.dequeue()
q.enqueue(4)
q.display()
```

Okay, absolutely! I will update the detailed notes for Queues and Linked Lists, changing all programming code examples from C/C++ to Python. I'll maintain the exact structure, numbering, and level of detail.
Here are the revised notes with Python code examples:

Data Structures and Algorithms: Queues
Unit: Stacks and Queues (or relevant unit where Queues are covered)
Course: BE Computer Engineering / BE Information Technology (as per SPPU curriculum)
Preamble: The Queue is another fundamental linear data structure that adheres to the First-In, First-Out (FIFO) principle. This means the element inserted first is the first one to be removed. Queues are widely used in various computing scenarios, modeling real-world waiting lines.

3.2) Queues
3.2.1) Queue Operations
A Queue is an ordered list in which insertions are done at one end (rear/tail) and deletions are done at the other end (front/head). This behavior is often referred to as FIFO (First-In, First-Out).
Key Terminologies:

- **Front (Head):** The end of the queue from which elements are deleted (dequeued).
- **Rear (Tail):** The end of the queue where new elements are inserted (enqueued).

Basic Queue Operations:

1. **enqueue(item) / insert(item) / add(item):**
    - **Purpose:** Adds an item to the rear of the queue.
    - **Logic:**
        - Check if the queue is full (overflow condition). If full, an error or exception is returned.
        - If not full, add the item to the rear.
        - If implemented with an array, this involves managing the rear pointer.
    - **Time Complexity:** O(1) (constant time).
2. **dequeue() / delete() / remove():**
    - **Purpose:** Removes and returns the element from the front of the queue.
    - **Logic:**
        - Check if the queue is empty (underflow condition). If empty, an error or exception is returned.
        - If not empty, retrieve the element at the front position and remove it.
        - If implemented with an array, this involves managing the front pointer.
    - **Time Complexity:** O(1) (constant time).
3. **peek() / front() / top():**
    - **Purpose:** Returns the element at the front of the queue without removing it.
    - **Logic:**
        - Check if the queue is empty. If empty, an error or exception is returned.
        - If not empty, return the element at the front position.
    - **Time Complexity:** O(1) (constant time).
4. **isEmpty():**
    - **Purpose:** Checks if the queue contains any elements.
    - **Logic:** Returns True if the queue has no elements. Otherwise, returns False.

- **Time Complexity:** O(1) (constant time).

5. **isFull():**
   - **Purpose:** Checks if the queue has reached its maximum capacity.
   - **Logic:** Returns True if the queue is at its maximum size. Otherwise, returns False. (Relevant for fixed-size implementations like arrays).
   - **Time Complexity:** O(1) (constant time).

Queue Representation (using Python Lists - as an array):

Python's built-in list can be used to simulate an array-based queue. For efficient dequeue from the front, collections.deque is generally preferred as list.pop(0) is O(n). However, for demonstrating array-like front/rear pointers, a standard list can be used with index management.

Let's illustrate with a simple array-like implementation to understand front and rear pointers:

Python

```
class Queue:
    def __init__(self, capacity):
        self.queue = [None] * capacity
        self.capacity = capacity
        self.front = -1
        self.rear = -1
        self.size = 0 # To distinguish full/empty if front/rear overlap

    def is_empty(self):
        return self.size == 0

    def is_full(self):
        return self.size == self.capacity

    def enqueue(self, item):
        if self.is_full():
            print("Queue Overflow: Cannot enqueue item, queue is full.")
            return

        if self.is_empty(): # First element being enqueued
            self.front = 0

        self.rear = (self.rear + 1) % self.capacity # Linear queue: self.rear += 1
        self.queue[self.rear] = item
        self.size += 1
        print(f"Enqueued: {item}")

    def dequeue(self):
        if self.is_empty():
            print("Queue Underflow: Cannot dequeue item, queue is empty.")
            return None

        item = self.queue[self.front]
        self.queue[self.front] = None # Optional: Clear the spot
```

```
        self.front = (self.front + 1) % self.capacity # Linear queue: self.front += 1
        self.size -= 1

        if self.is_empty(): # Reset pointers if queue becomes empty
            self.front = -1
            self.rear = -1

        print(f"Dequeued: {item}")
        return item

    def peek(self):
        if self.is_empty():
            print("Queue is empty.")
            return None
        return self.queue[self.front]

    def display(self):
        if self.is_empty():
            print("Queue: []")
            return

        print("Queue elements:")
        # For linear queue, just iterate from front to rear
        # For circular queue, proper iteration needed
        if self.front <= self.rear:
            print(self.queue[self.front : self.rear + 1])
        else: # Wrap around case for circular queue
            print(self.queue[self.front:] + self.queue[:self.rear + 1])


# Example Usage (Linear Queue behavior):
# (Note: This simple array implementation has the "unused space" limitation)
# q = Queue(5)
# q.enqueue(10)
# q.enqueue(20)
# q.enqueue(30)
# q.display() # Queue: [10, 20, 30]
# q.dequeue() # Dequeued: 10
# q.display() # Queue: [20, 30]
# q.enqueue(40)
# q.enqueue(50)
# q.enqueue(60) # Queue Overflow
# q.display() # Queue: [20, 30, 40, 50]
# q.dequeue() # Dequeued: 20
# q.dequeue() # Dequeued: 30
# q.dequeue() # Dequeued: 40
# q.display() # Queue: [50]
```

```
# # Now, despite two empty slots at the beginning, we can't enqueue
# # more if rear reaches capacity-1 in a simple linear implementation.
# # This is why Circular Queue is needed.
```

Limitations of Linear Array Queue:

A major limitation of a simple linear array-based queue is the **"Unused Space"** or **"Queue Full"** problem, even when there are empty slots at the beginning of the array.

- After several enqueue and dequeue operations, front keeps moving forward.
- Eventually, rear might reach the end of the array, indicating a "full" queue, even if front is far from the beginning, leaving empty spaces at the front of the array that cannot be reused.
- This leads to inefficient memory utilization.

3.2.2) Circular Queue

A **Circular Queue** (also known as a Ring Buffer) is an enhanced version of a linear queue that overcomes the "unused space" problem by treating the array as a circular structure. When the rear pointer reaches the end of the array, it wraps around to the beginning if there are empty slots there.

How it works:

- The last element of the queue is connected to the first element, forming a circle.
- Modulo operator (%) is extensively used to achieve the circular movement of front and rear pointers.

State Management:

- **Initial State:** front = -1, rear = -1 (or front = 0, rear = 0 and size = 0 to distinguish full/empty).
- **Queue Empty Condition:**
  - self.size == 0 is the most robust way.
- **Queue Full Condition:**
  - self.size == self.capacity is the most robust way.
  - Alternatively, (self.rear + 1) % self.capacity == self.front (if one slot is left empty to distinguish full/empty when not tracking size).

Circular Queue Operations (using Python List as underlying array):

Python

```
class CircularQueue:
    def __init__(self, capacity):
        self.queue = [None] * capacity
        self.capacity = capacity
        self.front = -1
        self.rear = -1
        self.size = 0 # Crucial for distinguishing full/empty

    def is_empty(self):
        return self.size == 0

    def is_full(self):
        return self.size == self.capacity

    def enqueue(self, item):
        if self.is_full():
```

```
            print("Circular Queue Overflow: Cannot enqueue item, queue is full.")
            return

        if self.is_empty(): # Handle first element insertion
            self.front = 0

        self.rear = (self.rear + 1) % self.capacity
        self.queue[self.rear] = item
        self.size += 1
        print(f"Enqueued: {item}")

    def dequeue(self):
        if self.is_empty():
            print("Circular Queue Underflow: Cannot dequeue item, queue is empty.")
            return None

        item = self.queue[self.front]
        self.queue[self.front] = None # Optional: Clear the spot

        if self.size == 1: # If this was the last element
            self.front = -1
            self.rear = -1
        else:
            self.front = (self.front + 1) % self.capacity

        self.size -= 1
        print(f"Dequeued: {item}")
        return item

    def peek(self):
        if self.is_empty():
            print("Circular Queue is empty.")
            return None
        return self.queue[self.front]

    def display(self):
        if self.is_empty():
            print("Circular Queue: []")
            return

        print("Circular Queue elements:")
        elements = []
        if self.front <= self.rear:
            for i in range(self.front, self.rear + 1):
                elements.append(self.queue[i])
        else:
            for i in range(self.front, self.capacity):
```

```
            elements.append(self.queue[i])
        for i in range(0, self.rear + 1):
            elements.append(self.queue[i])
    print(elements)

# Example Usage:
# cq = CircularQueue(5)
# cq.enqueue(10) # F=0, R=0, S=1 -> [10,N,N,N,N]
# cq.enqueue(20) # F=0, R=1, S=2 -> [10,20,N,N,N]
# cq.enqueue(30) # F=0, R=2, S=3 -> [10,20,30,N,N]
# cq.display() # Circular Queue elements: [10, 20, 30]

# cq.dequeue() # Dequeued: 10. F=1, R=2, S=2 -> [N,20,30,N,N]
# cq.dequeue() # Dequeued: 20. F=2, R=2, S=1 -> [N,N,30,N,N]
# cq.display() # Circular Queue elements: [30]

# cq.enqueue(40) # F=2, R=3, S=2 -> [N,N,30,40,N]
# cq.enqueue(50) # F=2, R=4, S=3 -> [N,N,30,40,50]
# cq.enqueue(60) # F=2, R=0, S=4 -> [60,N,30,40,50] (Wraps around)
# cq.display() # Circular Queue elements: [30, 40, 50, 60] - This display logic needs careful handling for wrap around
# # Correct display for wrap around:
# # Expected internal state: [60, None, 30, 40, 50]
# # front = 2, rear = 0, size = 4
# # Elements are: 30 (idx 2), 40 (idx 3), 50 (idx 4), 60 (idx 0)
# cq.enqueue(70) # Circular Queue Overflow
# cq.display()
```

Advantages of Circular Queue:

- **Efficient Memory Utilization:** Overcomes the problem of unused space in linear queues by reusing empty slots at the beginning of the array.
- **Improved Performance:** Constant time complexity for enqueue and dequeue operations (O(1)).
- **Simpler Logic for Fixed Size:** Ideal for scenarios where a fixed-size buffer is required, and elements need to be processed in FIFO order.

Disadvantages of Circular Queue:

- **Fixed Size:** Like linear arrays, circular queues implemented with arrays have a fixed maximum size, which must be determined at creation time.
- **Difficulty in Distinguishing Full/Empty:** Requires careful handling of front and rear pointers (or using a size counter, as in the example) to distinguish between a full queue and an empty queue.

Applications of Circular Queue:

- **CPU Scheduling:** Managing processes in an operating system (e.g., Round Robin scheduling).
- **Traffic Light Systems:** Sequencing traffic flow.
- **Buffer Management:** Data buffering in applications like media players, network packet handling, and printer spooling.
- **Resource Sharing:** Managing shared resources in a system.

3.2.3) Priority Queue and its Advantages and Applications

A **Priority Queue** is an abstract data type that is similar to a regular queue, but with a crucial difference: each element has an associated **priority**. Elements are dequeued (removed) based on their priority, not strictly on their arrival time.

Key Principle:

- The element with the highest priority is served before the element with lower priority.
- If two elements have the same priority, they are served according to their order in the queue (FIFO).

Implementation Approaches:

Priority queues are typically implemented using:

1. **Heaps (most common and efficient):** Binary heaps (min-heap or max-heap) are the preferred choice due to their efficient O(log n) time complexity for insertion and deletion. Python's heapq module provides a min-heap implementation.
2. **Unordered List:** Simple to implement, but enqueue is O(1) and dequeue (finding the highest priority) is O(n).
3. **Ordered List:** enqueue is O(n) (to maintain order), and dequeue is O(1).
4. **Linked List:** Similar to array implementations in terms of complexity, but more flexible with size.

Python Implementation Example (using heapq for efficiency):

Python's heapq module implements the heap queue algorithm, also known as the priority queue algorithm. It uses a list to represent the heap and provides functions to maintain the heap property. By default, heapq implements a min-heap. To simulate a max-priority queue (where higher numbers mean higher priority), store items as (-priority, item).

Python

```
import heapq

class PriorityQueue:
    def __init__(self):
        self._queue = [] # This list will be maintained as a min-heap
        self._index = 0  # For handling ties and stable sorting (FIFO for same priority)

    def enqueue(self, item, priority):
        # Items are tuples: (-priority, index, item)
        # We negate priority to simulate a max-priority queue with a min-heap
        heapq.heappush(self._queue, (-priority, self._index, item))
        self._index += 1
        print(f"Enqueued: {item} with priority {priority}")

    def dequeue(self):
        if self.is_empty():
            print("Priority Queue is empty. Cannot dequeue.")
            return None
        # Pop the smallest item (which is the highest priority due to negation)
        priority, index, item = heapq.heappop(self._queue)
        print(f"Dequeued: {item}")
        return item
```

```
    def peek(self):
        if self.is_empty():
            print("Priority Queue is empty. No item to peek.")
            return None
        priority, index, item = self._queue[0] # The smallest element is always at index 0 in a
heap
        return item

    def is_empty(self):
        return len(self._queue) == 0

    def size(self):
        return len(self._queue)

# Example Usage:
# pq = PriorityQueue()
# pq.enqueue("Task A", 3)
# pq.enqueue("Task B", 1)
# pq.enqueue("Task C", 5)
# pq.enqueue("Task D", 2)
# pq.enqueue("Task E", 5) # Same priority as Task C

# print(f"Peek: {pq.peek()}") # Peek: Task C

# pq.dequeue() # Should be Task C (priority 5, earlier index)
# pq.dequeue() # Should be Task E (priority 5, later index)
# pq.dequeue() # Should be Task A (priority 3)
# pq.dequeue() # Should be Task D (priority 2)
# pq.dequeue() # Should be Task B (priority 1)
# pq.dequeue() # Empty
```

Advantages of Priority Queue:

- **Prioritized Processing:** Allows for efficient processing of tasks based on their importance, rather than just arrival order.
- **Flexible Ordering:** Can be configured to serve either the highest or lowest priority elements first.
- **Dynamic Prioritization:** Priorities can be changed (though this often involves re-inserting or updating the element, which can be more complex and might take O(n) for direct update or O(log n) for re-insertion after deletion).
- **Optimized Resource Allocation:** Ensures critical tasks or data are handled promptly.

Applications of Priority Queue:

1. **Operating Systems:**
    - **CPU Scheduling:** Processes with higher priority (e.g., interactive tasks) get CPU time before lower priority processes (e.g., batch jobs).
    - **Interrupt Handling:** High-priority interrupts are processed immediately.
    - **Load Balancing:** Distributing tasks to servers based on their load and task priority.

2. **Networking:**
    - **Packet Scheduling:** Routers use priority queues to prioritize network packets (e.g., voice/video data over file transfers) to ensure quality of service (QoS).
    - **Bandwidth Management:** Allocating bandwidth based on application priority.
3. **Graph Algorithms:**
    - **Dijkstra's Shortest Path Algorithm:** Uses a priority queue to efficiently select the next vertex with the smallest known distance.
    - **Prim's Algorithm for Minimum Spanning Tree:** Uses a priority queue to select the next edge with the minimum weight.
4. **Event Simulation:**
    - Managing events in a simulation where events need to be processed in chronological order (time as priority).
5. **Artificial Intelligence:**
    - **Best-First Search Algorithms (e.g., A* search):** Uses a priority queue to explore the most promising paths first.
6. **Data Compression (Huffman Coding):**
    - Uses a priority queue to build the Huffman tree by repeatedly extracting the two nodes with the lowest frequencies.
7. **Task Management Systems:**
    - Managing a list of tasks where urgent tasks need to be completed first.
8. **Medical Systems:**
    - Prioritizing patient care in emergency rooms based on the severity of their condition.

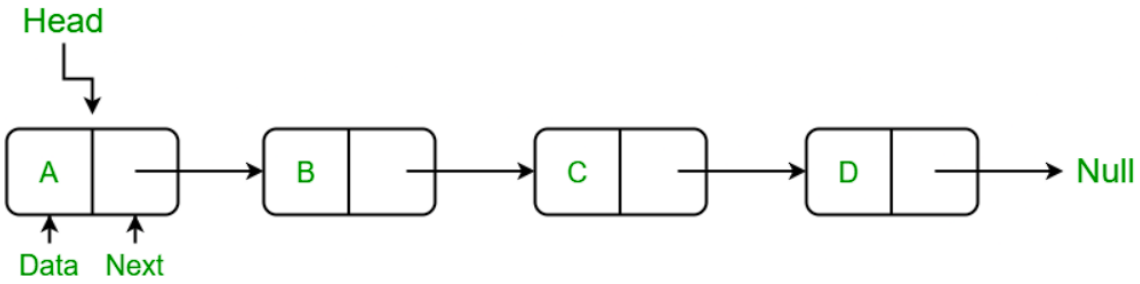
3.3) Linked List

3.3.1) Introduction to Linked Lists

A **Linked List** is a linear data structure where elements are not stored at contiguous memory locations. Instead, each element, called a **node**, contains two parts:

1. **Data Part:** Stores the actual data of the element.
2. **Pointer (or Link) Part:** Stores the address (reference) of the next node in the sequence.

The last node in a linked list points to None in Python, signifying the end of the list. The entire list is typically accessed via a special pointer called the **head** (or start / first) pointer, which points to the first node of the list. If head is None, the list is empty.



Key Characteristics:

- **Dynamic Size:** Linked lists can grow or shrink in size during execution. Memory is allocated only when needed for new nodes.

- **Non-Contiguous Memory Allocation:** Nodes can be stored anywhere in memory, linked together by pointers. This contrasts with arrays, which require contiguous memory.
- **Efficient Insertions/Deletions:** Adding or removing elements in the middle of the list is generally more efficient than in arrays, as it only requires updating a few pointers.
- **Inefficient Random Access:** Accessing an element at a specific index (e.g., the 5th element) requires traversing from the beginning of the list, making random access O(n) unlike O(1) for arrays.

Representation of a Node (Python Class):

Python

```
class Node:
    def __init__(self, data):
        self.data = data      # Data part
        self.next = None      # Pointer to the next node (initially None)
```

3.3.2) Primitive Operations on Linked List - Create, Traverse

Linked lists support a range of operations, each designed to manipulate the list structure or access its data.

1. Create (Initialization and Node Creation):

- **Purpose:** To initialize an empty linked list or to create a new node to be added to the list.
- **Logic:**
  - Initialize the head pointer to None to signify an empty list.
  - When creating a new node, an instance of the Node class is created, and its data and next attributes are set.
- **Time Complexity:** O(1) for creating a single node or initializing the list.

Example (Python - initializing a list and creating a node):

```
class LinkedList:
    def __init__(self):
        self.head = None # Initialize head to None for an empty list

    # Node creation is handled by the Node class itself
    # e.g., new_node = Node(data_value)
```

2. Traverse (Display):

- **Purpose:** To visit each node in the linked list, typically to print its data or perform some operation on each element.
- **Logic:**
  - Start from the head of the list.
  - Use a temporary pointer (e.g., current) initialized to self.head.
  - Iterate as long as current is not None.
  - In each iteration, access the data of the current node.
  - Move current to the next node: current = current.next
- **Time Complexity:** O(n), where 'n' is the number of nodes in the list, as each node must be visited once.

Example (Python):

```
Python
class LinkedList:
    # ... (init and Node class as above)

    def traverse_list(self):
        current = self.head
        if current is None:
            print("Linked List: Empty")
            return

        print("Linked List: ", end="")
        while current is not None:
            print(f"{current.data} -> ", end="")
            current = current.next
        print("None")

# Example Usage:
# my_list = LinkedList()
# my_list.traverse_list() # Linked List: Empty
# # (Nodes will be added in later sections for demonstration)
```

3.3.3) Search

- **Purpose:** To find a specific element (based on its data value) within the linked list.
- **Logic:**
  - Start from the head of the list.
  - Use a temporary pointer (e.g., current) initialized to self.head.
  - Iterate as long as current is not None.
  - In each iteration, compare the data of the current node with the key to be searched.
  - If a match is found, return True (or the current node).
  - If the end of the list is reached (current becomes None) without finding the key, return False (or None).
- **Time Complexity:** O(n) in the worst case (element at the end or not found), O(1) in the best case (element at the head).

Example (Python):

```
class LinkedList:
    # ... (init and Node class as above)

    def search_list(self, key):
        current = self.head
        while current is not None:
            if current.data == key:
                print(f"{key} found in the list.")
                return current # Return the node where key is found
            current = current.next
        print(f"{key} not found in the list.")
```

```
        return None # Key not found
```

3.3.4) Insert

Inserting a new node into a linked list can be done at various positions:

1. Insert at the Beginning (Front/Head):

- **Purpose:** To add a new node as the first node of the list.
- **Logic:**
  - Create a new_node.
  - Set new_node.next = self.head.
  - Update self.head = new_node.
- **Time Complexity:** O(1).

Example (Python):

Python

```
class LinkedList:
    # ... (init and Node class as above)

    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node
        print(f"Inserted {data} at beginning.")
```

2. Insert at the End (Tail):

- **Purpose:** To add a new node as the last node of the list.
- **Logic:**
  - Create a new_node.
  - If the list is empty (self.head is None), set self.head = new_node.
  - Otherwise, traverse the list to find the last node (current.next is None).
  - Set current.next = new_node.
- **Time Complexity:** O(n) in the worst case (traversing the entire list), O(1) if a tail pointer is maintained.

Example (Python):

Python

```
class LinkedList:
    # ... (init and Node class as above)

    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            print(f"Inserted {data} at end (list was empty).")
            return

        current = self.head
        while current.next is not None:
```

```
        current = current.next
    current.next = new_node
    print(f"Inserted {data} at end.")
```

3. Insert at a Specific Position (After a Given Node/Value):

- **Purpose:** To add a new node after a node with a specific value or at a given index.
- **Logic (After a specific value key):**
  - Create a new_node.
  - Search for the node containing key. Let this be prev_node.
  - If prev_node is not found, the insertion cannot occur.
  - Set new_node.next = prev_node.next.
  - Set prev_node.next = new_node.
- **Time Complexity:** O(n) due to the search operation.

Example (Python - after a node with specific key data):

Python

```
class LinkedList:
    # ... (init and Node class as above)

    def insert_after_node(self, key, new_data):
        prev_node = self.search_list(key) # Reuse search method
        if prev_node is None:
            print(f"Node with key {key} not found, cannot insert {new_data}.")
            return

        new_node = Node(new_data)
        new_node.next = prev_node.next
        prev_node.next = new_node
        print(f"Inserted {new_data} after {key}.")
```

3.3.5) Delete

Deleting a node from a linked list also involves managing pointers carefully:

1. Delete from the Beginning (Head):

- **Purpose:** To remove the first node of the list.
- **Logic:**
  - Check if the list is empty (self.head is None).
  - Store the head node in a temporary pointer (e.g., temp).
  - Update self.head = self.head.next.
  - Python's garbage collector will automatically reclaim memory for temp once no references point to it.
- **Time Complexity:** O(1).

Example (Python):

Python

```
class LinkedList:
    # ... (init and Node class as above)
```

```
    def delete_from_beginning(self):
        if self.head is None:
            print("List is empty, nothing to delete.")
            return None # Or raise an exception

        temp = self.head
        self.head = self.head.next
        deleted_data = temp.data
        del temp # Explicitly remove reference, for clarity
        print(f"Deleted {deleted_data} from beginning.")
        return deleted_data
```

2. Delete from the End (Tail):

- **Purpose:** To remove the last node of the list.
- **Logic:**
  - Check if the list is empty or has only one node. Handle these edge cases.
  - Traverse the list using two pointers: current and prev. prev trails current by one node.
  - Stop when current.next is None (meaning current is the last node). At this point, prev points to the second-to-last node.
  - Set prev.next = None.
  - Delete current.
- **Time Complexity:** O(n).

Example (Python):

Python

```
class LinkedList:
    # ... (init and Node class as above)

    def delete_from_end(self):
        if self.head is None:
            print("List is empty, nothing to delete.")
            return None

        if self.head.next is None: # Only one node
            deleted_data = self.head.data
            del self.head
            self.head = None
            print(f"Deleted {deleted_data} (last remaining node).")
            return deleted_data

        current = self.head
        prev = None
        while current.next is not None:
            prev = current
            current = current.next
```

```
    prev.next = None
    deleted_data = current.data
    del current
    print(f"Deleted {deleted_data} from end.")
    return deleted_data
```

3. Delete a Specific Node (by Value):

- **Purpose:** To remove the first occurrence of a node with a specific data value.
- **Logic:**
  - Handle empty list case.
  - If the head node itself contains the key, update head and free the old head.
  - Otherwise, traverse the list using current and prev pointers.
  - Find the node to be deleted (current) and the node before it (prev).
  - Once found, set prev.next = current.next.
  - Delete current.
- **Time Complexity:** O(n) due to traversal.

Example (Python - delete first occurrence of key data):

Python

```
class LinkedList:
    # ... (init and Node class as above)

    def delete_by_key(self, key):
        current = self.head
        prev = None

        # If head node itself holds the key to be deleted
        if current is not None and current.data == key:
            self.head = current.next
            deleted_data = current.data
            del current
            print(f"Deleted {deleted_data} (was head node).")
            return deleted_data

        # Search for the key, keeping track of the previous node
        while current is not None and current.data != key:
            prev = current
            current = current.next

        # If key was not present in linked list
        if current is None:
            print(f"{key} not found in list, nothing to delete.")
            return None

        # Unlink the node from linked list
        prev.next = current.next
        deleted_data = current.data
```

```
        del current
        print(f"Deleted {deleted_data} from list.")
        return deleted_data

# Comprehensive Example for Singly Linked List Operations:
# my_sll = LinkedList()
# my_sll.traverse_list() # Empty

# my_sll.insert_at_end(10) # 10 -> None
# my_sll.insert_at_end(20) # 10 -> 20 -> None
# my_sll.insert_at_beginning(5) # 5 -> 10 -> 20 -> None
# my_sll.insert_at_end(30) # 5 -> 10 -> 20 -> 30 -> None
# my_sll.traverse_list() # Linked List: 5 -> 10 -> 20 -> 30 -> None

# my_sll.insert_after_node(10, 15) # 5 -> 10 -> 15 -> 20 -> 30 -> None
# my_sll.traverse_list()

# my_sll.search_list(20) # 20 found
# my_sll.search_list(99) # 99 not found

# my_sll.delete_from_beginning() # 10 -> 15 -> 20 -> 30 -> None
# my_sll.traverse_list()

# my_sll.delete_from_end() # 10 -> 15 -> 20 -> None
# my_sll.traverse_list()

# my_sll.delete_by_key(15) # 10 -> 20 -> None
# my_sll.traverse_list()

# my_sll.delete_by_key(5) # 5 not found (already deleted)
# my_sll.delete_by_key(10) # 20 -> None
# my_sll.traverse_list()

# my_sll.delete_by_key(20) # Empty
# my_sll.traverse_list()
# my_sll.delete_from_beginning() # Empty
```

3.3.6) Sort

Sorting a linked list can be done using various sorting algorithms. However, directly applying array-based sorts (like Bubble Sort, Selection Sort, Insertion Sort) on linked lists is often inefficient because:

- **Random Access:** Linked lists lack random access, so accessing an element at a specific index is O(n), making comparisons and swaps difficult.
- **Swapping:** Swapping elements involves re-linking pointers, which is more complex than simply swapping values in an array.

Common Sorting Algorithms for Linked Lists:

1. **Bubble Sort (by swapping data):**
    - Simplest to implement.
    - Repeatedly steps through the list, compares adjacent nodes, and swaps their *data* if they are in the wrong order.
    - Less efficient than array Bubble Sort due to traversal overhead.
    - **Time Complexity:** O(n^2).
2. **Insertion Sort (by re-linking nodes):**
    - Builds the sorted list one node at a time.
    - Takes nodes from the unsorted part and inserts them into their correct position in the sorted part.
    - Can be implemented by either swapping data or, more efficiently, by changing links.
    - **Time Complexity:** O(n^2).
3. **Merge Sort:**
    - **Most efficient for linked lists.**
    - A divide-and-conquer algorithm.
    - Divides the list into two halves, recursively sorts them, and then merges the sorted halves.
    - Merging sorted linked lists is straightforward as it only involves changing pointers.
    - **Time Complexity:** O(n log n).
    - **Space Complexity:** O(log n) for recursion stack or O(n) for creating new lists in some implementations.
4. **Quick Sort:**
    - Also a divide-and-conquer algorithm.
    - Selects a pivot, partitions the list around the pivot, and recursively sorts the sub-lists.
    - Partitioning a linked list efficiently is challenging, as swapping nodes requires careful pointer manipulation.
    - **Time Complexity:** Average O(n log n), Worst-case O(n^2).

Example (Python - Merge Sort for Singly Linked List):

Python

```
class LinkedList:
    # ... (init, Node class, traverse_list, etc. methods)

    def _get_middle(self, head):
        if head is None or head.next is None:
            return head

        slow = head
        fast = head
        # Fast pointer moves twice as fast as slow pointer
        # When fast reaches end, slow will be at middle
        while fast.next is not None and fast.next.next is not None:
            slow = slow.next
            fast = fast.next.next
```

```
        return slow

    def _sorted_merge(self, a, b):
        result = None
        # Base cases
        if a is None:
            return b
        if b is None:
            return a

        # Pick either a or b, and recur
        if a.data <= b.data:
            result = a
            result.next = self._sorted_merge(a.next, b)
        else:
            result = b
            result.next = self._sorted_merge(a, b.next)
        return result

    def sort_list_merge_sort(self):
        self.head = self._merge_sort(self.head)
        print("Linked list sorted using Merge Sort.")

    def _merge_sort(self, head):
        # Base case : if head is None or contains only one node
        if head is None or head.next is None:
            return head

        # Get the middle of the list
        middle = self._get_middle(head)
        next_to_middle = middle.next

        # Split the list into two halves
        middle.next = None

        # Recursively sort the two halves
        left_half = self._merge_sort(head)
        right_half = self._merge_sort(next_to_middle)

        # Merge the sorted halves
        sorted_list = self._sorted_merge(left_half, right_half)
        return sorted_list

# Example Usage of Sort:
# my_sll_sort = LinkedList()
# my_sll_sort.insert_at_end(30)
# my_sll_sort.insert_at_end(10)
```

```
# my_sll_sort.insert_at_end(50)
# my_sll_sort.insert_at_end(20)
# my_sll_sort.insert_at_end(40)
# my_sll_sort.traverse_list() # Linked List: 30 -> 10 -> 50 -> 20 -> 40 -> None

# my_sll_sort.sort_list_merge_sort()
# my_sll_sort.traverse_list() # Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> None
```

3.3.7) Concatenate

- **Purpose:** To join two linked lists together to form a single, longer linked list.
- **Logic:**
  - Assume two linked lists, list1 (head head1) and list2 (head head2).
  - Traverse list1 until its last node is reached.
  - Set the next pointer of the last node of list1 to point to the head of list2.
  - The head of the concatenated list remains head1.
- **Edge Cases:**
  - If list1 is empty, the result is list2.
  - If list2 is empty, the result is list1.
- **Time Complexity:** O(n1), where n1 is the number of nodes in list1, as it requires traversing list1 to find its end.

Example (Python):

```
class LinkedList:
    # ... (init, Node class, traverse_list, etc. methods)

    def concatenate_lists(self, other_list_head):
        if self.head is None:
            self.head = other_list_head
            print("Lists concatenated (original list was empty).")
            return

        if other_list_head is None:
            print("Lists concatenated (second list was empty).")
            return

        current = self.head
        while current.next is not None:
            current = current.next

        current.next = other_list_head # Link the end of list1 to the head of list2
        print("Lists concatenated.")

# Example Usage of Concatenate:
# list1 = LinkedList()
# list1.insert_at_end(1)
# list1.insert_at_end(2)
# list1.traverse_list() # Linked List: 1 -> 2 -> None
```

```
# list2 = LinkedList()
# list2.insert_at_end(3)
# list2.insert_at_end(4)
# list2.traverse_list() # Linked List: 3 -> 4 -> None

# list1.concatenate_lists(list2.head)
# list1.traverse_list() # Linked List: 1 -> 2 -> 3 -> 4 -> None

# # Demonstrate with empty list1
# empty_list = LinkedList()
# non_empty_list = LinkedList()
# non_empty_list.insert_at_end(99)
# empty_list.concatenate_lists(non_empty_list.head)
# empty_list.traverse_list() # Linked List: 99 -> None
```

3.3.7) Types of Linked List

Linked lists come in several variations, each offering different advantages for specific applications.

1. Singly Linked List (Linear Linked List):

- **Structure:** Each node contains data and a pointer (next) to the *next* node in the sequence. The last node points to None.
- **Traversal:** Unidirectional (only forward).
- **Operations:**
  - Insertion/Deletion at beginning: O(1)
  - Insertion/Deletion at end/middle: O(n) (requires traversal to find the previous node)
- **Advantages:**
  - Simple to implement.
  - Requires less memory per node (only one pointer).
- **Disadvantages:**
  - Cannot traverse backward.
  - Finding the previous node for deletion or insertion before a specific node requires traversing from the head.
- **Applications:** Implementing stacks, queues, polynomial representation, dynamic memory allocation.

2. Circular Linked List:

- **Structure:** Similar to a singly linked list, but the next pointer of the *last node* points back to the *first node* (head), forming a circle. There is no None pointer at the end.
- **Traversal:** Can traverse indefinitely in a loop.
- **Advantages:**
  - **No None pointer:** Simplifies some operations, as there's no special end-of-list condition to check during traversal (if starting from a known node).
  - **Efficient access to both ends:** If a pointer to the *last node* is maintained, both the first and last nodes can be accessed in O(1) time (last node's next is the first node). This makes operations like inserting at the beginning or end O(1).
- **Disadvantages:**

- Infinite loop potential if traversal logic isn't careful (need a stopping condition, e.g., reaching the starting node again).
- Deletion of the last node is more complex without a prev pointer (still O(n) for singly circular).

- **Applications:**
  - Round-robin CPU scheduling (processes wait in a circle).
  - Music/video playlists (looping playback).
  - Multi-player games (taking turns).
  - Buffer management where continuous cycling is desired.

Python Example (Singly Circular Linked List - simplified):

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None # Head points to the first node
        self.tail = None # Tail points to the last node

    def is_empty(self):
        return self.head is None

    def insert_at_end(self, data):
        new_node = Node(data)
        if self.is_empty():
            self.head = new_node
            self.tail = new_node
            new_node.next = self.head # Points to itself
        else:
            new_node.next = self.head
            self.tail.next = new_node
            self.tail = new_node
        print(f"Inserted {data} into circular list.")

    def traverse_list(self):
        if self.is_empty():
            print("Circular Linked List: Empty")
            return

        print("Circular Linked List: ", end="")
        current = self.head
        while True:
            print(f"{current.data} -> ", end="")
            current = current.next
            if current == self.head: # Stop when we loop back to head
```

```
            break
        print("...") # Indicates circularity

# Example Usage:
# my_cll = CircularLinkedList()
# my_cll.insert_at_end(10)
# my_cll.insert_at_end(20)
# my_cll.insert_at_end(30)
# my_cll.traverse_list() # Circular Linked List: 10 -> 20 -> 30 -> ...
```

3. Doubly Linked List:

- **Structure:** Each node contains data, a pointer to the next node, AND a pointer to the previous node.
- **Traversal:** Bidirectional (can traverse forward and backward).
- **Operations:**
  - Insertion/Deletion at any position (if the node is known or previous node found): O(1)
  - Searching: O(n)
- **Advantages:**
  - **Bidirectional Traversal:** Allows movement in both directions.
  - **Efficient Deletion (of a given node):** If you have a pointer to a node, you can delete it in O(1) time without needing to traverse from the head to find its predecessor.
  - **Efficient Insertion before a node:** Directly accessible.
- **Disadvantages:**
  - **More Memory:** Each node requires an extra pointer, increasing memory overhead.
  - **More Complex Implementation:** Maintaining two pointers (prev and next) makes insertion and deletion logic slightly more intricate due to more pointer updates.
- **Applications:**
  - Implementing LRU (Least Recently Used) cache where elements need to be quickly moved to the front on access.
  - Browser history (forward and backward navigation).
  - Undo/Redo functionality in applications.
  - Representing a deck of cards.

Python Example (Doubly Linked List - simplified):

```
class DoublyNode:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None
```

```
def is_empty(self):
    return self.head is None

def insert_at_beginning(self, data):
    new_node = DoublyNode(data)
    if self.head is not None:
        self.head.prev = new_node
    new_node.next = self.head
    self.head = new_node
    print(f"Inserted {data} at beginning of doubly list.")

def insert_at_end(self, data):
    new_node = DoublyNode(data)
    if self.is_empty():
        self.head = new_node
        print(f"Inserted {data} at end (list was empty).")
        return

    current = self.head
    while current.next is not None:
        current = current.next
    current.next = new_node
    new_node.prev = current
    print(f"Inserted {data} at end of doubly list.")

def traverse_forward(self):
    if self.is_empty():
        print("Doubly Linked List (Forward): Empty")
        return

    print("Doubly Linked List (Forward): ", end="")
    current = self.head
    while current is not None:
        print(f"{current.data} <-> ", end="")
        current = current.next
    print("None")

def traverse_backward(self):
    if self.is_empty():
        print("Doubly Linked List (Backward): Empty")
        return

    print("Doubly Linked List (Backward): ", end="")
    current = self.head
    while current.next is not None: # Go to the last node first
        current = current.next
```

```
        while current is not None:
            print(f"{current.data} <-> ", end="")
            current = current.prev
        print("None")

# Example Usage:
# my_dll = DoublyLinkedList()
# my_dll.insert_at_end(10)
# my_dll.insert_at_end(20)
# my_dll.insert_at_beginning(5)
# my_dll.traverse_forward() # Doubly Linked List (Forward): 5 <-> 10 <-> 20 <-> None
# my_dll.traverse_backward() # Doubly Linked List (Backward): 20 <-> 10 <-> 5 <-> None
```

4. Doubly Circular Linked List:

- **Structure:** Combines the features of both doubly linked lists and circular linked lists. The next pointer of the last node points to the first node, and the prev pointer of the first node points to the last node.
- **Traversal:** Bidirectional and continuous.
- **Advantages:**
  - All advantages of doubly linked lists (bidirectional traversal, efficient deletion of known node).
  - All advantages of circular linked lists (easy access to both ends if one pointer is maintained, no None checks for general traversal).
  - Allows starting traversal from any node and going in any direction.
- **Disadvantages:**
  - Most complex to implement due to four pointer updates for each insertion/deletion.
  - Highest memory overhead per node.
- **Applications:**
  - Similar to doubly linked lists and circular linked lists, but for scenarios requiring both features (e.g., highly dynamic circular buffers where elements can be removed/added from anywhere).

3.4) Case study: Implementation of Stack and Queue operations using Linked lists

1. Introduction

Stacks and Queues are fundamental linear data structures in computer science. While they can be implemented using arrays, a **linked list-based implementation** offers better flexibility by avoiding fixed-size limitations and allowing dynamic memory allocation.

In this case study, we will discuss how to implement:

- Stack using a **Singly Linked List**
- Queue using a **Singly Linked List**

2. Stack Implementation Using Linked List

2.1. Overview

A **Stack** follows the **LIFO (Last In, First Out)** principle.

- **Push:** Add element at the beginning (Head) of the linked list.
- **Pop:** Remove element from the beginning (Head) of the linked list.

2.2. Node Structure

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

2.3. Stack Operations

Push Operation

```
def push(self, data):
    new_node = Node(data)
    new_node.next = self.top
    self.top = new_node
```

Pop Operation

```
def pop(self):
    if self.top is None:
        print("Stack Underflow")
    else:
        print("Popped:", self.top.data)
        self.top = self.top.next
```

Display Operation

```
def  display(self):
    temp = self.top
    while temp:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")
```

2.4. Example Usage

```
class Stack:
    def __init__(self):
        self.top = None

    # Include push, pop, display here (same as above)

s = Stack()
s.push(10)
s.push(20)
```

```
s.display()
s.pop()
s.display()
```

3. Queue Implementation Using Linked List

3.1. Overview

A **Queue** follows the **FIFO (First In, First Out)** principle.

- **Enqueue:** Add element at the end (**rear**) of the linked list.
- **Dequeue:** Remove element from the front (**front/head**) of the linked list.

3.2. Queue Operations

Enqueue Operation

```
def enqueue(self, data):
    new_node = Node(data)
    if self.rear is None:
        self.front = self.rear = new_node
    else:
        self.rear.next = new_node
        self.rear = new_node
```

Dequeue Operation

```
def dequeue(self):
    if self.front is None:
        print("Queue Underflow")
    else:
        print("Dequeued:", self.front.data)
        self.front = self.front.next
        if self.front is None:
            self.rear = None
```

Display Operation

```
def display(self):
    temp = self.front
    while temp:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")
```

3.3. Example Usage

```
class Queue:
    def __init__(self):
        self.front = self.rear = None

    # Include enqueue, dequeue, display here (same as above)

q = Queue()
q.enqueue(10)
```

```
q.enqueue(20)
q.display()
q.dequeue()
q.display()
```

4. Advantages of Using Linked List

Feature	Linked List Implementation	Array Implementation
Size limitation	No (dynamic)	Yes (fixed)
Memory utilization	Efficient	May waste memory
Insert/Delete operations	Fast (no shifting)	Shifting required
Overflow condition	No (until memory full)	Yes (when array full)

5. Real-life Applications

- **Stack using Linked List:**
  - Undo/Redo functionalities in text editors.
  - Backtracking in puzzles.
- **Queue using Linked List:**
  - Print spooling.
  - Task scheduling in operating systems.

UNIT 4 : Hashing

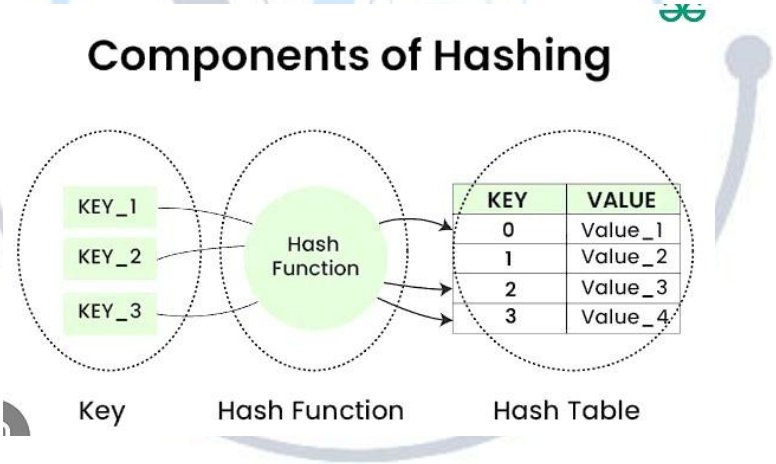
Syllabus

Hash Table : Concepts-hash table, hash function, basic operations, bucket, collision, probe, synonym, overflow, open hashing, closed hashing, perfect hash function, load density, full table, load factor, rehashing, properties of good hash function, Collision resolution strategies- open addressing and chaining, Hash table overflow- open addressing and chaining, extendible hashing, closed addressing and separate chaining

Case study : Dictionary Application using Hash Tables, Description: Implement a dictionary where words and meanings are stored and retrieved using hashing with collision resolution

4.1 Hash Table

A hash table, also known as a hash map or dictionary, is a data structure that stores data in an associative manner, using a "key-value" pair format. It utilizes a hash function to map keys to specific indices (or locations) within an underlying array, allowing for fast data retrieval, insertion, and deletion operations.



4.1.1) Hash Table

- **What it is:** Imagine a special kind of table or a list where you can store information, like names, phone numbers, or product details. But instead of just putting things in order, a Hash Table uses a trick to find things super fast!
- **How it works (the trick):** It uses a special formula (called a "hash function," we'll get to that next) to figure out exactly where each piece of information should go in the table. This means when you want to find something later, you don't have to search through the whole list; you just apply the formula again, and it tells you right where to look!
- **Think of it like:** A library where every book has a specific shelf and spot determined by a formula based on its title. You don't have to wander aimlessly; you calculate the spot and go straight to it.

4.1.2) Hash Function

- **What it is:** This is the "trick" or the special mathematical formula we just talked about! Its job is to take any piece of information (like a name, a number, or a word) and turn it into a smaller, fixed-size number. This number then tells the Hash Table where to store that information.
- **Why it's important:** It's the heart of hashing! Without a hash function, a hash table wouldn't know where to put things or where to find them.
- **Example:** Imagine you have a list of people's names. A simple hash function might take the first letter of the name and assign it a number (A=1, B=2, etc.). So, "Alice" might get stored at location 1, "Bob" at location 2, and so on. (Real hash functions are much more complex, but you get the idea!).

4.1.3) Basic Operations

- **What they are:** These are the main things you do with a Hash Table.
    - **Insertion:** Putting new information into the Hash Table. You use the hash function to figure out where it goes.
    - **Searching:** Finding information that's already in the Hash Table. Again, you use the hash function to quickly pinpoint its location.
    - **Deletion:** Removing information from the Hash Table. You find it using the hash function and then remove it.
- **Why they're fast:** Because of the hash function, these operations are typically very quick, often taking almost the same amount of time no matter how much information is in the table (unlike searching a regular list, which takes longer as the list grows).

4.1.4) Bucket

- **What it is:** Think of a bucket as a specific spot or slot in the Hash Table where data can be stored. It's the place that the hash function points to.
- **Analogy:** In our library example, a "bucket" would be a specific shelf or even a group of shelves where books that hash to a certain location are stored.
- **Capacity:** A bucket can sometimes hold more than one item, especially if multiple items "hash" (get mapped) to the same location (we'll talk about this more with "collision").

4.1.5) Collision

- **What it is:** This is when two or more different pieces of information, when put through the hash function, produce the *same* storage location (the same bucket). It's like two

different books in our library example trying to occupy the exact same spot on the same shelf.

- **Why it happens:** Even with a good hash function, it's very difficult (almost impossible for large amounts of data) to guarantee that every unique piece of information will get its own unique spot. There are simply more possible pieces of information than there are spots in your table.
- **The challenge:** Collisions are a problem because you can't store two different things in the exact same spot. Dealing with collisions is a major part of designing good hashing systems.

4.1.6) Probe

- **What it is:** When a collision happens (two items want the same spot), we can't just give up! A "probe" is the act of looking for the *next available* spot when the original spot (the one the hash function suggested) is already taken.
- **How it works:** It's a strategy to resolve collisions. You try the first spot, if it's full, you "probe" (check) the next spot, then the next, and so on, until you find an empty spot.
- **Think of it like:** You go to your assigned seat in a movie theater, but someone is already sitting there (collision!). You then "probe" by looking for the next empty seat in that row, or the next row, until you find one.

4.1.7) Synonym

- **What it is:** In the context of hashing, "synonyms" are two or more different pieces of information (keys) that, when processed by the hash function, result in the *same* hash value (meaning they want to go into the same initial bucket).
- **Relationship to collision:** Synonyms are the *cause* of collisions. When synonyms occur, a collision *will* happen if you try to insert them into the hash table.
- **Example:** If your hash function simply uses the first letter of a name, then "Alice" and "Arthur" are synonyms because both start with 'A' and would initially hash to the same location.

4.1.8) Overflow

- **What it is:** This happens when a bucket or a certain area in the Hash Table becomes completely full, and there's no more space to put new items, even after trying to "probe" for available spots within that strategy.
- **When it's a problem:** Overflow means you can't insert new data without expanding the table or using a different strategy. It can happen if too many collisions occur in a small area, or if the table is simply too small for the amount of data you're trying to store.

- **Think of it like:** Your library shelf is completely packed, and even after shifting books around, there's absolutely no room for a new one.

4.1.9) Open Hashing (also known as Separate Chaining)

- **What it is:** This is one of the main ways to deal with collisions. Instead of trying to find a *new* spot in the main table when a collision occurs, each bucket in the hash table actually points to a *separate list* (like a linked list or an array) where multiple items can be stored.
- **How it resolves collisions:** When two items hash to the same bucket, they are both simply added to the list associated with that bucket.
- **Analogy:** Imagine each shelf in our library has a small, extra bookcase attached to it. If multiple books want to go on "shelf A," they all just get added to the small bookcase for "shelf A."
- **Advantages:** It's relatively simple to implement, and the table never truly "overflows" in the sense that you can always add more items to the linked lists (memory permitting).
- **Disadvantage:** Searching can become slower if one list gets very long.

4.1.10) Closed Hashing (also known as Open Addressing)

- **What it is:** This is the *other* main way to deal with collisions. Instead of using separate lists, all items are stored directly within the main hash table itself. When a collision occurs, you "probe" (as we discussed) to find an *alternative empty spot* within that *same* main table.
- **How it resolves collisions:** If the original spot is taken, you use a specific "probing sequence" (like linear probing, quadratic probing, etc.) to find the next available slot in the *main table*.
- **Analogy:** You're in a crowded theatre. If your assigned seat is taken, you don't go to a different room; you look for the *next empty seat* in the same theatre.
- **Advantages:** Can have better "cache performance" (meaning data might be retrieved faster by the computer) because all data is in one continuous block of memory. No extra memory overhead for pointers in linked lists.
- **Disadvantages:** Can suffer from "clustering" (where many items bunch up together), which can slow down operations. The table *can* truly fill up, leading to overflow.

4.1.11) Perfect Hash Function

- **What it is:** Imagine the *ideal* hash function. A perfect hash function is one that, for a *specific, fixed set* of data items, maps every single item to a *unique* and *different* location in the hash table.

- **Key Characteristics:**
    - **No Collisions:** This is the most important feature. If you have a perfect hash function, you will *never* have a collision for the data you're currently storing.
    - **One-to-One Mapping:** Each item goes to its own dedicated spot.
    - **Fixed Data Set:** Perfect hash functions are usually designed for situations where the set of data you need to store is known in advance and doesn't change much (e.g., keywords in a programming language, reserved words in a database). If the data set changes often, finding and maintaining a perfect hash function becomes very difficult or impossible.
- **Why it's "Perfect":** Because there are no collisions, searching for an item takes just *one* step – calculate the hash, go to that spot, and the item is there. This is incredibly fast!
- **Practicality:** While theoretically wonderful, perfect hash functions are rarely achievable for large, dynamic (changing) sets of data. They are more common in specialized applications where the data is static.

4.1.12) Load Density

- **What it is:** "Load density" is another term for how "full" your hash table is. It describes the proportion of the hash table's capacity that is currently occupied by data.
- Calculation: It's usually expressed as a percentage or a fraction:

$$\text{Load Density} = \frac{\text{Number of items currently stored in the table}}{\text{Total capacity of the hash table (total number of buckets)}}$$

- **Importance:**
    - **Performance Indicator:** A higher load density generally means more collisions will occur (unless you have a perfect hash function, which is rare). More collisions mean more "probing" or longer linked lists (depending on your collision resolution strategy), which slows down insertion, searching, and deletion.
    - **Trigger for Rehashing:** When the load density reaches a certain threshold (e.g., 70% or 80%), it's often a signal that the table is getting too crowded and needs to be "reshaped" (rehashed) to maintain good performance.

4.1.13) Full Table

- **What it is:** A "full table" specifically refers to a hash table using **Closed Hashing (Open Addressing)** that has run out of available empty slots.
- **How it happens (Closed Hashing):** In closed hashing, every item occupies a slot *within the main table itself*. If you keep adding items, eventually all the slots will be

taken. Even with probing, if all slots are occupied, you literally cannot insert any new item.

- **Contrast with Open Hashing:** In Open Hashing (Separate Chaining), a table technically never becomes "full" in the same way, because each bucket can theoretically hold an infinite number of items in its linked list (limited only by system memory). While the *buckets themselves* might all have a list attached, you can always add more to those lists.
- **Consequence:** If a closed hash table is full, any attempt to insert a new item will fail until some items are deleted or the table is rehashed to a larger size.

4.1.14) Load Factor

- **What it is:** The "load factor" (α) is essentially the same concept as "load density." It's a critical measure of how efficiently a hash table is being used and is primarily used to predict its performance.
- Calculation: It's a ratio:

$$\alpha = \frac{\text{Number of elements (n)}}{\text{Size of the table (m)}}$$

  - n = the number of items (key-value pairs) currently stored in the hash table.
  - m = the total number of available slots (buckets) in the hash table.
- **Impact on Performance:**
  - **Low Load Factor (α<0.5):** The table is relatively empty. Less chance of collisions, so operations are very fast. However, you might be wasting memory by having too many empty slots.
  - **High Load Factor (α>0.7 or 0.8):** The table is getting crowded. The probability of collisions increases significantly. This leads to more time spent resolving collisions (longer probes, longer linked lists), which degrades the performance of insertion, search, and deletion operations.
- **Ideal Range:** For good performance, the load factor is often kept below a certain threshold (e.g., 0.7 or 0.75 for closed hashing; it can be higher for open hashing, sometimes even above 1, as lists can grow). When this threshold is crossed, it triggers **rehashing**.

4.1.15) Rehashing

- **What it is:** "Rehashing" (also known as resizing or rebuilding) is the process of creating a *new, larger* hash table and then taking *all* the existing items from the old table and re-inserting them into the new table using the hash function.

- **When it's done:** Rehashing is typically performed when the hash table's load factor exceeds a predefined threshold. It's a crucial step to maintain good performance as the amount of data grows.
- **Steps:**
    1. **Create a New Table:** Allocate a new hash table, usually with double the size (or some other larger prime number size) of the old table.
    2. **Choose a New Hash Function (Often the Same Type):** While you might use the same *type* of hash function, the hash function often depends on the table size. So, the output values will be different for the new, larger table.
    3. **Re-insert All Elements:** Iterate through every item in the *old* hash table. For each item, calculate its *new* hash value using the (potentially adjusted) hash function and the *new table size*, and then insert it into the *new* table.
    4. **Discard Old Table:** Once all items are moved, the old, smaller table is discarded.
- **Cost:** Rehashing is an expensive operation because you have to re-process every single item. This can cause a temporary pause or slowdown in your application. However, it's necessary to prevent performance degradation over time.
- **Amortized Analysis:** While one rehashing operation is costly, if it's done infrequently (e.g., doubling the size each time), the *average* cost of insertions over a long period remains very efficient.

4.1.16) Properties of a Good Hash Function

- **What it means:** A "good" hash function is one that helps the hash table perform efficiently by minimizing collisions and distributing data evenly. It's not just any formula; it needs to have specific characteristics.
- **Key Properties:**
    1. **Uniform Distribution (Minimizes Collisions):** This is the most crucial property. A good hash function should distribute the keys (data items) as evenly as possible across all the buckets in the hash table. This means that each bucket should have a roughly equal chance of being chosen. If some buckets get chosen much more often, you'll have lots of collisions and slow performance.
    2. **Deterministic:** Given the same input key, the hash function must always produce the *exact same* hash value. If it produced different values, you'd never be able to find the item you stored!
    3. **Fast to Compute:** The hash function itself should be very quick to calculate. If calculating the hash takes a long time, it negates the speed benefits of hashing.
    4. **Utilizes the Entire Key:** A good hash function should use all the information in the input key, not just a part of it (e.g., not just the first letter or last digit). This helps in creating a more unique and varied hash value, reducing collisions.

5. **Sensitive to Small Changes:** Even a tiny change in the input key (like changing one letter in a word) should ideally result in a significantly different hash value. This further helps in distributing keys evenly and avoiding patterns that could lead to many collisions.
6. **Produces a Fixed-Size Output:** The hash value (the number that tells you the bucket index) should always be within the valid range of your hash table size.

4.1.17) Collision Resolution Strategies: Open Addressing and Chaining

As we discussed, a **collision** happens when two different keys, after being processed by the hash function, try to map to the same location (bucket) in the hash table. Since a single bucket can't hold two items simultaneously (unless it's designed to, as in chaining), we need strategies to "resolve" these collisions. The two primary strategies are Open Addressing and Chaining.

A) Open Addressing (also known as Closed Hashing)

- **Core Idea:** In Open Addressing, all data items are stored *directly within* the hash table array itself. When a collision occurs, the algorithm "probes" (searches) for the *next available empty slot* within the same table. It's like finding a new seat in the *same* theater when your assigned one is taken.
- **Probing Sequence:** The way the algorithm searches for the next slot is defined by a "probing sequence." There are several common types:
    1. **Linear Probing:**
        - **How it works:** If the initial hash location h(key) is occupied, it checks h(key) + 1, then h(key) + 2, h(key) + 3, and so on, cyclically (using modulo table size) until an empty spot is found.
        - **Formula:** h(key, i) = (h'(key) + i) % TableSize where h'(key) is the initial hash, i is the probe number (0, 1, 2, ...), and % TableSize ensures it wraps around the table.
        - **Advantages:**
            - Simple to implement.
            - Good **cache performance**: Because it checks adjacent memory locations, it often benefits from CPU caching (data that is physically close together in memory is often loaded into the cache at the same time).
        - **Disadvantages:**
            - **Primary Clustering:** This is the biggest problem. If a collision occurs, nearby slots start filling up. This creates "clusters" of occupied slots. Any new key that hashes into or near a cluster will have to probe through the entire cluster, making insertions and searches very slow for those keys. Imagine a long line forming at a counter.

- Performance degrades significantly as the load factor increases.

2. **Quadratic Probing:**
   - **How it works:** Instead of checking adjacent slots, it checks slots that are farther apart, using a quadratic function of the probe number. If h(key) is occupied, it checks h(key) + 1^2, then h(key) + 2^2, h(key) + 3^2, and so on.
   - **Formula:** h(key, i) = (h'(key) + c1*i + c2*i^2) % TableSize (where c1 and c2 are constants, often c1=0 and c2=1 for simplicity, resulting in h(key, i) = (h'(key) + i^2) % TableSize).
   - **Advantages:**
     - Mitigates **Primary Clustering**: By jumping around, it avoids the long linear sequences of occupied slots.
   - **Disadvantages:**
     - **Secondary Clustering:** While it avoids primary clustering, keys that *initially hash to the same spot* will follow the *exact same probing sequence*. This creates a "secondary cluster" for those specific synonyms, which can still degrade performance.
     - Guaranteed to find an empty slot only if the table is less than half full ($\alpha<0.5$) and the table size is a prime number.
3. **Double Hashing:**
   - **How it works:** This is one of the best open addressing techniques. It uses *two* hash functions. If the first hash function h1(key) results in a collision, a *second* hash function h2(key) is used to determine the *step size* for probing. This step size is unique for each key.
   - **Formula:** h(key, i) = (h1(key) + i * h2(key)) % TableSize
   - **Advantages:**
     - Eliminates both primary and secondary clustering. Since h2(key) is different for different keys, even synonyms follow different probing paths. This leads to much better distribution.
     - Very good performance, close to ideal hashing.
   - **Disadvantages:**
     - More complex to implement because you need two well-chosen hash functions.
     - h2(key) must *never* return zero, and it's best if it's relatively prime to the TableSize to ensure all slots are eventually visited.

B) Chaining (also known as Open Hashing or Separate Chaining)

- **Core Idea:** In chaining, each slot (bucket) in the hash table array does *not* directly store a data item. Instead, each slot stores a *pointer* (or reference) to the head of a **linked list** (or sometimes another data structure like a small array or even a self-balancing binary search tree, especially for very long chains). When multiple keys hash to the same bucket, they are simply added to the linked list at that bucket.
- **How it resolves collisions:** If h(key) maps to a bucket that already has items, the new item is just appended to the linked list at that bucket.
- **Operations:**
    - **Insertion:** Calculate h(key), then insert the new item at the beginning or end of the linked list at hashTable[h(key)]. Before inserting, you might check if the item already exists in the list to avoid duplicates.
    - **Search:** Calculate h(key), then traverse the linked list at hashTable[h(key)] to find the desired item.
    - **Deletion:** Calculate h(key), find the item in the linked list at hashTable[h(key)], and remove it from the list.
- **Advantages:**
    - **Simple to implement.**
    - **Hash table never truly "fills up":** You can always add more elements by extending the linked lists, limited only by available memory. This makes it very flexible, especially when the number of items is unpredictable.
    - Less sensitive to the quality of the hash function or high load factors (performance degrades linearly rather than sharply).
    - Deletion is easier than in open addressing.
- **Disadvantages:**
    - **Requires extra memory:** For the pointers in the linked list nodes.
    - **Cache performance can be worse:** Linked list nodes can be scattered throughout memory, leading to more cache misses compared to open addressing, where data is often more contiguous.
    - **Degraded performance with long chains:** If many keys hash to the same bucket, the linked list for that bucket can become very long. Searching that list then becomes like searching a regular linked list, which is O(N) in the worst case (where N is the number of items in that chain), defeating the purpose of fast hashing.

4.1.18) Hash Table Overflow - Open Addressing and Chaining, Extendible Hashing, Closed Addressing and Separate Chaining

- **Closed Addressing** is another term often used synonymously with **Separate Chaining** (i.e., each bucket stores a pointer to a list of elements).

- **Open Addressing** is the technique where all elements are stored *within* the hash table itself, and collisions are resolved by finding another *open* slot in that same table (as discussed above).
- So, the terms **Open Hashing** and **Separate Chaining** are typically used interchangeably.
- And **Closed Hashing** and **Open Addressing** are typically used interchangeably.

Hash Table Overflow in Open Addressing

- **Definition of Overflow:** In Open Addressing, overflow occurs when the entire hash table array becomes completely full, and there are no more empty slots available to store new elements, even after applying the probing sequence.
- **Cause:** This happens when the number of elements (N) becomes equal to the total size of the hash table (M), i.e., the load factor (α=N/M) reaches 1.0.
- **Consequence:** When an overflow occurs in an open addressing scheme, you *cannot* insert any more elements into the table. Attempts to insert will fail or result in an infinite loop if not handled correctly.
- **Solution:** The primary solution to avoid or deal with overflow in open addressing is **Rehashing**. When the load factor approaches a critical threshold (e.g., 0.7 or 0.8), a larger hash table is allocated, and all existing elements are re-inserted into the new table. This prevents the table from truly "filling up" and ensures continued performance.

Hash Table Overflow in Chaining (Separate Chaining/Closed Addressing)

- **Definition of Overflow:** In Chaining, the hash table itself (the array of pointers to linked lists) technically *never* overflows in the same sense as open addressing. This is because each bucket points to a separate linked list, and linked lists can grow dynamically, limited only by the total available memory of the system.
- **Behavior when "full":** While the array of buckets never literally fills up, if the *load factor becomes very high* (e.g., much greater than 1), it means that on average, each linked list will become very long. This leads to:
    - **Degraded Performance:** Operations like search, insertion, and deletion will require traversing longer linked lists. In the worst case (all keys hash to the same bucket), searching becomes O(N), where N is the total number of elements in the table, effectively turning the hash table into a slow linked list.
    - **Memory Overhead:** A very high load factor in chaining also means more memory is used for the linked list nodes (data + pointer).
- **Solution:** Even though it doesn't "overflow" in the same way, to maintain good performance, hash tables using chaining also employ **Rehashing**. When the load factor (number of elements / number of buckets) exceeds a certain threshold (which can be

higher than open addressing, sometimes even 1.0 or 2.0, depending on implementation), the table is rehashed to a larger size to shorten the average length of the chains.

Extendible Hashing

- **What it is:** Extendible Hashing is a dynamic hashing technique primarily used for databases and file systems, where data is often stored on **disk** (external memory) rather than entirely in main memory. It's designed to handle a very large number of records and minimize disk I/O operations.
- **Key Idea:** Instead of just a single hash table array, it uses a *directory* (which is an array of pointers to data blocks). When a data block (a "bucket" on disk) becomes full, it splits into two, and the directory is adjusted. If the directory itself becomes full, it can double in size, similar to rehashing, but on a more granular level involving disk blocks.
- **How it addresses "overflow":**
    - It doesn't rely on a fixed-size table for data. Instead, it dynamically adds new data blocks as needed.
    - When a block overflows, it splits, and the hash function effectively gets "more specific" for the keys that were in that overflowing block, directing them to the new, split blocks.
    - The goal is to ensure that *at most two disk accesses* are needed for any search, insertion, or deletion operation, even with a massive amount of data. This is crucial because disk I/O is much slower than main memory operations.
- **Difference from traditional hashing:** Traditional hashing usually assumes data is in main memory. Extendible hashing is optimized for situations where data is too large for main memory and needs to reside on external storage.

Case Study: Dictionary Application using Hash Tables

Description: Implement a dictionary application where words (keys) and their corresponding meanings (values) are stored and retrieved efficiently using hashing, specifically incorporating collision resolution strategies.

1. The Core Idea: Why Hash Tables for a Dictionary?

Imagine a traditional dictionary. To find a word, you flip through pages alphabetically. This works, but it's not instant. A Hash Table is like a super-fast, intelligent dictionary that, most of the time, can tell you *exactly* where a word's meaning is stored almost instantly, without flipping through anything.[1]

- **Words as Keys:** Each unique word (e.g., "apple", "banana", "cat") will be our **key**.

- **Meanings as Values:** The definition or meaning of that word will be our **value**.
- **Goal:** We want to quickly add new words/meanings, look up meanings of existing words, and remove words.

2. Hash Table Structure for a Dictionary

Our hash table will be an array (or list) of fixed size, let's call it dictionaryTable. Each position in this array is a **bucket**.

```
dictionaryTable = [
   Bucket 0,
   Bucket 1,
   Bucket 2,
   ...
   Bucket N-1
]
```

Where N is the size of our hash table.

3. The Hash Function for Words

The hash function is crucial. It takes a word (a string) and converts it into a numerical index that tells us which bucket in dictionaryTable to look into.

- **Input:** A word (string).
- **Output:** An index (integer from 0 to N-1).

Example Hash Function Idea (Simple for illustration):

A common way to hash strings is to treat each character as a number (e.g., its ASCII value) and combine them.

```
def hash_word(word, table_size):
   """
   A simple polynomial rolling hash function for strings.
   This aims to distribute words evenly.
   """
   hash_value = 0
   prime = 31  # A common prime number to use in string hashing

   for char in word:
      # Ord() converts character to its ASCII/Unicode integer value
      hash_value = (hash_value * prime + ord(char)) % table_size

   return hash_value
```

```
# Example:
# table_size = 100
# hash_word("apple", 100) -> 97 (example output, actual might differ)
# hash_word("banana", 100) -> 89 (example output)
# hash_word("aple", 100) -> A DIFFERENT number (ideally)
```

This hash_word function takes a word and table_size and returns an index. When we get this index, we go directly to that bucket in dictionaryTable.

4. Collision Resolution Strategies: How We Handle Conflicts

Since different words might produce the same hash value (i.e., they are **synonyms**), we need strategies to handle these **collisions**.

Strategy A: Chaining (Separate Chaining / Open Hashing)

- **Concept:** Each bucket in dictionaryTable doesn't hold a single word-meaning pair directly. Instead, it holds a reference to a **linked list**. If multiple words hash to the same bucket, they are all stored in that bucket's linked list.
- **Structure:**
- dictionaryTable[index] = -> [("word1", "meaning1")] -> [("word2", "meaning2")] -> None
- **Operations:**
    1. **insert(word, meaning):**
        - Calculate index = hash_word(word, table_size).
        - Go to dictionaryTable[index].
        - Traverse the linked list at that index.
        - **Check for Duplicate:** If word already exists in the list, update its meaning.
        - **Insert:** If word is new, create a new (word, meaning) pair and add it to the *end* or *front* of the linked list. (Adding to the front is often simpler).
    2. **search(word):**
        - Calculate index = hash_word(word, table_size).
        - Go to dictionaryTable[index].
        - Traverse the linked list at that index.
        - **Found:** If you find a (word, meaning) pair where the word matches, return its meaning.
        - **Not Found:** If you reach the end of the list without finding the word, it's not in the dictionary.
    3. **delete(word):**
        - Calculate index = hash_word(word, table_size).

- Go to dictionaryTable[index].
- Traverse the linked list at that index.
- **Delete:** When you find the (word, meaning) pair, remove it from the linked list. Handle cases for deleting the first node or a middle/last node carefully.

- **Overflow Handling (Chaining):**
  - A chaining hash table never truly "overflows" in the sense that you can always add more items by extending the linked lists.[3]
  - **Degradation:** However, if too many words hash to the same bucket, its linked list becomes very long. This leads to performance degradation because searching that list becomes O(L) where L is the length of the list, potentially approaching O(N) (total words) in the worst case.
  - **Solution: Rehashing:** When the **load factor** (total_words / table_size) exceeds a certain threshold (e.g., 1.0 or 2.0), we initiate **rehashing**.[4] We create a new, larger dictionaryTable (e.g., double the size), and re-insert *all* existing word-meaning pairs into the new table using the updated hash function (which uses the new larger table size). This shortens the average length of the linked lists.

Strategy B: Open Addressing (Closed Hashing)

- **Concept:** All word-meaning pairs are stored *directly* in the dictionaryTable array itself. If a word hashes to an already occupied bucket (a **collision**), we use a "probing sequence" to find the *next available empty slot* in the same array.[5]

Structure:

```
dictionaryTable = [
  ("wordA", "meaningA"),  // Bucket 0
  None,                   // Bucket 1 (empty)
  ("wordB", "meaningB"), // Bucket 2 (collision resolved by probing)
  ...
]
```

(Where None signifies an empty slot)

- **Operations (Example: Linear Probing)**:
  1. **insert(word, meaning):**
     - Calculate initial_index = hash_word(word, table_size).
     - Start probing from current_index = initial_index.

- **Loop:**
  - If dictionaryTable[current_index] is None (empty) or marked as "deleted" (see below):
    - Insert (word, meaning) into dictionaryTable[current_index]. Break loop.
  - If dictionaryTable[current_index] contains the word already (duplicate):
    - Update its meaning. Break loop.
  - If dictionaryTable[current_index] contains a *different* word (collision):
    - Increment current_index = (current_index + 1) % table_size.
    - **Check for Full Table:** If you loop back to initial_index after trying all slots, the table is full. (This is where proper overflow handling or rehashing is needed *before* this happens).

2. **search(word):**
   - Calculate initial_index = hash_word(word, table_size).
   - Start probing from current_index = initial_index.
   - **Loop:**
     - If dictionaryTable[current_index] is None (empty):
       - The word is *not* in the dictionary. (It would have been inserted here if it existed). Break loop.
     - If dictionaryTable[current_index] contains the word:
       - Return its meaning. Break loop.
     - If dictionaryTable[current_index] contains a *different* word (or a "deleted" marker):
       - Increment current_index = (current_index + 1) % table_size.
       - **Check for Full Loop:** If you loop back to initial_index, the word is not found.

3. **delete(word):**
   - This is trickier in Open Addressing. Simply setting a slot to None can break the search chain for other words that probed *past* that deleted slot.
   - **Solution: Lazy Deletion:** Instead of None, we mark the slot with a special "DELETED" marker.

- Calculate initial_index = hash_word(word, table_size).
- Start probing from current_index = initial_index.
- **Loop:**
  - If dictionaryTable[current_index] is None: Word not found. Break loop.
  - If dictionaryTable[current_index] contains the word:
    - Mark dictionaryTable[current_index] as "DELETED". Break loop.
  - If dictionaryTable[current_index] contains a different word (or "DELETED" marker):
    - Increment current_index = (current_index + 1) % table_size.
    - **Check for Full Loop:** If you loop back to initial_index, word not found.

- **Overflow Handling (Open Addressing):**
  - **True Overflow:** Unlike chaining, an Open Addressing hash table *can* become genuinely full. If total_words == table_size, there are no more empty slots, and further insertions are impossible.
  - **Degradation:** Even before it's completely full, performance degrades rapidly as the **load factor** approaches 1.0 due to **clustering** (especially with linear probing). Long probe sequences make operations slow.
  - **Solution: Rehashing:** When the load factor reaches a critical threshold (e.g., 0.7 or 0.75), **rehashing** is mandatory. A new, much larger dictionaryTable is created, and all existing (non-deleted) word-meaning pairs are re-inserted. This distributes them better and frees up space.

5. Choosing Between Chaining and Open Addressing for a Dictionary

- **Chaining (Separate Chaining):**
  - **Pros:** Simpler deletion, never truly "overflows," performance degrades more gracefully with high load factors.
  - **Cons:** Higher memory overhead due to pointers, potentially worse cache performance, requires separate list implementations.
  - **Good for:** When the number of words is highly unpredictable, and you want simplicity in handling growth.
- **Open Addressing:**

- **Pros:** Better cache performance (data is contiguous), no extra pointer overhead, can be faster when load factor is low.[6]
- **Cons:** Deletion is complex ("lazy deletion" required), prone to clustering, *must* rehash before it becomes completely full.
- **Good for:** When memory efficiency is paramount, and you can predict the approximate number of words to size the table effectively, or when using a good probing strategy like Double Hashing.

SPPU_FE_ENGG Click to join telegram

UNIT 5 : Graphs and Trees

Syllabus

1. **Graphs:** Basic Concepts, Storage representation, Adjacency matrix, adjacency list, Traversals-depth first and breadth first, Minimum spanning Tree, Greedy algorithms for computing minimum spanning tree- Prims and Kruskal Algorithms
2. **Trees:** General tree and its representation: sequential and linked organization, Binary tree-properties, converting tree to binary tree, binary tree traversals (recursive and non-recursive) - inorder, preorder, post order, Operations on binary tree. Binary Search Tree (BST) and its operation
3. **Case study:** GPS/Navigation system that models a city map as a weighted graph and applies core graph algorithms ZIP/GZIP file compression using frequency-based encoding. using Huffman tree

5.1) Graphs :

5.1.1) Basic Concepts

Graph Data Structure is a non-linear data structure consisting of vertices and edges. It is useful in fields such as social network analysis, recommendation systems, and computer networks. In the field of sports data science, graph data structure can be used to analyze and understand the dynamics of team performance and player interactions on the field.

Components of Graph Data Structure

- **Vertices:** Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertex or nodes. Every node/vertex can be labeled or unlabelled.
- **Edges:** Edges are drawn or used to connect two nodes of the graph. It can be ordered pair of nodes in a directed graph. Edges can connect any two nodes in any possible way. There are no rules. Sometimes, edges are also known as arcs. Every edge can be labelled/unlabelled.

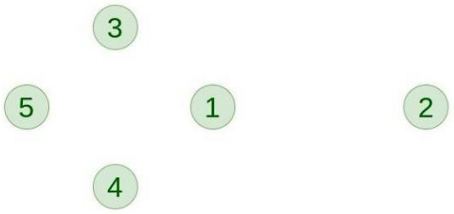
Types Of Graphs in Data Structure and Algorithms

1. Null Graph

A graph is known as a null graph if there are no edges in the graph.

2. Trivial Graph

Graph having only a single vertex, it is also the smallest graph possible.



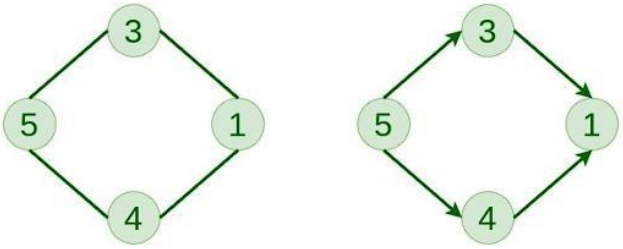
Null Graph Trivial Graph

3. Undirected Graph

A graph in which edges do not have any direction. That is the nodes are unordered pairs in the definition of every edge.

4. Directed Graph

A graph in which edge has direction. That is the nodes are ordered pairs in the definition of every edge.



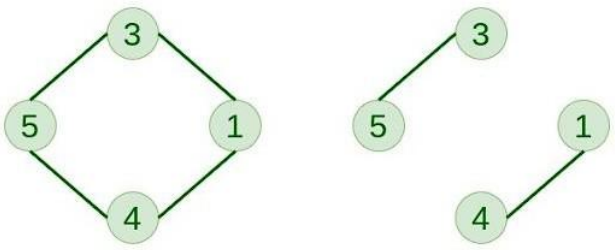
Undirected Graph Directed Graph

5. Connected Graph

The graph in which from one node we can visit any other node in the graph is known as a connected graph.

6. Disconnected Graph

The graph in which at least one node is not reachable from a node is known as a disconnected graph.



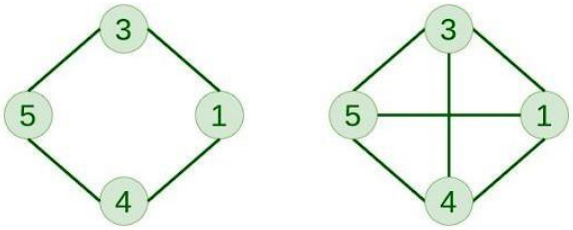
Connected Graph Disconnected Graph

7. Regular Graph

The graph in which the degree of every vertex is equal to K is called K regular graph.

8. Complete Graph

The graph in which from each node there is an edge to each other node.



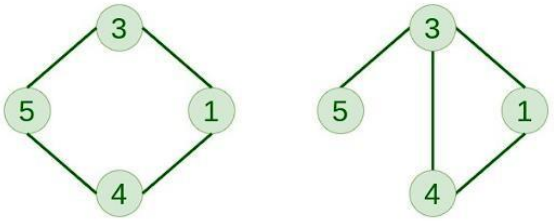
2-Regular Complete Graph

9. Cycle Graph

The graph in which the graph is a cycle in itself, the minimum value of degree of each vertex is 2.

10. Cyclic Graph

A graph containing at least one cycle is known as a Cyclic graph.



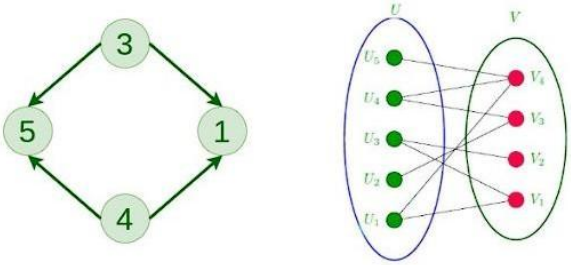
Cycle Graph Cyclic Graph

11. Directed Acyclic Graph

A Directed Graph that does not contain any cycle.

12. Bipartite Graph

A graph in which vertex can be divided into two sets such that vertex in each set does not contain any edge between them.



Directed Acyclic Graph Bipartite Graph

13. Weighted Graph

- A graph in which the edges are already specified with suitable weight is known as a weighted graph.
- Weighted graphs can be further classified as directed weighted graphs and undirected weighted graphs.

5.1.2) Storage Representation

The way a graph is stored in computer memory significantly impacts the efficiency of various graph algorithms. The two primary storage representations are the adjacency matrix and the adjacency list, each with its own advantages and disadvantages.

5.1.3) Adjacency Matrix

An adjacency matrix is a 2D array (or matrix) used to represent a graph.

- **Concept:** For a graph with 'n' vertices, an adjacency matrix A will be an n x n matrix.
  - If there is an edge from vertex i to vertex j, then A[i][j] = 1.
  - If there is no edge from vertex i to vertex j, then A[i][j] = 0.
  - For weighted graphs, A[i][j] can store the weight of the edge instead of 1. If no edge exists, a special value (e.g., 0 or infinity) is used.
  - For undirected graphs, the adjacency matrix is symmetric, meaning A[i][j] = A[j][i].
- **Example (Undirected Graph):** Consider a graph with vertices {0, 1, 2, 3} and edges (0,1), (0,2), (1,2), (2,3).

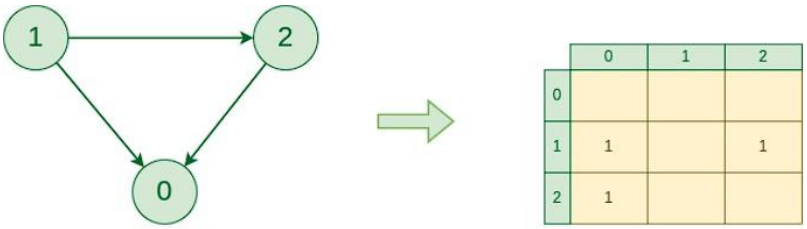
```
    0  1  2  3
0 [[0, 1, 1, 0],
1  [1, 0, 1, 0],
2  [1, 1, 0, 1],
3  [0, 0, 1, 0]]
```

- **Space Complexity:** O(V2), where V is the number of vertices. This can be problematic for sparse graphs (graphs with few edges).

- **Time Complexity:**
  - **Adding/Removing an edge:** O(1)
  - **Checking if an edge exists:** O(1)
  - **Finding all neighbors of a vertex:** O(V) (requires iterating through an entire row/column)

- **Advantages:**
  - Simple to implement.
  - Quick checking for the existence of an edge between two specific vertices.
  - Efficient for dense graphs (graphs with many edges).

- **Disadvantages:**
  - Wastes a lot of space for sparse graphs.
  - Iterating through neighbors is inefficient.

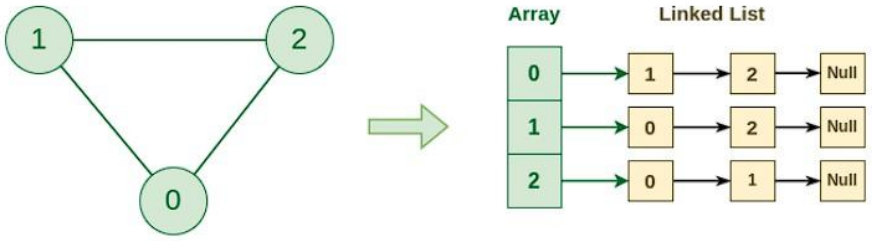


Directed Graph Adjacency Matrix

5.1.4) Adjacency List

An adjacency list is a collection of linked lists or dynamic arrays used to represent a graph.

- **Concept:** For each vertex u in the graph, there is a list containing all vertices v such that there is an edge from u to v.
  - Each element in the list typically stores the destination vertex. For weighted graphs, it might store a pair: (destination vertex, weight).



Undirected Graph Adjacency List

- **Example (Undirected Graph - same as above):** Vertices {0, 1, 2, 3} and edges (0,1), (0,2), (1,2), (2,3).

```
0: [1, 2]
1: [0, 2]
2: [0, 1, 3]
3: [2]
```

- **Space Complexity:** O(V+E), where V is the number of vertices and E is the number of edges. This is more efficient for sparse graphs. For undirected graphs, each edge is stored twice (once for each endpoint), so it's O(V+2E), which simplifies to O(V+E).

- **Time Complexity:**
  - **Adding an edge:** O(1) (assuming appending to the list is O(1) average)
  - **Checking if an edge exists:** O(textdeg(V)), where textdeg(V) is the degree of vertex V. In the worst case, this can be O(V).
  - **Finding all neighbors of a vertex:** O(textdeg(V))

- **Advantages:**
  - Space-efficient for sparse graphs.
  - Efficient for iterating through all neighbors of a vertex.
  - Easy to modify (add/remove vertices and edges).

- **Disadvantages:**
  - Checking for the existence of a specific edge is less efficient than with an adjacency matrix.

5.1.5) Traversals - Depth-First Search (DFS)

Graph traversal is the process of visiting (checking and/or updating) each vertex in a graph. DFS is one such method.

- **Concept:** DFS explores as far as possible along each branch before backtracking. It's like exploring a maze by always going forward until you hit a dead end, then turning back.
- **Algorithm (Recursive):**
  1. Start at an arbitrary source vertex s.
  2. Mark s as visited.
  3. Process s (e.g., print it).
  4. For each unvisited neighbor v of s:
     - Recursively call DFS on v.
- **Algorithm (Iterative using Stack):**
  1. Create an empty stack S.
  2. Push the starting vertex s onto S.
  3. While S is not empty:
     - Pop a vertex u from S.
     - If u has not been visited:
       - Mark u as visited.
       - Process u.
       - For each unvisited neighbor v of u:
         - Push v onto S.
- **Characteristics:**
  - Uses a stack (implicitly in recursion, explicitly in iteration).
  - Goes deep before going wide.
  - Applications: Cycle detection, topological sorting, finding connected components, pathfinding.
- **Time Complexity:** O(V+E) for both adjacency list and adjacency matrix representations (though with adjacency matrix, finding neighbors is O(V) for each vertex, leading to O(V2) in total if not optimized).

5.1.6) Traversals - Breadth-First Search (BFS)

BFS is another fundamental graph traversal algorithm.

- **Concept:** BFS explores all the neighbor nodes at the present depth level before moving on to the nodes at the next depth level. It's like ripples spreading out in a pond.
- **Algorithm (Iterative using Queue):**
  1. Create an empty queue Q.
  2. Enqueue the starting vertex s into Q.
  3. Mark s as visited.
  4. While Q is not empty:
     - Dequeue a vertex u from Q.
     - Process u.

        - For each unvisited neighbor v of u:
            - Mark v as visited.
            - Enqueue v into Q.
- **Characteristics:**
    - Uses a queue.
    - Goes wide before going deep.
    - Applications: Finding the shortest path in an unweighted graph, finding connected components, network broadcasting.
- **Time Complexity:** O(V+E) for both adjacency list and adjacency matrix representations (similar considerations as DFS regarding adjacency matrix efficiency).

5.1.7) Minimum Spanning Tree (MST)

A Minimum Spanning Tree (MST) is a subgraph of an undirected, connected, and weighted graph that connects all the vertices together with the minimum possible total edge weight, and does not contain any cycles.

- **Key Properties:**
    - **Spanning:** It includes all vertices of the original graph.
    - **Tree:** It is acyclic and connected. For a graph with V vertices, an MST will always have V-1 edges.
    - **Minimum:** The sum of the weights of its edges is less than or equal to the sum of the weights of any other spanning tree.
- **Applications:** Network design (e.g., laying cables), cluster analysis, circuit design.
- **Cut Property (or Bridge Property):** For any cut (a partition of the vertices into two non-empty sets), if an edge crosses the cut and has a strictly smaller weight than any other edge crossing the cut, then this edge must be part of *every* MST. If there are multiple edges with the minimum weight crossing the cut, at least one of them must be in some MST. This property is fundamental to both Prim's and Kruskal's algorithms.

5.1.8) Greedy Algorithms for Computing Minimum Spanning Tree

Greedy algorithms make locally optimal choices at each step with the hope of finding a global optimum. Both Prim's and Kruskal's algorithms are greedy and correctly find an MST.

Prim's Algorithm

Prim's algorithm builds an MST by starting from an arbitrary vertex and iteratively adding the cheapest edge that connects a vertex already in the MST to a vertex outside the MST.

- **Concept:** Grows the MST from a starting point. It's similar to Dijkstra's algorithm for shortest paths, but instead of minimizing path cost, it minimizes edge cost to connect a new vertex.
- **Steps:**
    1. Initialize an empty MST.
    2. Choose an arbitrary starting vertex and add it to the MST.
    3. Maintain a set of "candidate edges" – all edges connecting a vertex in the MST to a vertex not yet in the MST.
    4. Repeatedly select the candidate edge with the minimum weight.
    5. Add this edge and its connected (new) vertex to the MST.
    6. Update the set of candidate edges.
    7. Repeat steps 4-6 until all vertices are included in the MST (or V−1 edges have been added).

- **Data Structures for Efficiency:** A min-priority queue (e.g., using a min-heap) is typically used to efficiently retrieve the minimum-weight edge among candidate edges.
- **Time Complexity:**
  - With a binary min-heap: O(ElogV) or O(E+VlogV) if using an adjacency list.
  - With a Fibonacci heap (more complex, theoretical): O(E+VlogV)
- **Advantages:**
  - Simple to understand conceptually.
  - Well-suited for dense graphs.

Kruskal's Algorithm

Kruskal's algorithm builds an MST by sorting all edges in ascending order of their weights and iteratively adding edges if they do not form a cycle with already chosen edges.

- **Concept:** Adds edges greedily, but only if they don't create a cycle. It operates on the edges directly, rather than growing from a single vertex.
- **Steps:**
  1. Create a list of all edges in the graph.
  2. Sort all edges in non-decreasing order of their weights.
  3. Initialize an empty MST.
  4. Initialize a Disjoint Set Union (DSU) data structure (also known as Union-Find) where each vertex is initially in its own set.
  5. Iterate through the sorted edges:
     - For each edge (u,v) with weight w:
       - If u and v are in different sets (i.e., adding this edge does not create a cycle):
         - Add the edge (u,v) to the MST.
         - Union the sets containing u and v.
  6. Stop when V−1 edges have been added to the MST.

- **Data Structures for Efficiency:** The Disjoint Set Union (DSU) data structure with path compression and union by rank/size is crucial for efficient cycle detection and set merging.

- **Advantages:**
  - Conceptually straightforward.
  - Well-suited for sparse graphs.
  - Can be implemented using a simple sort and DSU.

Both Prim's and Kruskal's algorithms are widely used and guaranteed to find an MST. The choice between them often depends on the specific characteristics of the graph (dense vs. sparse) and implementation preferences.

5.2) Trees

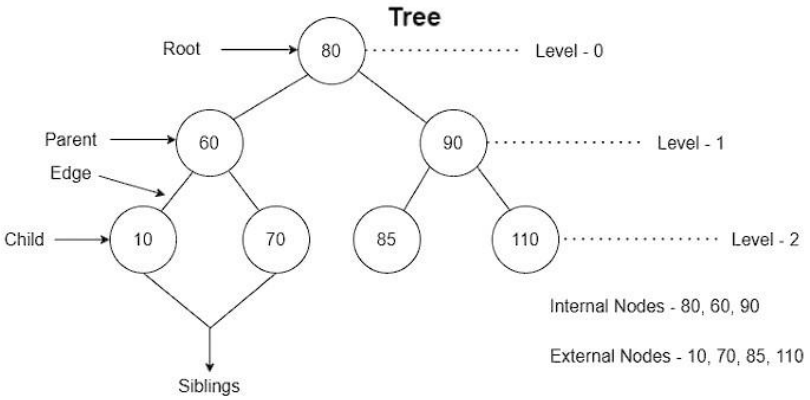
Tree data structure is a hierarchical structure that is used to represent and organize data in the form of parent child relationship. The following are some real-world situations which are naturally a tree.

- Folder structure in an operating system.
- Tag structure in an HTML (root tag the as html tag) or XML document.

The topmost node of the tree is called the **root**, and the nodes below it are called the child nodes. Each node can have multiple child nodes, and these child nodes can also have their own child nodes, forming a recursive structure.

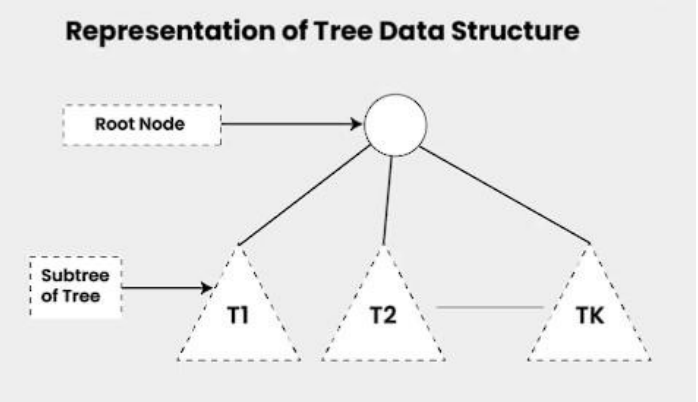
Basic Terminologies In Tree Data Structure:

- **Parent Node:** The node which is an immediate predecessor of a node is called the parent node of that node. **{B}** is the parent node of **{D, E}**.
- **Child Node:** The node which is the immediate successor of a node is called the child node of that node. Examples: **{D, E}** are the child nodes of **{B}.**
- **Root Node:** The topmost node of a tree or the node which does not have any parent node is called the root node. {A} is the root node of the tree. A non-empty tree must contain exactly one root node and exactly one path from the root to all other nodes of the tree.
- **Leaf Node or External Node:** The nodes which do not have any child nodes are called leaf nodes. **{I, J, K, F, G, H}** are the leaf nodes of the tree.
- **Ancestor of a Node:** Any predecessor nodes on the path of the root to that node are called Ancestors of that node. **{A,B}** are the ancestor nodes of the node **{E}**
- **Descendant:** A node x is a descendant of another node y if and only if y is an ancestor of x.
- **Sibling:** Children of the same parent node are called siblings. **{D,E}** are called siblings.
- **Level of a node:** The count of edges on the path from the root node to that node. The root node has level **0**.
- **Internal node:** A node with at least one child is called Internal Node.
- **Neighbour of a Node:** Parent or child nodes of that node are called neighbors of that node.
- **Subtree**: Any node of the tree along with its descendant.



5.2.1) General tree and its representation

A tree consists of a root node, and zero or more subtrees T1, T2, ... , Tk such that there is an edge from the root node of the tree to the root node of each subtree. Subtree of a node X consists of all the nodes which have node X as the ancestor node.



5.2.2) Sequential and Linked Organization of Trees

Trees, like other data structures, can be stored in computer memory using different organizational schemes. The two primary methods are sequential (array-based) and linked (pointer-based) organizations.

Sequential Organization (Array-Based Representation)

- **Concept:** A tree can be stored in a one-dimensional array. This method is particularly suitable for complete or nearly complete binary trees, where nodes can be mapped to array indices systematically.
- **Mapping for Binary Trees:**
  - If a node is at index i:
    - Its left child is at index 2*i + 1.
    - Its right child is at index 2*i + 2.
    - Its parent is at index (i - 1) / 2 (integer division).
  - The root is typically at index 0.
- Example:

Consider a complete binary tree where:

- Root: A (index 0)
- Left child of A: B (index 1)
- Right child of A: C (index 2)
- Left child of B: D (index 3)
- Right child of B: E (index 4)

The array might look like: [A, B, C, D, E, _, _, ...] (where _ represents an empty slot).

- **Space Complexity:** O(2h−1) or O(N) for a complete binary tree, where h is the height and N is the number of nodes. In the worst case (skewed tree), it can be O(2V), where V is the number of nodes, leading to significant wasted space.
- **Advantages:**
  - **Space Efficiency (for complete/full trees):** No need for pointers, reducing memory overhead.
  - **Cache Locality:** Nodes are stored contiguously, which can improve cache performance during traversals.
  - **Easy Navigation:** Parent/child relationships are simple arithmetic calculations.

- **Disadvantages:**

- **Wasted Space (for sparse/skewed trees):** If the tree is not complete, many array slots will remain empty.
- **Insertion/Deletion Difficulty:** Adding or removing nodes (especially internal nodes) can require shifting many elements, which is inefficient (O(N)).
- **Fixed Size:** Typically requires knowing the maximum possible size of the tree beforehand. Dynamic resizing can be costly.

Linked Organization (Pointer-Based Representation)

- **Concept:** Each node in the tree is represented as an object or struct containing:
  - Data (value of the node)
  - Pointers (or references) to its children (e.g., left_child_ptr, right_child_ptr for binary trees).
  - Optionally, a pointer to its parent (parent_ptr).

- **Example (Binary Tree Node Structure):**

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
```

- **Space Complexity:** O(N), where N is the number of nodes. Each node requires memory for its data and pointers.

- **Advantages:**
  - **Dynamic Size:** Trees can grow or shrink easily without pre-allocating large blocks of memory.
  - **Efficient Insertion/Deletion:** Adding or removing nodes only involves updating a few pointers (O(1) after finding the insertion/deletion point).
  - **Space Efficient (for sparse/skewed trees):** Only allocates memory for existing nodes.

- **Disadvantages:**
  - **Memory Overhead:** Pointers consume extra memory for each node.
  - **No Cache Locality:** Nodes are scattered in memory, potentially leading to more cache misses during traversals.
  - **More Complex Navigation:** Requires pointer dereferencing for parent/child relationships.

5.2.3) Binary Tree - Properties

A **binary tree** is a tree data structure in which each node has at most two children, typically referred to as the left child and the right child.

- **Fundamental Properties:**
  - **Root:** The topmost node in the tree. A tree has exactly one root.
  - **Child:** A node directly connected to another node (its parent) one level below it.

- **Parent:** A node that has one or more children directly connected to it one level above.
- **Sibling:** Nodes that share the same parent.
- **Leaf (External Node):** A node that has no children.
- **Internal Node:** A node that has at least one child.
- **Edge:** The link between two nodes.
- **Path:** A sequence of nodes connected by edges from one node to another.
- **Depth of a Node:** The number of edges from the root to that node. The root has a depth of 0.
- **Height of a Node:** The number of edges on the longest path from the node to a leaf. The height of a leaf node is 0.
- **Height of a Tree:** The height of its root node.
- **Level of a Node:** Same as depth + 1 (sometimes, depth is referred to as level).

- **Types of Binary Trees:**
  - **Full Binary Tree (Proper Binary Tree):** Every node has either 0 or 2 children. No node has only one child.
  - **Complete Binary Tree:** All levels are completely filled except possibly the last level, and all nodes at the last level are as far left as possible. This is the type that maps well to array representation.
  - **Perfect Binary Tree:** A binary tree in which all internal nodes have two children and all leaf nodes are at the same depth. It is both full and complete. The number of nodes at depth d is 2d. The total number of nodes is 2h+1−1, where h is the height.
  - **Degenerate (or Skewed) Binary Tree:** Each parent node has only one child. This essentially becomes a linked list.
  - **Balanced Binary Tree:** A tree where the heights of the left and right subtrees of any node differ by at most 1. Examples include AVL trees and Red-Black trees, which are self-balancing.

- **Key Relationships/Formulas:**
  - The maximum number of nodes at level i is 2i.
  - The maximum number of nodes in a binary tree of height h is 2h+1−1.
  - For a non-empty binary tree with N nodes, the minimum height is lfloorlog_2Nrfloor.
  - For a non-empty binary tree with N nodes, the maximum height is N−1 (for a skewed tree).

5.2.4) Converting Tree to Binary Tree (General Tree to Binary Tree)

A "general tree" is a tree where a node can have an arbitrary number of children. Converting a general tree into a binary tree is a common transformation that allows general trees to be processed by algorithms designed for binary trees. The most common method is the **"Left-Child, Right-Sibling" representation**.

- **Concept (Left-Child, Right-Sibling Representation):**
  - Each node in the transformed binary tree will have at most two pointers:
    - The **left pointer** points to its **first (leftmost) child** in the original general tree.

        - The **right pointer** points to its **next sibling** (if any) in the original general tree.
    - The original parent-child relationships are maintained indirectly.
    - The "right child" pointer of the binary tree is used to link siblings in the original tree.
- **Algorithm:**
    1. **For the Root:** The root of the general tree becomes the root of the binary tree. Its left pointer points to its first child. Its right pointer is NULL (as the root has no siblings).
    2. **For any other node N:**
        - Its **left pointer** (in the binary tree) points to its **first child** (in the general tree). If N has no children, its left pointer is NULL.
        - Its **right pointer** (in the binary tree) points to its **next sibling** (in the general tree). If N is the last child of its parent, its right pointer is NULL.

Actual Binary Tree Structure (using left/right pointers):

- A: left points to B, right is NULL
- B: left points to E, right points to C
- C: left is NULL, right points to D
- D: left points to G, right is NULL
- E: left is NULL, right points to F
- F: left is NULL, right is NULL
- G: left is NULL, right is NULL

- **Properties of the Converted Binary Tree:**
    - The height of the resulting binary tree can be greater than the height of the original general tree, especially if there are many siblings.
    - The number of nodes remains the same.
    - This conversion allows algorithms designed for binary trees to operate on general trees.

5.2.5) Binary Tree Traversals (Recursive and Non-Recursive)

Traversing a tree means visiting each node in the tree exactly once. For binary trees, there are three primary traversal methods, each with a specific order of visiting the root, left subtree, and right subtree.

1. In-order Traversal (LNR: Left-Node-Right)

- **Order:** Visit the left subtree, then the root, then the right subtree.
- **Property:** For a Binary Search Tree (BST), in-order traversal visits nodes in non-decreasing (sorted) order.
    - **Recursive Implementation:**

Example

```
def inorder_recursive(node):
    if node:
        inorder_recursive(node.left)  # L
        print(node.data, end=" ")     # N
        inorder_recursive(node.right) # R
```

- **Non-Recursive (Iterative) Implementation (using Stack):**

Example

```
def inorder_iterative(root):
    stack = []
    current = root
    while current or stack:
        # Reach the leftmost node of the current subtree
        while current:
            stack.append(current)
            current = current.left
        current = stack.pop()
        print(current.data, end=" ")
        current = current.right
```

- **Applications:** Sorting (for BSTs), converting a tree into a sorted list.

2. Pre-order Traversal (NLR: Node-Left-Right)

- **Order:** Visit the root, then the left subtree, then the right subtree.
- **Property:** Creates a copy of the tree. Useful for expressing tree structure.
  - **Recursive Implementation:**

Example

```
def preorder_recursive(node):
    if node:
        print(node.data, end=" ")     # N
        preorder_recursive(node.left)  # L
        preorder_recursive(node.right) # R
```

  - **Non-Recursive (Iterative) Implementation (using Stack):**

Example

```
def preorder_iterative(root):
    if not root:
        return
    stack = [root]
    while stack:
        node = stack.pop()
        print(node.data, end=" ")
        # Push right child first so that left child is processed first
        if node.right:
            stack.append(node.right)
        if node.left:
            stack.append(node.left)
```

- **Applications:** Creating a prefix expression (Polish notation) for expression trees, copying trees.

3. Post-order Traversal (LRN: Left-Right-Node)

- **Order:** Visit the left subtree, then the right subtree, then the root.
- **Property:** Processes children before their parent.

- **Recursive Implementation:**

Example

```
def postorder_recursive(node):
    if node:
        postorder_recursive(node.left)  # L
        postorder_recursive(node.right) # R
        print(node.data, end=" ")     # N
```

- Non-Recursive (Iterative) Implementation (using Two Stacks or one Stack with a Visited Flag):

Method 1: Two Stacks

Example

```
def postorder_iterative_two_stacks(root):
    if not root:
        return
    s1 = []
    s2 = []
    s1.append(root)
    while s1:
        node = s1.pop()
        s2.append(node)
        if node.left:
            s1.append(node.left)
        if node.right:
            s1.append(node.right)
    while s2:
        print(s2.pop().data, end=" ")
```

Method 2: One Stack with careful logic (more complex)[1]

Example

```
def postorder_iterative_one_stack(root):
    if not root:
        return
    stack = []
    current = root
    while True:
        while current:
            if current.right:
                stack.append(current.right)
            stack.append(current)
            current = current.left

        current = stack.pop()

        if current.right and stack and stack[-1] == current.right:
            stack.pop() # Remove right child from stack
            stack.append(current) # Push parent back
            current = current.right
```

```
    else:
        print(current.data, end=" ")
        current = None # Finished processing this subtree

    if not stack:
        break
```

- **Applications:** Deleting a tree (deleting children before parent to avoid dereferencing freed memory), evaluating postfix expressions for expression trees.

4. Level-order Traversal (Breadth-First Traversal)

- **Order:** Visits nodes level by level, from left to right.
- **Property:** Uses a queue.
    - **Iterative Implementation (using Queue):**

Example :

```
from collections import deque

def levelorder_traversal(root):
    if not root:
        return
    queue = deque([root])
    while queue:
        node = queue.popleft()
        print(node.data, end=" ")
        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)
```

- **Applications:** Finding the shortest path in unweighted trees, level-by-level processing, building a tree from a serialized format.

Binary Tree Traversal and Operations, and Binary Search Trees

5.2.6) In-order Traversal

In-order traversal is one of the three primary ways to visit nodes in a binary tree. It follows a specific sequence:

- **Order of Visit:**
    1. Recursively traverse the **Left** subtree.
    2. Visit the **Node** (root).
    3. Recursively traverse the **Right** subtree.
- **Abbreviation:** LNR (Left-Node-Right)
- **Process:** Imagine you are walking through the tree. You always try to go left as far as possible. Once you can't go left anymore, you visit the current node, and then you try to go right.

- **Key Characteristic for Binary Search Trees (BSTs):** When an in-order traversal is performed on a Binary Search Tree, the nodes are visited in **non-decreasing (sorted) order** of their values. This is a very important property used for various applications.
- Example:

Consider a simple BST:

```
    4
   / \
  2   5
 / \
1   3
```

In-order traversal: 1, 2, 3, 4, 5

- **Applications:**
  - Retrieving elements of a BST in sorted order.
  - Converting a BST into a sorted list or array.
  - Used in some algorithms for balancing trees.
- **Recursive Implementation:**

Example

```
def inorder_recursive(node):
    if node:
        inorder_recursive(node.left)  # Go left
        print(node.data, end=" ")     # Visit node
        inorder_recursive(node.right) # Go right
```

- **Non-Recursive (Iterative) Implementation using Stack:**

Example

```
def inorder_iterative(root):
    stack = []
    current = root
    while current or stack:
        # Reach the leftmost node of the current subtree
        while current:
            stack.append(current)
            current = current.left

        # Current must be NULL at this point (reached leftmost)
        current = stack.pop()  # Pop the topmost element (leftmost unvisited)
        print(current.data, end=" ") # Visit node

        current = current.right # Now go right
```

5.2.7) Pre-order Traversal

Pre-order traversal is another fundamental way to visit nodes in a binary tree.

- **Order of Visit:**
  1. Visit the **Node** (root).

2. Recursively traverse the **Left** subtree.
3. Recursively traverse the **Right** subtree.

- **Abbreviation:** NLR (Node-Left-Right)
- **Process:** You visit the current node first, then explore its left side completely, and only then move to its right side.

- **Example (same tree as above):**

```
   4
  / \
 2   5
/ \
1   3
```

Pre-order traversal: 4, 2, 1, 3, 5

- **Applications:**
  - Creating a copy of a tree. If you build a new tree by inserting nodes in the order of a pre-order traversal of an original tree, you'll get an identical tree (assuming no duplicates or specific insertion rules).
  - Used to express tree structure, e.g., in prefix expressions (Polish notation) for expression trees.
  - Serialization/Deserialization of a tree.
- **Recursive Implementation:**

Example

```
def preorder_recursive(node):
    if node:
        print(node.data, end=" ")     # Visit node
        preorder_recursive(node.left)  # Go left
        preorder_recursive(node.right) # Go right
```

- **Non-Recursive (Iterative) Implementation using Stack:**

Example

```
def preorder_iterative(root):
    if not root:
        return
    stack = [root] # Start with the root
    while stack:
        node = stack.pop() # Pop the current node
        print(node.data, end=" ") # Visit it

        # Push right child first, so left child is processed first (LIFO)
        if node.right:
            stack.append(node.right)
```

```
    if node.left:
        stack.append(node.left)
```

5.2.8) Post-order Traversal

Post-order traversal visits the child nodes before the parent node.

- **Order of Visit:**
  1. Recursively traverse the **Left** subtree.
  2. Recursively traverse the **Right** subtree.
  3. Visit the **Node** (root).
- **Abbreviation:** LRN (Left-Right-Node)
- **Process:** You explore the left side completely, then the right side completely, and finally, you visit the current node. This means a node is visited only after all its descendants have been visited.
- **Example (same tree as above):**

```
    4
   / \
  2   5
 / \
1   3
```

Post-order traversal: 1, 3, 2, 5, 4

- **Applications:**
  - Deleting a tree: If you delete nodes in post-order, you ensure that you delete children before their parents. This prevents "dangling pointers" and allows for safe memory deallocation, especially in languages like C/C++.
  - Used to evaluate postfix expressions (Reverse Polish Notation) for expression trees.
- **Recursive Implementation:**

Example

```
def postorder_recursive(node):
    if node:
        postorder_recursive(node.left)  # Go left
        postorder_recursive(node.right) # Go right
        print(node.data, end=" ")     # Visit node
```

- Non-Recursive (Iterative) Implementation using Two Stacks:

This is generally simpler to implement than the single-stack approach.

Example

```
def postorder_iterative_two_stacks(root):
    if not root:
        return
    s1 = [] # Main stack for traversal
    s2 = [] # Stack to store the post-order sequence
    s1.append(root)

    while s1:
```

```
    node = s1.pop()
    s2.append(node) # Add node to output stack

    # Push left child first to s1 (so it's popped after right)
    if node.left:
        s1.append(node.left)
    if node.right:
        s1.append(node.right)

  while s2: # Print elements from s2 (which now holds post-order)
    print(s2.pop().data, end=" ")
```

- Non-Recursive (Iterative) Implementation using One Stack (More Complex):

This method requires checking if the right child has already been visited or if the right child is NULL before visiting the current node.

Example

```
def postorder_iterative_one_stack(root):
  if not root:
    return
  stack = []
  current = root
  while True:
    while current:
      # Push right child first if it exists, then current node
      if current.right:
        stack.append(current.right)
      stack.append(current) # Push current node
      current = current.left # Move to left child

    current = stack.pop() # Pop a node

    # If the popped node has a right child and it's the next element in stack,
    # it means the right child hasn't been processed yet.
    # So, re-push current, process right, then process current.
    if current.right and stack and stack[-1] == current.right:
      stack.pop()  # Pop the right child (it was pushed before current)
      stack.append(current) # Push current back
      current = current.right # Go to process right child
    else:
      # No right child or right child already processed, visit current
      print(current.data, end=" ")
      current = None # Indicate that this subtree is processed

    if not stack:
      break # All nodes processed
```

5.2.9) Operations on Binary Tree

Beyond traversals, several common operations are performed on binary trees.

- **Insertion:**
  - **Goal:** Add a new node to the tree while maintaining its structural properties (e.g., balance, completeness, or BST property).
  - **Process:** Typically involves traversing the tree to find the correct insertion point (often a leaf position). For general binary trees, insertion might involve finding an empty spot, while for BSTs, it's based on value comparison.
  - **Complexity:** Depends on tree height. O(h) where h is height. In worst case (skewed tree), O(N). Average case for balanced trees O(logN).
- **Deletion:**
  - **Goal:** Remove a node from the tree while preserving its structure.
  - **Complexity:** Can be complex, especially for internal nodes with two children, as it requires finding a suitable replacement (in-order predecessor or successor) to maintain the tree's properties.[1]
  - **Process:**[2]
    - **Node is a leaf:** Simply remove it.[3]
    - **Node has one child:** Replace the node with its child.[4]
    - **Node has two children:** Fi[5]nd the in-order successor (smallest node in the right subtree) or in-order predecessor (largest node in the left subtree), copy its value to the node to be deleted, and then delete the successor/predecessor (which will have at most one child).
  - **Complexity:** O(h) in the worst case, O(logN) on average for balanced trees.

- **Search/Lookup:**
  - **Goal:** Find if a particular value exists in the tree.
  - **Process:** Start from the root and traverse the tree based on the value. For BSTs, this is very efficient (compare target with node value and go left/right accordingly). For general binary trees, it may involve a traversal (like BFS or DFS) until the element is found.
  - **Complexity:** O(h) in the worst case, O(logN) on average for balanced trees.

- **Finding Minimum/Maximum (for BSTs):**
  - **Minimum:** Keep traversing left from the root until you reach a node with no left child.
  - **Maximum:** Keep traversing right from the root until you reach a node with no right child.
  - **Complexity:** O(h) in the worst case, O(logN) on average for balanced trees.

- **Height/Depth Calculation:**
  - **Height:** The length of the longest path from the node to a leaf. Often calculated recursively: max(height(left_subtree), height(right_subtree)) + 1.
  - **Depth:** The distance from the root to the node.

- **Counting Nodes/Leaves:** Traversing the tree and incrementing a counter.

5.2.10) Binary Search Tree (BST) and its Operations

A **Binary Search Tree (BST)** is a special type of binary tree that adheres to a specific ordering property, making search, insertion, and deletion operations very efficient on average.

- **BST Property:** For every node in the tree:
  - All values in its **left subtree** are **less than** the value of the node itself.
  - All values in its **right subtree** are **greater than** the value of the node itself.
  - Both the left and right subtrees must also be BSTs.
  - **No duplicate values** are typically allowed in a standard BST. If duplicates are allowed, they are usually placed consistently on either the left or right side (e.g., all duplicates go to the right subtree).

- **Advantages of BSTs:**
  - Efficient searching, insertion, and deletion operations on average (O(logN)).
  - Nodes can be retrieved in sorted order using in-order traversal.

- **Disadvantages of BSTs:**
  - Worst-case performance can be O(N) if the tree becomes skewed (degenerate), resembling a linked list. This happens when elements are inserted in strictly increasing or decreasing order.
  - This led to the development of **self-balancing BSTs** like AVL trees and Red-Black trees, which automatically adjust their structure to maintain logarithmic height.

Operations on Binary Search Tree (BST)

The fundamental operations in a BST leverage its ordering property.

- **1. Search (Lookup):**
  - **Goal:** Determine if a specific key exists in the BST.
  - **Process:**
    1. Start at the root.
    2. Compare the key with the current node's value.
    3. If key == current node's value, found! Return True.
    4. If key < current node's value, move to the **left child**.
    5. If key > current node's value, move to the **right child**.
    6. If current node becomes NULL (reach an empty subtree), the key is not in the tree.
  - **Complexity:** O(h) where h is the height of the tree. Best case O(1) (root), worst case O(N) (skewed), average case O(logN).

- **2. Insertion:**
  - **Goal:** Add a new value into the BST while maintaining its property.
  - **Process:**
    1. Start at the root.
    2. Traverse the tree similar to search, comparing the new value with the current node's value.
    3. If value < current node's value, go left.

4. If value > current node's value, go right.
5. Continue until you reach a NULL pointer (an empty spot). This is where the new node should be inserted as a new leaf.
6. If value == current node's value (and duplicates are not allowed), do nothing or handle as an error. If duplicates are allowed, place it consistently (e.g., to the right).

- **Complexity:** O(h) in the worst case, O(N) average case O(logN).

- **3. Deletion:**
  - **Goal:** Remove a node with a specific key from the BST. This is the most complex BST operation.
  - **Process (three cases):**
    1. **Case 1: Node to be deleted is a Leaf Node (no children).**
       - Simply remove the node by setting its parent's corresponding child pointer to NULL.
    2. **Case 2: Node to be deleted has One Child.**
       - Replace the node with its single child. The parent of the deleted node will now point to the child of the deleted node.
    3. **Case 3: Node to be deleted has Two Children.**
       - This is the trickiest case. To maintain the BST property, the deleted node must be replaced by either its **in-order predecessor** (the largest value in its left subtree) or its **in-order successor** (the smallest value in its right subtree).
       - **Standard approach:** Find the in-order successor (the leftmost node in the right subtree). Copy the successor's value to the node being deleted. Then, recursively delete the successor from the right subtree (which will fall into Case 1 or Case 2, as the successor cannot have a left child).
  - **Complexity:** O(h) in the worst case, O(N) average case O(logN).

- **4. Find Minimum:**
  - **Goal:** Find the smallest value in the BST.
  - **Process:** Start at the root and keep traversing to the left child until you reach a node whose left child is NULL. This node contains the minimum value.
  - **Complexity:** O(h) in the worst case, O(N) average case O(logN).

- **5. Find Maximum:**
  - **Goal:** Find the largest value in the BST.
  - **Process:** Start at the root and keep traversing to the right child until you reach a node whose right child is NULL. This node contains the maximum value.
  - **Complexity:** O(h) in the worst case, O(N) average case O(logN).

These operations make BSTs highly effective for managing ordered data where quick lookups, insertions, and deletions are essential.

5.3) Case study: GPS/Navigation system that models a city map as a weighted graph and applies core graph algorithms ZIP/GZIP file compression using frequency-based encoding. using Huffman tree

Case Study 1: GPS/Navigation System Modeling a City Map

Concept: A GPS (Global Positioning System) or navigation system fundamentally relies on representing a map as a graph. This allows it to compute optimal routes, identify points of interest, and provide real-time guidance.

5.1.2) Storage Representation (How the Map is Stored):

- **Vertices (Nodes):**
  - **Representation:** Each intersection, major turn, or significant landmark in the city can be represented as a **vertex** (or node) in the graph.
  - **Attributes:** Each vertex would store its geographical coordinates (latitude, longitude), possibly a unique ID, and any associated information (e.g., name of the intersection).
- **Edges (Connections):**
  - **Representation:** The roads or segments connecting these intersections are represented as **edges** between vertices.
  - **Attributes (Weights):** Edges are typically **weighted** to represent various factors that influence travel. Common weights include:
    - **Distance:** The physical length of the road segment.
    - **Time:** The estimated travel time, which can be dynamic (influenced by traffic, speed limits, road type).
    - **Cost:** Tolls, fuel consumption, or other factors.
    - **Direction:** For one-way streets, edges would be directed (e.g., from A to B, but not B to A).
  - **Directed vs. Undirected:** Roads can be either undirected (two-way streets, allowing travel in both directions) or directed (one-way streets).
- **Graph Type:** The city map is primarily modeled as a **weighted, directed graph** (or a mix of directed and undirected, which can be represented as two directed edges for each two-way street).

5.1.3) Adjacency Matrix vs. 5.1.4) Adjacency List for Map Storage:

The choice of storage representation for a city map is critical for performance.

- **Adjacency Matrix:**
  - **Pros:** O(1) to check if a direct road exists between two intersections.
  - **Cons:** A city map is a **sparse graph** (many intersections, but each intersection only connects to a few others). An adjacency matrix would be O(V2) in space, leading to enormous memory waste for large cities (millions of intersections). It would also be inefficient to iterate through all possible connections to find neighbors.
- **Adjacency List:**
  - **Pros: Much more efficient for sparse graphs.** Space complexity is O(V+E), which is significantly less than O(V2) for typical city maps. Iterating through neighbors of an intersection is efficient, as you only traverse its actual connections.

- **Cons:** Checking for a direct road between two *arbitrary* intersections might be O(degree of vertex), which is slower than O(1) for matrix. However, this specific operation is less frequent than finding all neighbors.
- **Conclusion: Adjacency List is the preferred and widely used representation for city maps in navigation systems** due to its memory efficiency and suitability for graph traversal algorithms. Each intersection (vertex) would have a list of its connected roads (edges), along with their weights (distance, time).

5.1.5 & 5.1.6) Traversals (Depth-First Search & Breadth-First Search) in Navigation:
While not directly used for *finding optimal routes* (which require weighted shortest path algorithms), DFS and BFS have conceptual relevance or can be used for other map-related tasks:

- **Breadth-First Search (BFS):**
  - **Application:** Finding the **shortest path in terms of number of segments (unweighted path)**. For example, "What's the fewest number of turns to get from A to B?" or "Find all intersections reachable within N turns."
  - **Process:** Explores all roads from a given intersection before moving to the next set of intersections. Uses a queue.
  - **Relevance:** Less common for core navigation (which cares about distance/time) but useful for specific queries.
- **Depth-First Search (DFS):**
  - **Application:**
    - **Finding all reachable locations:** "Can I get from this point to that point at all?" (connectivity check).
    - **Detecting one-way loops or dead ends:** Identifying cycles in the road network that might lead back to the start without alternatives (though not typically an issue for navigation, more for map validation).
    - **Topological sorting (if applicable to road segments):** Useful for dependencies (e.g., if you have to visit certain points in order).
  - **Process:** Explores one path as deeply as possible before backtracking. Uses a stack (or recursion).
  - **Relevance:** More for analytical tasks on the map data itself rather than live navigation.

Core Graph Algorithms for Navigation (Shortest Path):
The heart of a GPS system lies in shortest path algorithms.

- **Dijkstra's Algorithm:**
  - **Purpose:** Finds the **shortest path from a single source vertex to all other vertices** in a graph with non-negative edge weights (distances, times).
  - **Application in GPS:** Given a starting location, Dijkstra's algorithm can find the fastest (or shortest) route to a specific destination. It is the fundamental algorithm for route planning.
  - **How it Works:** It maintains a set of visited vertices and an array storing the shortest known distance to each vertex. It iteratively selects the unvisited vertex with the smallest known distance, marks it visited, and updates the distances of its neighbors. A min-priority queue is used for efficient selection.
  - **Relevance:** The primary algorithm for "get me from A to B".

- *A Search Algorithm:**
    - **Purpose:** An extension of Dijkstra's that is more efficient for single-source, single-destination shortest path problems, especially in large graphs like city maps. It finds the shortest path while being "informed" about the direction of the goal.
    - **Application in GPS:** Widely used in practical navigation systems due to its speed.
    - **How it Works:** Similar to Dijkstra's but uses a heuristic function (an estimated cost from the current node to the destination) in addition to the actual cost from the source. The heuristic guides the search towards the destination, reducing the number of nodes explored. A common heuristic for maps is the straight-line distance (Euclidean distance) to the destination.
    - **Relevance:** The workhorse for real-time, efficient route computation.
- **Bellman-Ford Algorithm:**
    - **Purpose:** Finds the shortest paths from a single source vertex to all other vertices, **even with negative edge weights**. Also detects negative cycles.
    - **Relevance in GPS:** City maps typically don't have "negative time" or "negative distance" roads, so Bellman-Ford is generally **not used** for standard GPS routing. It's more applicable in scenarios with financial costs or other abstract weights.

Real-time Traffic and Dynamic Weights:
Modern GPS systems dynamically update edge weights (travel times) based on real-time traffic data, accidents, and road closures. This turns the static graph problem into a dynamic one, requiring algorithms to re-compute or update routes on the fly. This often involves re-running A* or Dijkstra from the current location.

Case Study 2: ZIP/GZIP File Compression using Frequency-Based Encoding (Huffman Tree)

Concept: ZIP and GZIP are popular file compression formats. A core component of their compression strategy is **Huffman coding**, a frequency-based encoding technique that uses a special type of binary tree called a Huffman tree.

Core Idea of Huffman Coding:

- Assign shorter codes (bit sequences) to frequently occurring characters/symbols.
- Assign longer codes to less frequently occurring characters/symbols.
- This results in a smaller total file size compared to fixed-length encoding (e.g., ASCII, where every character takes 8 bits).

The Huffman Tree (5.2.3 Binary Tree Properties Applied):

- **Concept:** A Huffman tree is a **full binary tree** (every internal node has two children) specially constructed to represent the codes.
- **Properties:**
    - **Leaves:** Each leaf node in the Huffman tree represents a unique character/symbol from the input data. The value of the leaf node is its frequency in the input.
    - **Internal Nodes:** Each internal node represents the sum of the frequencies of its children. It does not correspond to a character itself.

- **Path to Leaf = Code:** The path from the root to a leaf node defines the binary code for that character. Typically, a left branch is assigned '0' and a right branch is assigned '1' (or vice-versa).
- **Prefix Code Property:** No character's code is a prefix of another character's code. This is crucial for unambiguous decoding. When you traverse the tree, as soon as you reach a leaf, you know you have the complete code for a character; you don't need to look further down the path.

5.3.4) Converting Tree to Binary Tree (Conceptual Link):

While a Huffman tree is inherently a binary tree, the "converting tree to binary tree" concept (Left-Child, Right-Sibling) is not directly applied during its construction or use. Instead, the Huffman algorithm directly builds a binary tree structure. The "tree" it starts with is a forest of single-node trees (each character as a tree of one node).

The Huffman Algorithm (Greedy Algorithm for Tree Construction):

This is a **greedy algorithm** that constructs the optimal prefix code.

1. **Calculate Frequencies:** Count the frequency of each unique character in the input file.
2. **Create Leaf Nodes:** Create a forest of single-node trees, one for each unique character, with the character's frequency as its value. Store these in a min-priority queue (ordered by frequency).
3. **Build Tree (Iterative Process):**
   - While there is more than one tree in the forest (i.e., while the priority queue has more than one element):
     - Extract the **two trees with the smallest frequencies** from the priority queue.
     - Create a **new internal node** whose frequency is the sum of the frequencies of the two extracted trees.
     - Make the two extracted trees the left and right children of this new internal node.
     - Insert the new internal node back into the priority queue.
4. **Final Tree:** The last remaining tree in the priority queue is the Huffman tree.

5.3.5) Binary Tree Traversals in Huffman Coding:

- **Encoding (Pre-order or DFS-like Traversal for Code Generation):**
  - Once the Huffman tree is built, you can traverse it (e.g., using a DFS-like approach) to determine the codes for each character.
  - Start from the root. Assign '0' to the left branch and '1' to the right branch. As you traverse, concatenate the '0's and '1's along the path. When you reach a leaf node (a character), the concatenated string is its Huffman code. This is effectively a pre-order traversal or a general DFS exploration.

- **Decoding (Path Traversal from Root):**
  - To decode a compressed bitstream:
    - Start at the root of the Huffman tree.
    - Read the bitstream one bit at a time.
    - If the bit is '0', move to the left child.
    - If the bit is '1', move to the right child.

- When you reach a leaf node, you have found a character. Append this character to the decoded output.
- Then, restart from the root of the Huffman tree for the next character. This is a path traversal dictated by the bitstream.

ZIP/GZIP Implementation:

- **Frequency Analysis:** The compression algorithm first analyzes the input data to determine character frequencies.
- **Huffman Tree Construction:** A Huffman tree is built based on these frequencies.
- **Encoding:** The original data is then traversed, and each character is replaced by its corresponding Huffman code (a bit sequence).
- **Header Information:** The Huffman tree structure (or the frequency table to reconstruct it) must be stored in the compressed file's header so that the decompressor knows how to decode the data. This is why a Huffman tree itself isn't stored as a general tree converted to a binary tree; its structure is derived from frequencies, and the rules for that derivation are implicitly understood by the compression/decompression algorithms.

Summary for Compression: Huffman coding is a prime example of a greedy algorithm utilizing a specific type of binary tree (the Huffman tree) to achieve optimal prefix coding for data compression. The construction of the tree and the subsequent encoding/decoding processes rely heavily on the properties and traversals of binary trees.

Vinay Bhadane omegaofts.news